融合多源预测的微网经济调度与线性优化研究

摘　要

随着我国碳达峰、碳中和目标的推进和分布式新能源的发展，光伏与储能逐步成为社区供能的重要组成。如何在供需和电价变化下合理购电，既避免高价紧急补购，又减少多订电量的损失，是微网运行的关键。本文以 10 分钟为基本时段，以电量平衡和储能约束为共同基础，逐步引入历史预测、日内预报和电价预测，形成由确定性调度向动态调整递进的购电策略。

针对问题一，采用**电量流线性规划**，统筹全天购电及储能充放电，在保持日初、日末储电量相等的条件下，将低价购电和富余光伏留到更有价值的时段使用。当天最低购电费为 **35126.85 元**，较峰谷策略降低 **8.23%**，体现了供需与储能联合安排的节费优势。

针对问题二，将**历史分项预测、近期加权准备与跨日线性规划**相结合。负载预测利用星期规律，光伏预测参考近期发电水平，再根据近期缺口误差设置购电准备，并通过 48 小时规划衔接次日用电。2025 年 2—12 月总费用为 **1358.38 万元**，较等权误差准备节省 **44.66 万元**。结果表明，适量增加普通购电准备能够减少更昂贵的紧急补购，预测与调度的配合比单独追求预测精度更贴合本题目标。

针对问题三，运用**融合预测与日内校正滚动规划**，将官方光伏预报与近期历史结合，利用当天已发生的负载偏差修正后续判断，并在 6、12、18 点权衡调整费与补购费，更新剩余订单。同期总费用为 **1301.42 万元**。同一模型下的信息对照表明，日内反馈与改单节省 **50.63 万元**；在此基础上，后发官方预报再节省 **6.95 万元**，并使紧急购电量下降 **30.10%**。两组期末库存相同，说明日内新预报能够带来额外节费，并减少对紧急补购的依赖。

针对问题四，在未来真实电价未知的条件下，零点分支使用**提升树预测与线性规划**，利用历史价格和供需信息刻画价格变化；日内分支选取**岭回归预测与滚动线性规划**，在候选比较中以接近最低的费用获得较少的紧急购电，并便于解释价格变化的影响。两套完整策略同期费用分别为 **1424.86 万元、1369.59 万元**，日内策略节省 **3.88%**，体现了供需判断、价格预测与订单调整的综合收益。

逐 10 分钟复核表明，各方案满足供需平衡和储能设备约束，费用可由实际订单逐项复算。参数比较揭示了提前准备与高价补购之间的费用权衡，信息对照进一步区分了反馈调整和新增预报的贡献。本文以**实际购电费用**连接预测与调度，为社区微网兼顾用能经济性与可再生能源利用提供了可核验的方案。

关键词：微网调度；线性规划；误差准备；滚动优化；电价预测

一、问题重述

1.1 问题背景

微网是一种将分布式发电、储能设备和用电负载进行统一协调的小型供电系统，能够提高本地能源的利用效率，并减少对外部电网的依赖。如图??所示，微网中的电力来自光伏发电和外部电网。光伏发出的电可以优先满足小区负载；当光伏有剩余时，可以将多余电量存入储能设备；当光伏不足时，则可以由储能设备放电或从外部电网购电来补充。

外网电价较低时，微网可以提前购电并存入储能设备，在之后电价较高或供电不足时使用。因此，微网每天都需要解决一个调度问题：**各时段应该购买多少电，储能设备何时充电、何时放电。**

买电过多会增加费用，买电过少又可能在实际运行中产生高价补购；同时，储能设备还受到容量、充放电功率和充放电效率等条件的限制。因此，需要结合供需预测、储能状态和电价，协调购电与充放电，在保证小区正常供电和储能安全运行的前提下，使总购电费用尽可能低。

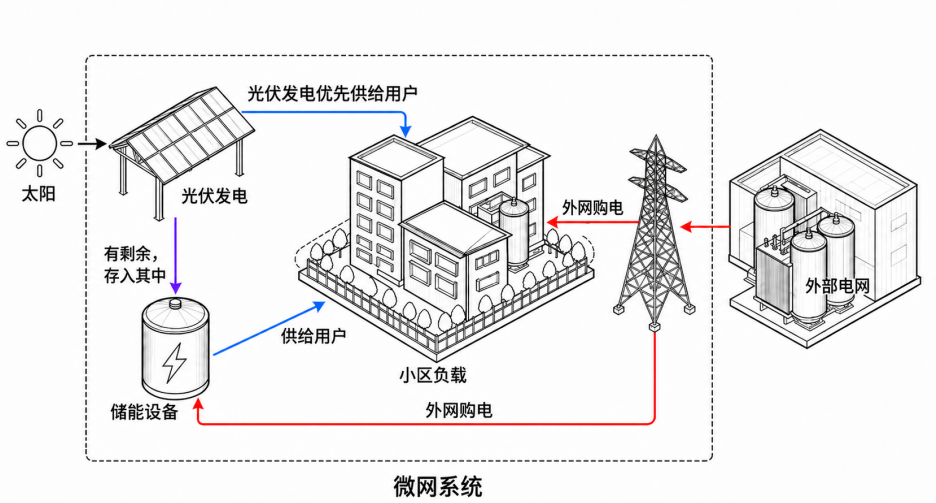


图 1　微网系统流程图

1.2 问题提出

问题一：确定条件下的微网计划购电策略。已知一天内的负载、电价和光伏预测数据，制定当天的购电与储能充放电计划。要求满足小区用电需求，并保证储能设备在 0:00 和 24:00 的电量相同，在此基础上尽量降低购电费用。

问题二：实际供需变化下的每日购电。小区实际负载和光伏发电会随时间变化，因此每天都需要根据已有信息重新制定购电计划。若实际供电不足，则需要按正常电价的 5 倍紧急购电，所以需要同时安排计划购电、储能使用和紧急购电，控制全年的总费用。

问题三：利用更新的光伏预测调整购电计划。每天 0:00 先根据光伏预测制定计划，之后在 6:00、12:00 和 18:00 得到新的预测信息后，可以调整尚未执行的购电安排。调整计划本身也会产生额外费用，因此需要比较“继续执行原计划”和“根据新预测进行调整”哪

一种更划算，并判断是否有必要引入后续时刻的预测。

问题四：波动电价下重新制定购电策略。进一步考虑外部电网电价随时间变化的情况，重新研究问题二和问题三中的购电与储能安排。需要比较波动电价下的计划购电、储能使用、紧急购电和计划调整情况，并分析电价变化对微网购电策略和总费用的影响。

二、问题分析

四问始终以购电费用为主线：先在供需与电价已知时统筹电量的跨时段安排，再为预测偏差设置准备，随后评价日内反馈与新预报的收益，最后将未知电价纳入决策。各问共享电量平衡和储能约束，随可用信息和调整权限变化逐步扩展，重点考察每项改进如何改变实际购电支出。

2.1 问题一的分析

问题一的关键是**供需与电价在时间上的错位**。光伏充足时未必用电最多，用电高峰也未必对应最低电价。储能可以转移电量，但容量、充放电功率和能量损耗限制了这种转移：提前充满可能挤占后续光伏的储存空间，过早放电又可能使高价时段缺少可用电量。因此，需要统筹全天的购电与充放电。

购电费用、电量平衡和储能变化均可用线性关系表示，适合用**电量流线性规划**求解。日初与日末储电量相等，则使每日安排能够重复执行，避免通过消耗初始库存压低当天费用。

2.2 问题二的分析

问题二在零点订电时尚不知道当天真实供需，需要先利用历史数据形成判断。负载具有星期规律，光伏的日际变化更强，因此二者应当**分别预测**。

预测偏差会造成两类损失：订电不足可能触发五倍电价的紧急补购，订电过多又可能付费后用不完。电池能够缓冲部分偏差，但受库存和功率限制。因此，需要根据近期误差设置适量准备，并考虑当天余电对次日购电的影响，将预测、误差准备和跨日调度结合起来。

2.3 问题三的分析

问题三允许获得新预报后修改尚未执行的订单。新光伏预报和已发生的负载可以修正零点判断，但短时偏差未必持续，部分缺口也可以由电池补足。因此，调整的关键是判断**未来供需变化是否值得支付改单费用**。

据此，将官方光伏预报与历史信息融合，利用当天负载偏差修正后续需求，再根据实际储电量滚动优化剩余订单。评价日内预报的价值时，应在相同调整权限下比较有、无后发预报的结果，区分新预报的收益与反馈改单本身的收益。

2.4 问题四的分析

问题四中，未来电价也需要预测。价格判断会改变提前购电和储能转移是否合算，因此应将**价格预测纳入调度**，而不只是更换最终结算单价。

两条分支分别承接问题二和问题三。零点分支主要依靠历史价格与供需特征判断全天价格；日内分支还可利用当天已发生的价格修正后续判断。针对两种信息条件分别选择预测方法，再将预测价格用于决策、实际价格用于结算，以评价价格预测对购电安排的实际作用。

四问的模型组成及递进关系如图 **??**所示。

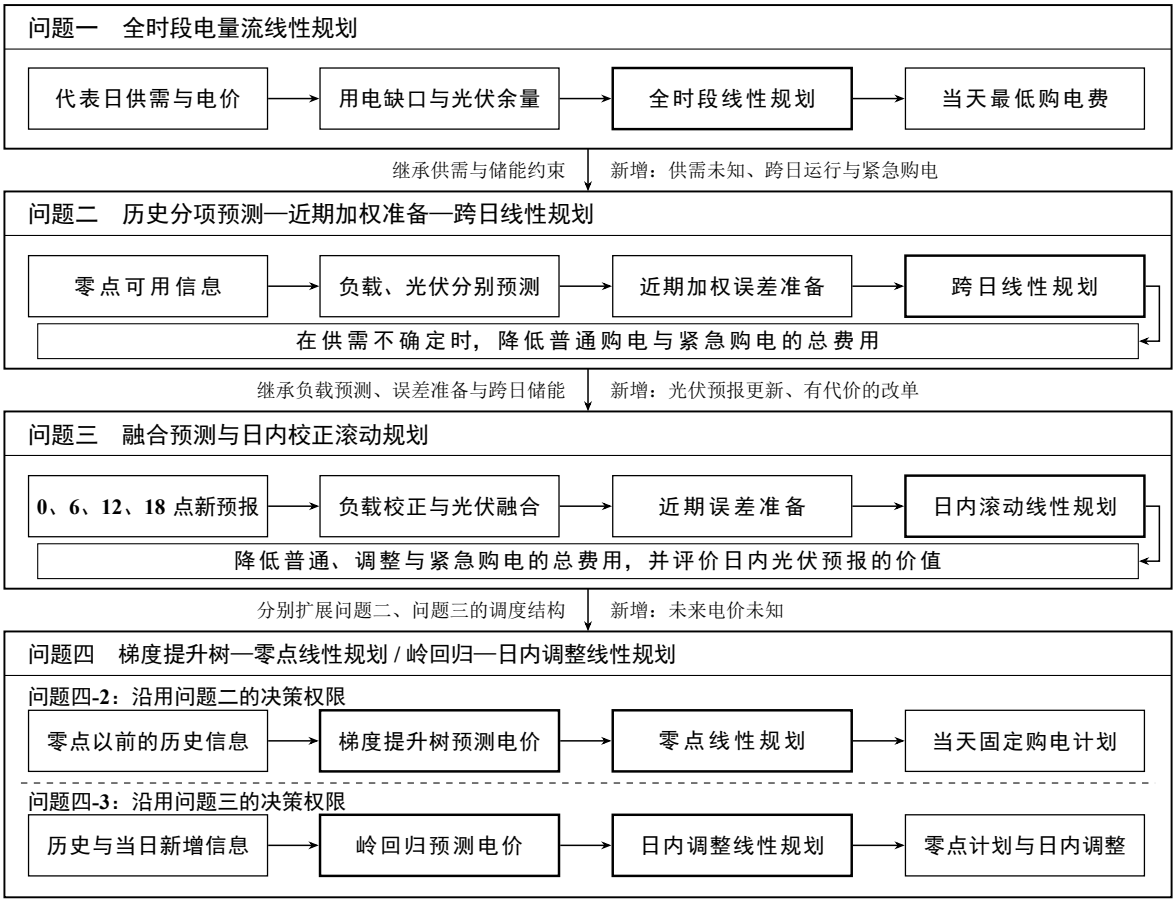


图 2　四问整体模型框架

三、模型假设与符号说明

3.1 模型假设

在题目给定的设备参数和交易规则基础上，为统一四个问题中的电量流向、时间尺度和信息使用范围，作如下假设。

假设 1：光伏优先满足负载，外网购电可直接供负载或用于充电；富余电量在容量和充电功率允许时存入储能。

假设 2：每个 10 分钟时段内，负载功率、光伏功率和电价保持不变。

假设 3：问题二至四的储电量跨日连续衔接，日初电量取前一日实际日末电量。

假设 4：决策仅使用当时已发生的观测和已发布的预报，不提前使用未来真实供需及电价。

假设 5：同一时段的调整费用，仅按最终订单相对零点原计划的净变化结算一次。

假设 6：不考虑电池自放电与寿命折旧，未利用的富余光伏不计售电收益。

3.2 符号说明

设 d 表示日期，k 表示一天中的第 k 个 10 分钟时段，其中

$$k = 1, 2, \ldots, 144, \qquad \Delta t = \frac{1}{6}\ \text{h}. \tag{1}$$

$S_{d,0}$ 表示第 d 天 0:00 的储电量，$S_{d,k}$ 表示第 k 个时段结束后的储电量。问题一仅研究一个代表日，因此相应公式中可省略日期下标 d。

主要公共符号如表所示。各问中特有的预测误差、准备量、订单调整量及价格模型参数，在对应模型中首次出现时再单独定义。

表 1　主要符号及其含义

符号	含义	单位或取值
d	日期下标	–
k	日内 10 分钟时段下标	$1, \ldots, 144$
Δt	单个调度时段长度	1/6 h
$L_{d,k}$	小区负载功率	kW
$P_{d,k}^{PV}$	光伏发电功率	kW
$E_{d,k}^{L}$	第 k 时段负载电量	kWh
$E_{d,k}^{PV}$	第 k 时段光伏电量	kWh
$p_{d,k}$	对应时段外网电价	元/kWh
$m_{d,k}$	光伏直接供给负载的电量	kWh
$n_{d,k}$	光伏直供后的负载缺口	kWh
$r_{d,k}$	光伏直供后的富余电量	kWh
$q_{d,k}$	普通外网购电量	kWh
$x_{d,k}$	外网购电中用于储能充电的电量	kWh
$y_{d,k}$	富余光伏用于储能充电的电量	kWh
$z_{d,k}$	储能设备向负载输出的电量	kWh
$S_{d,k}$	第 k 时段结束后的内部储电量	kWh
$w_{d,k}$	未被负载或储能利用的光伏电量	kWh
$S_{\min}$	储电量运行下限	1200 kWh
$S_{\max}$	储电量运行上限	10800 kWh
$P_{\max}$	最大充、放电功率	5000 kW
η_c	充电效率	0.90
η_d	放电效率	0.90

储能额定容量为 12000 kWh，其他运行参数见表 1。

四、数据预处理与探索性分析

4.1 数据预处理

本文数据来自赛题附件，涵盖负载、光伏、电价及不同时间发布的光伏预报。各附件的时间写法和排列方式有所不同，功率数据也需要转换为调度使用的电量。为使各项数据准确对应，进行以下处理。

1. 检查缺失与异常记录。原表已有记录未发现空单元格，但按实际时间排列后，全年实测数据缺少 1 月 1 日 00:00—00:10 这一段，启动处理见第 6.3 节。夜间光伏为零属于正常情况，予以保留；极端取值经原表核对后，也不因数值较大或较小而直接删除。

2. 统一时间与区间对应关系。各行数据用于标注时刻开始的 10 分钟区间。例如，首行“00:10”对应 00:10—00:20，末行“00:00+1”对应次日 00:00—00:10。问题一中，每天各时段的负载、光伏出力和电价均相同，因此将末行零点数据放到当天开头；全年连续数据则按实际日期衔接，不将次日零点移回当天。描述性统计仍按原表日期分组。

3. 区分预报批次。统一日期与时间格式，并为光伏预报同时保留“发布时间”和“目标时刻”，避免新预报覆盖旧预报。超出实测覆盖范围的预报予以保留，但不参与误差比较。

4. 转换电量并核对。每个时段为 10 分钟，即 $\Delta t = 1/6$ h，将功率按 $E = P\Delta t$ 转换为电量，并核对整理前后的数值及时间对应关系。全年统计用于识别数据特征，逐日决策仅使用当时已获得的信息。

4.2 探索性分析

经过整理与分析，得到以下四点发现。

(1)光伏富余与用电缺口存在时间错位

代表日午间光伏能够覆盖负载并产生富余，傍晚光伏回落后，外网需要补足的电量明显增加，且与较高电价有所重合。全年有 330 天出现过光伏功率高于负载的情况，说明光伏富余具有普遍性。

因此，储能需要将富余光伏或低价购电留到后续使用。能够转移多少电量，又受到容量、充放电功率和效率限制。这说明购电与储能应当按时段联合安排，全天电量总和不能代替逐时供需判断。

(2)负载与光伏的年内变化不同步

图 **??**(a) 显示，夏季负载较高，光伏也能提供较多电量；冬季负载虽有所回落，光伏下降却更加明显。以月平均功率差衡量，12 月负载减去光伏约为 3177 kW，接近 5 月的两倍。

这表明，**外网补电压力取决于负载与光伏的共同变化**。不同月份的电价也存在差异，因此需要随近期数据更新判断，不能将代表日的购电安排直接复制到全年。

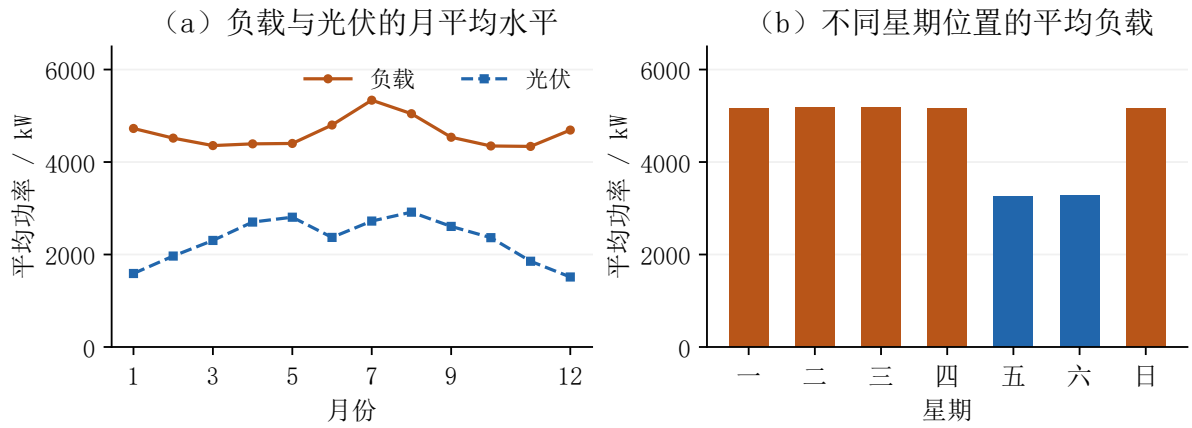


图 3　负载与光伏的季节差异及负载的星期规律

(3)负载具有星期规律，光伏与电价的日际变化更强

图 **??**(b) 显示，按实际星期分组后，周五、周六的平均负载比其余五天低约 36.8%；负载与一周前数据的相关性也高于与前一天数据的相关性。因此，同一星期位置的历史负载更有参考价值。

光伏在不同日期呈现相似的昼夜变化，但扣除各月相同时刻的平均水平后，其一周相关性接近于零。这说明，光伏的相似性主要来自白天发电、夜间为零的共同特征，负载的星期规律不宜直接用于光伏预测。

相同时刻在不同日期的电价也存在较大差异。由此，负载、光伏和电价应分别利用适合自身特点的历史信息，再共同参与购电决策。

(4)日内新预报提高预测精度，为滚动调整提供依据

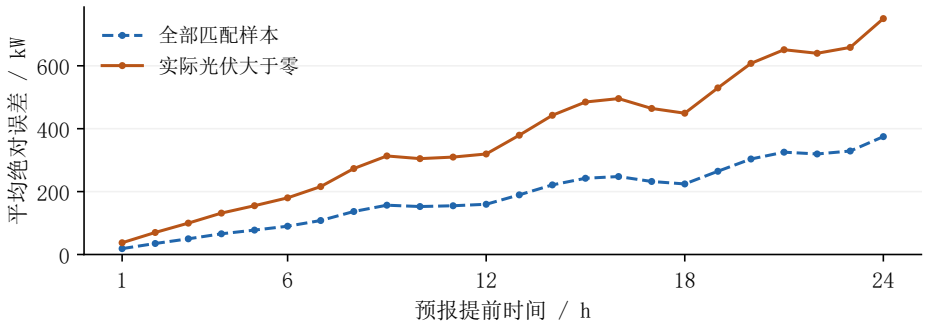


图 4　光伏预报提前时间与平均绝对误差

将不同发布时间的预报与相同目标时刻的实测值匹配后发现，预报提前时间越长，平均绝对误差总体越大，如图 **??**所示。在实际光伏大于零的 4177 组配对样本中，使用目标时刻前最新日内预报后，平均绝对误差由约 378.8 kW 降至 116.3 kW，下降约 69%。

新预报为修正购电提供了更准确的信息。考虑到约 17.4% 的样本更新后误差增大，且修改订单需要付费，后续由优化模型综合预报、储能余量和调整成本决定改单幅度，将预测精度的改善转化为实际购电收益。

五、问题一的模型建立与求解

5.1 电量流线性规划的建立

问题一已知各时段的供需和电价，适合统筹全天安排。本文考虑峰谷电价策略、光伏空间预留策略和线性规划三种方案。前两种策略直观易用，但需要事先设置价格门槛或目标储电量；线性规划则能同时考虑当前充放电对后续用电的影响。由于费用和设备约束均为线性关系，本文采用**电量流线性规划** [**?**]，并通过相同条件下的策略比较检验其效果。

(1)电量划分与决策变量

沿用前文的 144 个 10 分钟时段。第 k 段的负载电量和光伏电量为

$$E_k^L = L_k \Delta t, \qquad E_k^{PV} = P_k^{PV} \Delta t. \tag{2}$$

按照光伏优先满足负载的运行假设，定义

$$m_k = \min(E_k^L, E_k^{PV}), \qquad n_k = E_k^L - m_k, \qquad r_k = E_k^{PV} - m_k. \tag{3}$$

其中，m_k 为光伏直接供电量，n_k 为剩余负载缺口，r_k 为富余光伏。

以外网充电量 x_k、光伏充电量 y_k、电池向负载输出的电量 z_k 和时段末内部储电量 S_k 为决策变量。电量分配关系如图 **??**所示。

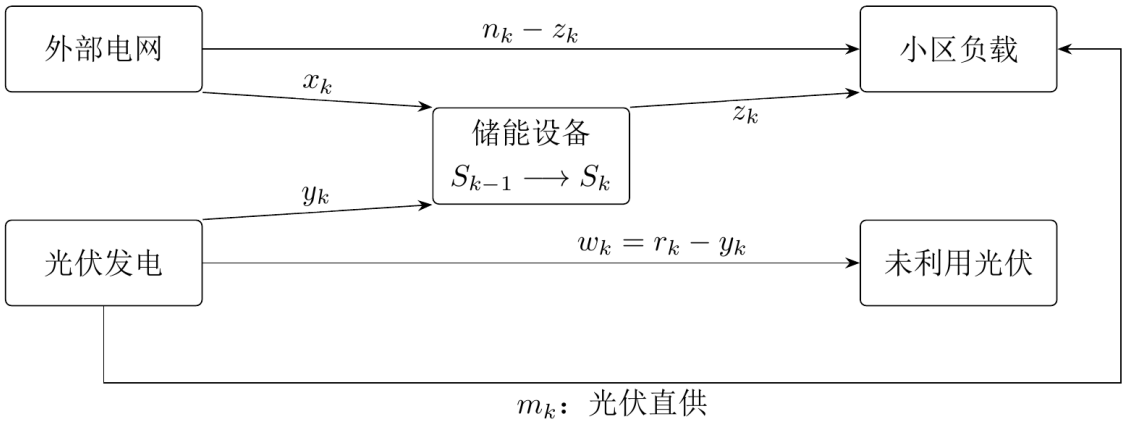


图 5　问题一的电量分配关系

外网既可补足负载缺口，也可给电池充电，购电量与未利用光伏电量为

$$q_k = n_k - z_k + x_k, \qquad w_k = r_k - y_k, \tag{4}$$

其中，q_k 为外网购电量，w_k 为未利用光伏电量。

(2)目标函数

以全天购电费用最低为目标：

$$\min C = \sum_{k=1}^{144} p_k (n_k - z_k + x_k). \tag{5}$$

该目标关注购电费用。即使储能损耗使总购电量有所增加，只要低价购电能够替代更昂贵的后续购电，仍可能降低总费用。

(3)约束条件

约束 1：电量分配与供电限制。光伏充电不超过富余量，电池放电不超过负载缺口：

$$x_k \geq 0, \qquad 0 \leq y_k \leq r_k, \qquad 0 \leq z_k \leq n_k, \tag{6}$$

约束 2：充放电功率限制。各时段充放电不超过设备允许功率：

$$x_k + y_k \leq P_{\max}\Delta t, \qquad z_k \leq P_{\max}\Delta t. \tag{7}$$

其中，$P_{\max}\Delta t = 5000/6$ kWh，计算时保留精确值。该限制作用于储能充放电，不限制外网直接向负载供电。

约束 3：储电量递推。考虑充、放电各 90% 的效率，内部储电量满足

$$S_k = S_{k-1} + \eta_c (x_k + y_k) - \frac{z_k}{\eta_d}, \qquad \eta_c = \eta_d = 0.9, \tag{8}$$

约束 4：储电量上下限。内部储电量保持在允许范围内：

$$S_{\min} \leq S_k \leq S_{\max}, \qquad S_{\min} = 1200, \quad S_{\max} = 10800. \tag{9}$$

约束 5：日初与日末电量相等。两者均取 6000 kWh：

$$S_0 = S_{144} = 6000. \tag{10}$$

这样既保证储能在允许范围内运行，也避免通过消耗初始电量降低当天表面费用，使各策略能够在相同起点和终点下比较。

5.2 模型求解

模型包含 576 个连续变量，各时段通过储电量递推相互连接，采用 SciPy[**?**] 的 `linprog` 接口调用 HiGHS 求解，其所含双单纯形算法的原理见文献 [**?**]。

本题无需额外设置 0—1 变量区分充放电。若某时段同时充放电，则该段存在负载缺口，没有富余光伏，充电只能来自外网。取 $\delta = \min(x_k, z_k/0.81) > 0$，将充电减少 δ、放电减少 0.81δ，不足的负载由外网直接补充。由于 $0.9\delta - 0.81\delta/0.9 = 0$，储电变化不变，功率约束仍满足，而购电量减少 0.19δ。电价为正，费用随之下降，故最优解不会同时充放电。

步骤 1：整理时段供需。按时间标签转换电量，计算光伏直供后的负载缺口和富余量。

步骤 2：求解购电与储能安排。将时段供需和电价代入模型，求得购电、充放电和储电量。
步骤 3：核对并汇总结果。复核供需、储能约束和费用，按模板时间标签填回结果。

5.3 调度结果与策略比较

(1)购电与储能结果

线性规划得到全天购电量 59482.70 kWh，购电费用 **35126.85 元**；储能累计充电 20740.67 kWh、向负载输出 16799.94 kWh。指定结果见表 **??**和表 **??**，电量单位为 kWh、费用单位为元。完整策略保存于 `result1.xlsx`。

表 2　微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	486.40	14:00-14:10	0.00
16:00-16:10	394.93	18:00-18:10	636.98	20:00-20:10	0.00
全天购电量		59482.70	全天购电费		**35126.85**

表 3　问题一各 4 小时区间的储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	4500.00	0.00	04:00–08:00	833.33	5947.42
08:00–12:00	3954.63	2121.42	12:00–16:00	6119.37	91.10
16:00–20:00	0.00	5068.69	20:00–24:00	5333.33	3571.31
0:00 储电量		6000.00	24:00 储电量		6000.00

表中充放电量按电池外侧计量，库存按内部电量计量。一个 4 小时区间同时出现充电和放电，表示不同 10 分钟时段采取了不同动作。

(2)策略比较

峰谷策略按价格门槛安排充放电，光伏空间预留策略进一步为午间富余光伏保留储能空间。两种规则均比较 27 组参数：低价门槛取电价的 15%、25%、35% 分位，高价门槛取 65%、75%、85% 分位，目标储电量取 6000、8400、10800 kWh。

分别选取已比较组合中费用最低的规则方案，与线性规划对照。各方案均在日末恢复 6000 kWh，恢复库存所需的购电全部计入费用，结果见表 **??**。

表 4　三种策略的全天购电费用比较

策略	全天购电费/元
峰谷电价策略	38276.64
光伏空间预留策略	37944.35
线性规划	**35126.85**

线性规划较峰谷策略节省 3149.79 元，降幅 **8.23%**；较光伏空间预留策略节省 2817.50 元，降幅 **7.43%**。三种策略的购电与储能安排如图 **??**所示。

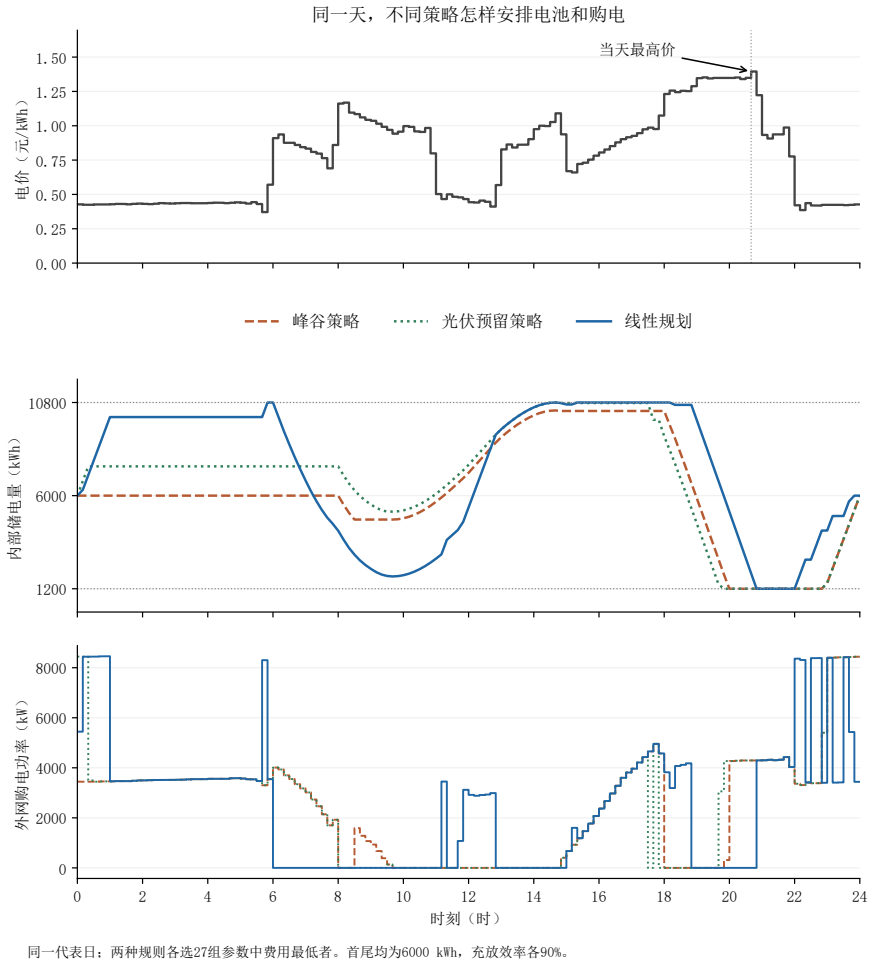


图 6　三种策略在相同代表日下的购电与储能安排

20:00 时，两种规则策略均已降至 1200 kWh 的储电下限，线性规划仍保留 5168.13 kWh。到 20:40—20:50，电价达到全天最高的 1.3952 元/kWh，线性规划利用储能满足负载，外网购电量为零；另外两种策略则需购买 715.65 kWh 的高价电。

因此，全天统筹的优势在于**根据供需和价格变化，将有限储能留给更有价值的时段**。虽然线性规划因储能损耗多购电 1740.28 kWh，但增加的低价购电替代了更昂贵的后续购电，最终费用更低。

5.4　模型检验与初始电量分析

逐时核对供需平衡、储电递推、容量与功率约束，并重新计算全天费用。三个初始电量案例的最大储电递推残差约为 1.34×10^{-12} kWh。利用对偶乘子独立重算费用下界后，求解费用与下界的最大差值约为 1.22×10^{-12} 元。两者均处于数值误差范围，支持所得解的可行性与数值最优性。

将初始电量分别设为 1200、6000 和 10800 kWh，并要求日末恢复原值，重新优化后的费用依次为 35199.30、35126.85 和 35409.75 元，前后两种状态较主方案增加 0.21% 和 0.81%。

三种初始储电量下，费用变化均不足 1%，且都为晚间最高价时段保留电量，说明主要调度规律在所比较条件下较为稳定。

六、问题二的模型建立与求解

6.1 供需预测与近期误差准备

问题二需要在当天供需尚未确定时提前订电。候选包括历史分项预测、岭回归 [?] 和提升树 [?]。历史参照预测方法 [?] 便于利用负载的星期规律和光伏的近期变化，解释供需判断的来源；后两者则可利用更多特征描述线性或非线性关系。

预测之后，还需要决定如何承担偏差带来的购电损失。因此，本文采用**历史分项预测、近期加权准备与跨日线性规划**，根据近期缺口误差设置准备，再结合储能余量确定订单，以减少预测不足时的高价补购。

(1)负载与光伏分别预测

负载取过去四个相同星期位置的同刻平均，光伏取最近七天的同刻平均：

$$\widehat{L}_{d,k} = \frac{1}{4}\sum_{j=1}^{4} L_{d-7j,k}, \qquad \widehat{P}^{PV}_{d,k} = \frac{1}{7}\sum_{j=1}^{7} P^{PV}_{d-j,k}. \tag{11}$$

四周同星期数据保留负载的星期规律，并平滑单日波动；七天光伏数据反映近期发电水平。上述窗口结合 1 月份的方案比较确定，历史不足时对当时已有的对应记录取平均。

跨日规划所需的次日负载，按次日星期位置选取历史；次日光伏仍参考当前零点以前最近七天的数据。两天预测均以**当前零点**为信息截止点，不使用未来真实值，也不引入附件 3 的光伏预报。

(2)根据近期误差设置准备

连续低估负载或高估光伏，会使电池不断补充缺口，最终可能触及库存下限。为此，定义历史净需求预测误差：

$$\varepsilon_{i,k} = \left[(L_{i,k} - P^{PV}_{i,k}) - (\widehat{L}_{i,k} - \widehat{P}^{PV}_{i,k})\right]\Delta t. \tag{12}$$

正值表示实际缺口大于预测。历史预测按当时能够获得的信息形成。

将一天分为六个 4 小时区间，收集最近 28 天同一区间的误差。设 $\mathcal{H}_{d,g}$ 为第 d 天零点可用的第 g 组记录，对历史误差赋予权重：

$$\omega_{d,i,k} = \frac{\beta^{d-1-i}}{\sum\limits_{(j,u)\in\mathcal{H}_{d,g}} \beta^{d-1-j}}, \qquad (i,k) \in \mathcal{H}_{d,g}. \tag{13}$$

较新的记录权重更高，使准备量跟随近期变化。按误差排序并累计权重，取相应分位：

$$F_{d,g}(u) = \sum_{(i,k)\in\mathcal{H}_{d,g}} \omega_{d,i,k}\mathbf{1}(\varepsilon_{i,k} \le u), \tag{14}$$

$$b_{d,k} = \max\left\{0, \inf\{u : F_{d,g(k)}(u) \ge \alpha\}\right\}. \tag{15}$$

其中，α **控制准备程度，**β **控制旧误差的衰减速度**。本文均取 0.7，但二者作用不同：70% 分位是误差分布中的位置，不是直接增加 70% 的购电量，也不是供电可靠率。

取 $\beta = 0.7$ 时，历史权重约每两天减半，既突出近期变化，也保留此前记录。较大的 β 保留更多早期误差，在本题比较中反而增加费用；继续减小则会使权重集中于少数日期，更容易受单日异常影响。结合 1 月 8—31 日的前期比较与评价期费用，本文选择 0.7，兼顾近期响应与历史平滑。全年费用比较覆盖 0.7、0.8 和 0.9 三档，详见第 6.5 节。

准备量进入规划净需求后，既可能增加预计缺口，也可能减少计划中的光伏富余，最终购电量仍由储能调度共同决定。

6.2 跨日购电与储能模型

只考虑当天，可能使模型在午夜前过早用尽储能。因此，每天零点规划未来 48 小时，共 288 个时段，**只固定当天前 144 个时段的订单**；次日安排用于辅助当前决策，到次日零点再重新制定。

用 t 连续编号规划时段，含准备量的净需求为

$$N_t = (\widehat{L}_t - \widehat{P}_t^{PV})\Delta t + b_t, \qquad n_t = \max(N_t, 0), \quad a_t = \max(-N_t, 0). \tag{16}$$

其中，n_t 为计划缺口，a_t 为扣除准备后的预计余量。

约束 1：计划供需平衡。普通订单满足

$$q_t = n_t - z_t + x_t. \tag{17}$$

以计划购电费用减去期末储电参考价值为目标：

$$\min J = \sum_{t=1}^{288} p_t q_t - \nu S_{288}, \qquad \nu = \eta_d p_{00:00}. \tag{18}$$

内部 1 kWh 电量能够向负载输出约 η_d kWh，因此按零点电价赋予参考价值。该项用于减轻规划末端的影响，不作为售电收入，也不从实际账单中扣除。

约束 2：储能运行限制。功率、储电递推与容量限制沿用式(??)—(??)，光伏充电上限改为预计余量 a_t，放电上限采用计划缺口 n_t。

约束 3：初始库存与跨日衔接。取 $S_0 = S_{d,0}$，日末实际余电传递至次日，不再要求每日首尾相等。

规划通过准备量应对补购风险，实际效果按普通电费与紧急电费之和评价。

6.3 模型求解与实际运行

岭回归与提升树候选由 scikit-learn 实现 [**?**]，各策略按以下步骤连续运行。

步骤 1：形成供需预测与准备量。每天零点，按第 6.1 节处理已有历史，形成未来 48 小时的供需判断。

步骤 2：求解跨日计划。从当前实际库存开始，采用 HiGHS 求解，只固定当天订单。

步骤 3：按实际供需执行。日内订单保持不变，储能根据实际供需充放电。

设某时段实际负载电量为 l，光伏电量为 v，普通订单为 q，时段开始时内部储电量为 S。供电不足时，先由电池补充，再紧急购买剩余缺口：

$$D = \max(l - v - q, 0), \tag{19}$$

$$z^{\mathrm{act}} = \min\{D, P_{\max}\Delta t, \eta_d(S - S_{\min})\}, \qquad e = D - z^{\mathrm{act}}. \tag{20}$$

存在余电时，在容量与充电功率允许范围内储存：

$$c^{\mathrm{act}} = \min\left\{\max(v + q - l, 0), P_{\max}\Delta t, \frac{S_{\max} - S}{\eta_c}\right\}. \tag{21}$$

充电优先使用富余光伏，再使用普通订单余量，内部储电量更新为

$$S^{\mathrm{next}} = S + \eta_c c^{\mathrm{act}} - \frac{z^{\mathrm{act}}}{\eta_d}. \tag{22}$$

步骤 4：结算并衔接次日。未能使用或储存的普通订单仍需付费，未利用光伏不计售电收益。每日实际费用及跨日衔接为

$$C_d = \sum_{k=1}^{144} \left(p_k q_{d,k} + 5p_k e_{d,k}\right), \qquad S_{d+1,0} = S_{d,144}. \tag{23}$$

运行从 1 月 1 日零点的 6000 kWh 开始，首日采用问题一的购电安排。实测序列缺少首个 00:00—00:10 区间，仅该段借用附件 1 零点供需作启动参考，不作为实测值或预测训练标签；此后使用附件 2 的连续实测数据。

1 月份用于历史积累、前期参数比较和库存衔接。2 月 1 日得到 8592.69 kWh，作为各候选及问题三、四的共同评价起点，此后各策略继承自身前一天的实际余电。2—12 月用于连续运行评价，也为近期加权方法的选择提供误差诊断依据。各日购电决策均只使用当时已获得的信息。

6.4 方案比较与运行结果

(1)误差准备的节费作用与方案比较

统一电价、设备参数、初始库存、规划长度与实际供电规则，比较 2025 年 2 月 1 日至 12 月 31 日共 334 天的实际费用，结果见表 **??**。

表中首先比较不设置准备的三种预测策略，再考察历史分项预测加入准备后的效果。

表 5　不同方案的 334 天实际购电费用

方案	普通电费/万元	紧急电费/万元	总电费/万元
历史分别预测，无误差准备	1218.58	374.70	1593.28
分别岭回归，无误差准备	1210.27	446.88	1657.15
分别提升树，无误差准备	1218.25	343.71	1561.96
历史分别预测，等权误差准备	1295.99	107.05	1403.04
历史分别预测，近期加权准备	1297.96	60.43	**1358.38**

最终方案较无准备的历史预测方案降低总费用约 14.74%。本题以实际购电费用评价预测、误差准备与储能调度组成的完整策略。历史分项预测利用负载的星期规律和光伏的近期变化，近期加权准备进一步缓冲预测偏差，两者与跨日规划结合后，在本次候选比较中取得最低总费用。

与等权准备相比，**近期加权以约 1.97 万元的新增普通电费，减少约 46.63 万元的紧急电费，净节省 44.66 万元**，降幅 3.18%。统一分位数计算方式后，仍节省约 44.54 万元，说明节费主要来自历史误差权重的调整。

这一组合让准备量跟随近期缺口变化，以适量普通购电减少更昂贵的临时补购，体现了预测与调度共同安排的优势。

(2)评价期与指定日期结果

最终总购电费用为 **13583810.84 元**，其中普通电费 12979560.57 元、紧急电费 604250.27 元。完整 10 分钟结果保存于 `result2.xlsx`。其中，计划购电表的全天购电量与费用只计普通订单，紧急购电另表列示；论文中的总费用为普通电费与紧急电费之和。

按月比较，11 个月的费用均低于等权准备方案，6 月和 12 月的节省较为明显，说明费用优势覆盖了不同季节。指定日期的结果见表 **??**、表 **??**及紧急购电表。

表 6　指定日期的计划购电及全天费用

项目	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00–10:10	0.00	0.00	0.00	0.00
12:00–12:10	610.16	0.00	484.48	943.63
14:00–14:10	0.00	0.00	0.00	0.00
16:00–16:10	434.90	116.65	453.86	798.03
18:00–18:10	716.29	437.88	826.38	742.86
20:00–20:10	0.00	0.00	5.90	0.00
全天计划购电量/kWh	64232.05	40298.69	68385.07	95446.27
全天紧急购电量/kWh	329.52	0.00	6.38	0.00
全天购电合计/kWh	64561.57	40298.69	68391.44	95446.27
普通电费/元	39469.74	22323.80	42730.05	61249.77
紧急电费/元	2298.74	0.00	43.00	0.00
总电费/元	41768.47	22323.80	42773.04	61249.77

6 月 21 日和 12 月 21 日均未发生紧急购电，未列出的时段紧急购电量为零。指定时段订单与充放电量的单位均为 kWh；全天合计由完整 144 个时段汇总。充放电按设备外侧计量，库存按内部电量计量。

表 7　指定日期的实际充放电量及日首尾储电量

时间段	项目	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
00:00–04:00	充电	1504.96	309.75	1739.22	1830.47
	放电	185.93	229.12	2.78	227.55
04:00–08:00	充电	1138.95	286.60	53.65	1476.38
	放电	4670.35	1949.50	5934.88	729.24
08:00–12:00	充电	5327.26	10134.27	4409.06	4706.14
	放电	3265.56	0.00	2387.94	8254.51
12:00–16:00	充电	4465.10	21.43	6147.58	8347.54
	放电	369.53	0.00	521.04	2318.98
16:00–20:00	充电	169.64	115.05	221.91	19.45
	放电	5340.10	5107.80	4920.36	4873.96
20:00–24:00	充电	4438.27	8636.77	8446.30	8360.50
	放电	3424.22	3022.72	3633.41	3626.10
0:00 储电量/kWh		8919.73	3543.84	9214.11	8886.94
24:00 储电量/kWh		5086.49	9642.73	8796.25	8897.44

表 8　问题二指定日期发生紧急购电的时段及电量

日期	时间段	紧急购电量/kWh
3 月 20 日	20:40–20:50	329.52
9 月 23 日	20:30–20:40	6.38

(3)储能对预测偏差的缓冲

以 3 月 20 日为例，储能补足了大部分订单与实际需求之间的缺口。但 20:40 开始时内部电量仅剩 1622.06 kWh，继续放电触及 1200 kWh 下限后，仍有 329.52 kWh 需要紧急购买。

6 月 21 日白天光伏充足，电池能够储存余电并在后续释放，全天未发生紧急购电。

两个日期说明，**缓冲能力取决于缺口出现时的库存与放电能力**，不能仅由全天供需总量判断。

6.5　参数检验与风险分析

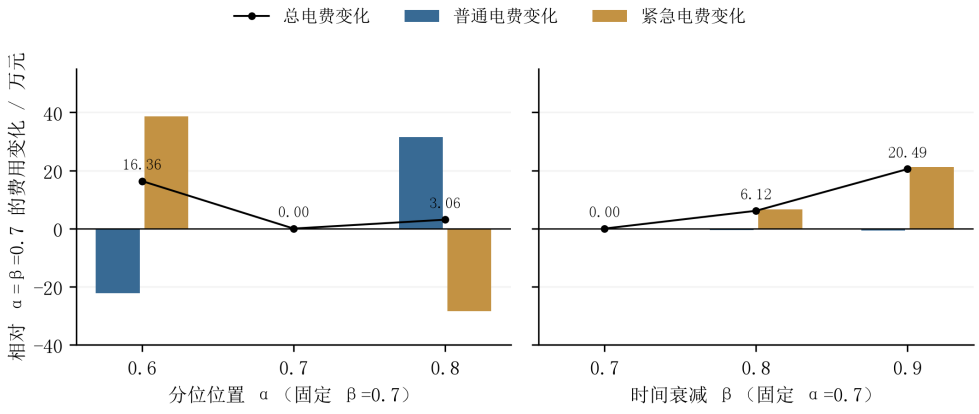


图 7　准备程度与近期权重对购电费用的影响

如图 **??**所示，固定 $\beta = 0.7$，将分位 α 提高至 0.8 后，紧急购电天数由 156 天降至 102 天，但新增普通电费超过紧急电费的节省，总费用上升约 3.06 万元；降至 0.6 时，准备不足使总费用上升约 16.36 万元。

固定 $\alpha = 0.7$，将 β 提高至 0.8 和 0.9，总费用分别增加约 6.12 万元和 20.49 万元。上述比较表明，两个参数取 0.7 能够较好地兼顾近期响应和购电准备，平衡两类费用。

48 小时规划将最长连续紧急补购时间由 340 分钟缩短至 200 分钟，并保留更多期末储电，评价期费用仅增加 3215.61 元。因此，本文以较小的费用代价改善跨日衔接，保留 48 小时规划。去除期末储电参考价值后，总费用不变，说明节费结果不依赖该项。

从历史误差中选取连续三天的不利片段，构造持续低估负载、高估光伏及两者叠加的情景，保持实际供需不变。近期加权方案仍比等权准备节省 3.09%—3.52%，保持了总费用优势。其中，持续高估光伏带来的费用增幅略大于等权方案，后续可重点加强对连续光伏偏差的识别。

七、问题三的模型建立与求解

7.1 供需预测与动态准备

新预报和当天观测有助于修正零点判断，但短时偏差未必持续，修改订单又需要付费。因此，本文采用**融合预测与日内校正滚动规划**。其中，负载预测结合同星期历史与当天偏差，判断后续用电需求；光伏预测融合官方预报和近期历史，再经插值统一到 10 分钟尺度；误差准备则为可能低估的缺口预留余量。三者共同进入滚动规划，结合储能状态和各项费用决定是否改单及调整幅度。

光伏预测考虑直接使用官方预报、历史融合和提升树修正三种方法。历史融合能够结合当天预报与近期发电水平，权重含义清楚，实际费用比较也支持这一选择。

(1)兼顾星期规律与近期变化的负载预测

负载保留问题二的同星期参照思路，但对较近的日期赋予更高权重。设 d 为当天，k 为日内第 k 个 10 分钟时段，零点负载预测为

$$\widehat{L}^{0}_{d,k} = \sum_{j=1}^{4} \gamma_j L_{d-7j,k}, \qquad \gamma_j = \frac{0.5^{j-1}}{\sum_{i=1}^{4} 0.5^{i-1}}. \tag{24}$$

最近四个同星期日的权重依次为 8/15、4/15、2/15 和 1/15。周权重按 0.5 逐周递减，使最近两周合计占 80% 的权重，同时保留更早两周的信息。这样既突出近期用电变化，也保留同星期历史的参考作用。后文比较表明，该近期加权方案较四周等权方案费用更低。

(2)用当天已发生的负载修正后续判断

用 $\tau \in \{0, 36, 72, 108\}$ 表示决策时已经结束的时段数，分别对应 0:00、6:00、12:00 和 18:00。在日内决策时，先计算当天已发生时段的平均负载偏差：

$$B_{d,\tau} = \frac{1}{\tau} \sum_{k=1}^{\tau} \left(L_{d,k} - \widehat{L}^{0}_{d,k} \right), \qquad \tau > 0. \tag{25}$$

正偏差表示当天至今的用电高于预期，负偏差则表示低于预期。随后，将尚未发生的时段按日内时刻划为 3 小时组，考察过去 28 个完整日中，同一决策时刻的已发生偏差与各组后续偏差之间的关系。

设 $Y_{i,\tau,g}$ 为历史日期 i 在未来第 g 组的平均负载预测误差，$\mathcal{H}_{d,\tau,g}$ 为可用历史日期集合，则修正系数为

$$\theta_{d,\tau,g} = \mathrm{clip}_{[0,1]} \left(\frac{\sum_{i \in \mathcal{H}_{d,\tau,g}} B_{i,\tau} Y_{i,\tau,g}}{\sum_{i \in \mathcal{H}_{d,\tau,g}} B_{i,\tau}^{2}} \right). \tag{26}$$

该式是无截距最小二乘估计。当天没有偏差时不作额外修正；历史上偏差延续得越明显，修正比例就越大。将系数限制在 0 到 1 之间，可以避免反向修正或放大短时波动。有效历史不足 7 天，或分母接近零时，取 $\theta_{d,\tau,g} = 0$。

当天后续负载据此更新为

$$\widehat{L}^{\tau}_{d,k} = \max \left\{ 0, \widehat{L}^{0}_{d,k} + \theta_{d,\tau,g(k)} B_{d,\tau} \right\}, \qquad k > \tau. \tag{27}$$

系数只由此前完整日期估计，当前偏差只使用已经结束的时段。零点不作日内偏差修正。次日的辅助负载预测仍由零点已知的同星期历史构造，不直接延续当天偏差。

(3)融合官方光伏预报与近期历史

光伏与负载分别预测。附件 3 的官方预报按整点给出，本文先在整点融合官方预报与历史参考，再转换为 10 分钟序列，以适应购电安排的时间尺度。

以 ℓ 表示预报所对应的整点，当前官方光伏预报为 $F_{d,\tau,\ell}$，近期同钟点的历史参考为 $H_{d,\tau,\ell}$，则整点融合预测为

$$\widetilde{P}^{PV}_{d,\tau,\ell} = \lambda_{d,\tau,h} F_{d,\tau,\ell} + (1 - \lambda_{d,\tau,h}) H_{d,\tau,\ell}. \tag{28}$$

历史参考取该钟点最近 7 次已结束时段的光伏均值。这里的 h 按预报提前 1–6、7–12、13–18 和 19–24 小时分组。记 $\mathcal{I}_{d,\tau,h}$ 为过去 28 天中相同发布钟点、同一提前量组内已经兑现的样本集合，融合权重取为

$$\lambda_{d,\tau,h} = \mathrm{clip}_{[0,1]} \left(\frac{\sum_{i \in \mathcal{I}_{d,\tau,h}} (F_i - H_i)(P^{PV}_i - H_i)}{\sum_{i \in \mathcal{I}_{d,\tau,h}} (F_i - H_i)^2} \right). \tag{29}$$

该式选择使历史融合平方误差最小的比例，让两类信息相互补充。融合权重随发布时间和预测提前量变化，而不是全年固定不变。同组样本不足 24 条时，合并相同发布钟点的

其他提前量样本；仍不足 24 条，或分母接近零时，取 $\lambda = 1$，直接使用官方预报。

对融合后的相邻整点值作线性插值，得到 10 分钟光伏预测 $\widehat{P}^{PV}_{d,\tau,k}$。决策时刚结束时段的光伏观测作为起点，尚未发生的真实光伏不参与计算。当前预报只覆盖未来 24 小时，超过其覆盖范围至次日 24:00 的辅助时段，采用历史同钟点参考。

(4)根据近期净需求误差设置安全准备

沿用第 6.1 节的净需求误差与加权分位方法，为可能低估的用电缺口设置准备。历史误差按原发布时间的预测计算，保证它反映当时能够形成的供需判断。

每次决策使用此前 28 天发布且已兑现的误差，按相同发布时间、目标日内 3 小时组，以及预测距发布时间是否达到 24 小时分别归集。历史误差仍按 $\omega_j \propto 0.7^{a_j}$ 加权，a_j 为相隔天数。28 天窗口保留四周信息，3 小时分组将每个 6 小时更新区间分成前后两段，使准备量兼顾近期变化和日内差异。组内样本不足时合并相同发布时间和覆盖范围的误差，仍不足 24 条时不附加准备。

取加权误差的 0.60 分位，并将负值截为零，得到准备量

$$M_{d,\tau,k} = \max\left\{0, Q^{(\omega)}_{0.60}(\varepsilon)\right\}. \tag{30}$$

0.60 表示误差排序中的位置，不是多买 60% 的电，也不是供电保证概率。它与负载的周权重 0.5、误差的日衰减 0.7 各有不同作用。分位选得过低容易漏掉缺口，过高则增加无效订电；本问的分位比较见第 7.6 节。

7.2 滚动购电模型与求解

(1)供需关系与储能约束

为简化表达，以下用 t 连续编号当前时刻至次日 24:00 的时段，省略日期和发布时间下标。考虑安全准备后的预计净需求为

$$N_t = (\widehat{L}_t - \widehat{P}^{PV}_t)\Delta t + M_t, \qquad n_t = \max(N_t, 0), \quad a_t = \max(-N_t, 0). \tag{31}$$

N_t 为带符号的净需求，n_t 为预计需要外网或电池补充的电量，a_t 为预计可用于储能的光伏富余。准备量只改变规划中的供需判断。沿用前文的订单、充放电与库存变量，另设预计紧急购电量 e_t 和已订购但预计无法接收的电量 u_t，用于权衡改单、补购与订单过量的代价。

约束 1：计划供需平衡。光伏优先直接满足负载后，其余供需满足

$$q_t - x_t + z_t + e_t - u_t = n_t. \tag{32}$$

普通订单既能供负载，也能给电池充电。若订单有剩余且电池无法接收，剩余部分计入 u_t，仍按订单付费。

约束 2：储能与供电范围限制。功率、储电递推与容量限制沿用式(??)—(??)，另满足

$$0 \le y_t \le a_t, \qquad z_t + e_t \le n_t, \qquad q_t, x_t, y_t, z_t, e_t, u_t \ge 0. \tag{33}$$

光伏充电不超过预计富余，电池放电和紧急补购不超过预计缺口。

约束 3：库存衔接与订单锁定。每次从实际库存开始规划，日末余电传递至次日；已经执行的订单不再修改，仅优化剩余时段。

(2)订单调整费用

设零点原始订单为 q_t^0，执行前最后确定的订单为 q_t^F。按本文统一采用的结算口径，调减部分退回原计划电费，再收取原电价 50% 的违约费；调增部分支付电费及 50% 的附加费。普通订单与调整费用合计为

$$C_{\mathrm{ord}} = \sum_t \left[p_t q_t^F + \frac{1}{2} p_t |q_t^F - q_t^0| \right]. \tag{34}$$

例如，100 kWh 订单在 1 元/kWh 时减至 80 kWh，结算为 80 + 10 = 90 元。同一时段多次调整，只比较最后订单与零点订单的净变化，重叠部分不重复收费。该式也包含未能接收的普通订单费用，不能只按实际使用量结算。记 e_t^{act} 为实际运行中电池仍无法补足的紧急购电量，则实际总费用为

$$C_{\mathrm{act}} = C_{\mathrm{ord}} + 5 \sum_t p_t e_t^{\mathrm{act}},$$

其中求和只覆盖实际执行的时段。它与规划中用于权衡风险的预计紧急购电量 e_t 不同，最终账单按实际补购计费。

(3)目标函数

每次规划都延伸至次日 24:00，使模型考虑留电对下一天的作用。记 $\mathcal{T}_\tau$ 为全部规划时段，$\mathcal{A}_\tau$ 为其中当天尚未执行、且已经在零点订过电的时段。零点决策时 $\mathcal{A}_0$ 为空。目标函数为

$$\min J = \sum_{t \in \mathcal{T}_\tau} (p_t q_t + 5 p_t e_t) + \frac{1}{2} \sum_{t \in \mathcal{A}_\tau} p_t \delta_t - \nu S_T, \tag{35}$$

辅助约束：调整量的线性化。对 $t \in \mathcal{A}_\tau$，令

$$\delta_t \ge q_t - q_t^0, \qquad \delta_t \ge q_t^0 - q_t, \qquad \delta_t \ge 0. \tag{36}$$

只有当日已有订单的修改产生调整费。次日订单只是辅助规划，不在本次签订，也不收调整费。

末端库存系数取 $\nu = 0.9 p_{0:00} = 0.38484$ 元/kWh，以一度内部存电经放电后可替代的低价外网电作为参考。$-\nu S_T$ 用于抑制规划末端放空电池的倾向，不是实际售电收入，不计入最终购电账单。绝对值通过上述辅助变量转化为线性约束，采用 HiGHS 求解。

(4)滚动求解步骤

步骤 1：制定零点原计划。每天零点，读取已有历史、当前库存和官方预报，形成供需预测与准备量，求得并保存当天原始订单。次日的辅助安排仅用于判断当前应保留多少电。

步骤 2：执行购电与储能安排。按第 6.3 节的实际供电规则运行至下一决策时点。光伏和普通电不足时由电池补充，剩余缺口紧急购买；余电在容量和充电功率允许时储存，并逐时更新实际库存。

步骤 3：利用新信息调整订单。在 6:00、12:00 和 18:00，更新负载偏差、光伏融合预测和准备量，从当前实际储电量重新求解。已经执行的订单锁定，未执行部分是否调整由费用权衡决定。随后继续执行步骤 2，直至当天结束。

步骤 4：结算费用并衔接次日。保留各时段执行前最后确定的订单，按照其相对零点订单的净变化计算调整费用，再加上实际紧急购电费。日末实际余电传递至下一天。

费用评价期为 2025 年 2 月 1 日至 12 月 31 日，共 334 天，起始库存沿用第 6.3 节的 8592.69 kWh。各日连续运行，完整方案每天求解 4 次，共完成 1336 次滚动线性规划。

7.3 候选方案比较

为区分光伏预测方法的作用，先固定四周同星期等权负载、不作日内负载校正，且不附加误差准备，只比较三种光伏处理方式。各方案采用相同的设备、起始库存和结算规则，结果如表 **??**所示。

表 9　固定负载与准备条件下的光伏预测方案比较

光伏处理方式	普通结算/万元	紧急购电/万元	总费用/万元
直接采用官方预报	1289.96	127.56	1417.51
历史误差融合预报	1276.36	132.51	1408.87
提升树修正预报	1279.46	136.80	1416.26

历史融合降低的普通结算费用超过紧急购电费的增加，使总费用降至三种光伏方案中的最低水平。这说明官方预报与近期历史能够相互补充。在此基础上，引入误差准备，进一步处理低估缺口所带来的高价补购损失。

负载方面，在同样使用日内校正、融合光伏和 0.60 分位准备的条件下，比较一周参照、两周等权、四周等权及四周近期加权，结果见表 **??**。各方法均由自身的历史预测误差计算准备量，使准备程度与预测方法相匹配。

表 10　同星期负载参照方式的费用比较

负载参照方式	总费用/万元	紧急购电/万元
最近一周	1306.22	18.82
最近两周等权	1303.74	17.85
最近四周等权	1304.61	15.68
最近四周近期加权	**1301.42**	16.13

四周近期加权兼顾历史平滑与近期响应，较四周等权节省约 3.20 万元。其普通结算

费用的下降超过了紧急购电费的增加，因而在本次比较中获得更低的整体费用。两组比较分别支持历史融合与近期加权负载，二者共同用于本问的滚动规划。

7.4 日内反馈与新预报的价值

为区分新预报与反馈改单各自的作用，本文在同一套负载预测、准备规则和调度结构下设置三组信息条件，共同从同一实际库存开始连续运行。第一组只在零点订电，全天不改单；第二组允许根据已发生的供需和库存调整，但不读取后发官方预报；第三组在第二组基础上加入后发官方预报。各组的预测误差与准备量均由自身可用信息构造，结果见表 **??**。

表 11　同一模型下三种信息条件的费用比较

信息与决策条件	总费用/万元	较上一条件节省/万元
零点订电，全天不修改订单	1359.00	—
日内反馈与改单，不使用后发官方预报	1308.37	**50.63**
日内反馈与改单，使用后发官方预报	1301.42	**6.95**

日内反馈与改单使费用减少 50.63 万元，后发光伏预报进一步节省 6.95 万元。新增预报使普通订单费和紧急电费分别减少约 7.78 万元和 6.03 万元，覆盖新增的 6.86 万元调整费后，仍实现上述净节省，降幅约 0.53%。后两组期末库存相同，说明额外节费来自信息改善，而非多消耗剩余电量。

新预报还降低了对临时补购的依赖：紧急购电量由 40956.71 kWh 降至 28627.32 kWh，**下降 30.10%**；紧急购电费下降 27.22%，出现紧急购电的日期由 191 天减少至 185 天。因此，新增调整支出换来了更大的购电节省，也减轻了临时高价补购的压力。

此外，未接收的普通订单由 38.83 万 kWh 降至 32.66 万 kWh，减少 15.88%；未利用光伏电量减少 3.07%。这说明新预报有助于协调购电与储能，减少订多却用不上的电量，并促进光伏就地利用，符合绿色低碳发展的方向。

上述对照表明，在给定条件下，**有必要将 6:00、12:00 和 18:00 发布的光伏预报纳入日内决策**。它们在已有反馈与改单能力的基础上，进一步降低费用、减少紧急补购和未利用电量。具体是否改单、调整多少，则由模型结合供需判断、储能余量与调整费用确定。

7.5 调度结果与分析

(1)费用构成

最终方案在 334 天内的普通订单费为 1257.23 万元，调整费为 28.06 万元，二者合计普通结算费 1285.29 万元；紧急购电费为 16.13 万元。总购电费用为 **13014172.33 元**。评价期内紧急购电量为 28627.32 kWh，共 185 天出现紧急购电。评价期内未接收的普通订单为 32.66 万 kWh，其费用已计入总支出。因此，需要同时权衡订电过量与紧急补购的代价。

(2)零点原计划与日内调整

题目指定日期在 0:00 制定的部分购电计划见表 **??**。表中每项均为对应 10 分钟区间的电量，原计划在当天后续决策中保留，作为调整计费的基准。

表 12　指定日期零点原计划的部分时段购电量（kWh）

时间段	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00–10:10	0.00	0.00	0.00	0.00
12:00–12:10	568.44	0.00	424.94	957.12
14:00–14:10	0.00	0.00	0.00	0.00
16:00–16:10	480.78	115.62	443.64	798.21
18:00–18:10	722.54	426.86	810.48	727.22
20:00–20:10	0.00	0.00	0.00	0.00

经过三次日内决策后，最终普通订单及全天费用见表 **??**。这里的最终订单是该区间执行前最后确定的总量，并非相对零点的增减量。6:00、12:00 和 18:00 分别形成剩余时段的订单版本，已执行部分保持不变；各次版本用于追溯调整过程，最终订单用于实际结算。零点计划与最终订单的差异反映了新供需判断和当前库存对后续安排的影响。

表 13　指定日期调整后订单及全天费用（时段订单单位为 kWh）

项目	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00–10:10	0.00	0.00	0.00	0.00
12:00–12:10	574.44	0.00	422.94	985.17
14:00–14:10	0.00	0.00	0.00	0.00
16:00–16:10	480.78	71.01	443.64	827.19
18:00–18:10	722.54	415.06	810.48	727.30
20:00–20:10	0.00	0.00	0.00	0.00
全天原计划购电量/kWh	62950.87	40273.79	67565.05	94803.82
全天最终普通订单/kWh	62434.82	40006.24	67338.24	95848.96
全天紧急购电量/kWh	18.54	0.00	242.51	73.49
普通结算费用/元	39514.95	22010.94	42247.91	62215.35
紧急购电费用/元	117.15	0.00	1277.76	334.35
全天总费用/元	39632.09	22010.94	43525.66	62549.70

前三个日期的最终订单低于零点计划，12 月 21 日则增加约 1045 kWh，说明日内调整并非统一加量或减量，而是随供需判断变化。表中各项分别四舍五入，费用合计由未舍入值计算。

(3)储能运行与紧急购电

四个指定日期的实际充放电和日首尾库存见表 **??**。充电量在电池输入侧计量，放电量在输出侧计量，首尾库存则为电池内部电量。某个 4 小时区间同时有充电和放电，表示内部不同 10 分钟时段采取了不同动作，不表示同一时段同时充放电。

指定日期发生紧急购电的时段见表 **??**，未列出的时段为零。6 月 21 日没有紧急购电；其余日期的缺口也并非都由库存触底造成。

表 14　指定日期的实际充放电及日首尾储电量（kWh）

时间段	项目	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
00:00–04:00	充电	1510.42	267.18	2048.71	1958.12
	放电	238.58	302.32	61.89	239.38
04:00–08:00	充电	1034.68	536.17	529.10	1617.46
	放电	5467.56	1753.56	6328.62	793.91
08:00–12:00	充电	4061.01	10134.27	4352.49	4419.79
	放电	3259.44	0.00	2369.61	8608.93
12:00–16:00	充电	6377.03	299.24	6833.50	8730.87
	放电	254.72	0.00	417.27	2012.03
16:00–20:00	充电	147.87	83.04	94.47	0.95
	放电	4675.69	5205.83	5101.73	4983.07
20:00–24:00	充电	4148.71	8559.86	8245.77	8334.82
	放电	3749.74	3041.10	3620.94	3624.47
0:00	内部储电量	8871.16	2971.14	8609.68	8695.56
24:00	内部储电量	4816.54	9415.36	8614.36	8738.25

表 15　指定日期紧急购电的时段、电量及费用

日期	时间段	紧急购电量/kWh	费用/元
3 月 20 日	10:00–10:10	6.12	30.53
	20:40–20:50	12.42	86.62
6 月 21 日	无	0.00	0.00
9 月 23 日	09:40–09:50	4.35	20.48
	09:50–10:00	13.98	66.88
	20:30–20:40	6.38	43.00
	20:40–20:50	58.51	408.19
	21:00–21:10	65.38	305.04
	21:10–21:20	28.13	127.56
	21:20–21:30	2.38	11.15
	21:30–21:40	34.04	159.48
	21:40–21:50	20.86	102.94
	21:50–22:00	8.51	33.03
12 月 21 日	08:00–08:10	22.46	130.48
	10:50–11:00	51.03	203.87

图 **??**展示 3 月 20 日和 9 月 23 日的供需与储能过程。3 月 20 日两次补购均发生在电池降至最低库存时。9 月 23 日 20:30，电池虽有余电，当段却已达到最大放电功率，仍需紧急补购；随后多个时段又因库存触底而补购。

这说明**库存下限和放电功率限制都可能触发紧急补购**，电池里有电并不等于任何时段都能及时供够。判断紧急购电是否必要，必须同时考虑缺口出现的时刻、可用库存和放电功率，不能只比较全天电量总和。

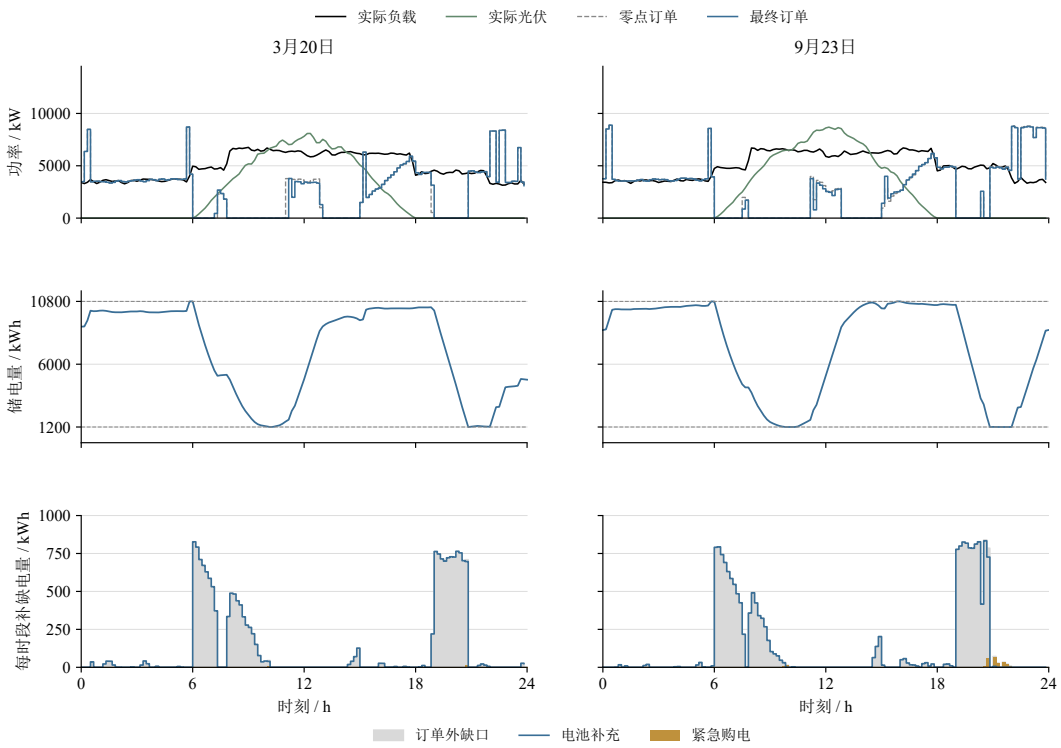


图 8　两个指定日期的供需、储电状态及缺口补充

7.6　参数检验

为考察准备程度是否合适，保持负载与光伏预测、调度方式和结算规则不变，将误差分位依次取 0.40、0.50、0.60、0.70 和 0.80。图 **??**显示，总费用随准备程度先降后升，在五档比较中，0.60 对应的总费用最低。

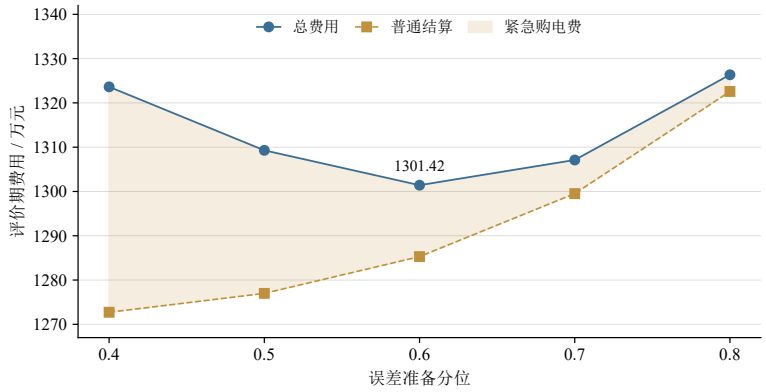


图 9　误差准备程度与购电费用的关系

准备不足会增加高价补购，准备过多又会增加普通订单及无法接收的电量。例如，分位从 0.60 提高到 0.70 后，紧急购电费减少约 8.54 万元，普通结算费却增加约 14.23 万元。因此，0.60 在五档比较中较好地平衡了两类损失，体现了**适量准备对整体购电经济性的作用**。

计算后，逐 10 分钟核对供需平衡、库存衔接、充放电功率和已执行订单锁定，并由实际订单与紧急购电量重新计算账单。各项约束得到满足，费用复算差异小于 0.01 元。上

述检验表明，方案在本文设定的信息与结算条件下能够连续执行，储能运行符合设备限制，购电费用可由实际订单逐项核算。

八、问题四的模型建立与求解

8.1 电价预测与方法选择

问题四中，未来电价同样不能提前获得。价格判断会影响提前购电和储能转移是否合算，因此需要将价格预测纳入调度。问题四-2 沿用问题二的零点订电方式，日内不改单；问题四-3 承接问题三的供需预测与日内调整，两条分支保留各自的信息条件和决策权限。

电价预测考虑历史同刻中位数、岭回归和提升树。同刻中位数保留常见的日内价格规律，但对近期变化反应有限；提升树能够描述价格与多种因素的非线性关系，适合零点缺少当天观测的情况；日内已有实际价格参照，岭回归则能用清楚的线性关系修正判断，并限制相关特征造成的系数波动。因此，两条分支分别选用**提升树预测与零点规划**和**岭回归预测与日内滚动规划**，再以实际费用检验选择。

预测输入包括附件 1 电价、最近四周同星期同刻价格中位数、最近 7 天同刻中位数、上周同刻价格，以及星期、季节位置和预测净负载。同星期历史不足时，改用近期同刻记录。日内再加入当天已发生价格相对历史同刻基准的偏差中位数、最后一个已结束时段的偏差，以及按 6 小时、1 小时时间尺度指数衰减的偏差特征，区分持续变化与短时波动。这些当天偏差只用于当天剩余时段，零点取零。

净负载由各分支自己的负载预测减去光伏预测得到。训练样本按当时可用信息构造，实际价格只有发生后才能作为训练目标或日内反馈。

(1)问题四-2：提升树价格预测

每天 0:00 尚未获得当天实际价格，主要依靠历史价格和供需背景进行预测。采用直方图梯度提升树，将多棵较小的回归树逐步相加 [**?**]：

$$\hat{p} = F_0(\mathbf{x}) + \gamma \sum_{m=1}^{M} h_m(\mathbf{x}), \tag{37}$$

其中，$\mathbf{x}$ 为上述价格特征，F_0 为初始预测，h_m 用于修正此前预测的残差。取 $M = 100$、学习率 $\gamma = 0.05$，每棵树最多包含 31 个叶节点。较小的学习率使每棵树只作有限修正，避免一次过度追随历史波动，叶节点数的比较见第 8.5 节。

(2)问题四-3：岭回归价格预测

在 6:00、12:00 和 18:00，可以利用当天已经发生的实际价格。历史同刻价格与近期价格往往较为接近，直接回归容易让它们的系数相互抵消并过度放大。岭回归在拟合误差之外加入系数惩罚 [**?**]，使价格判断更加平稳：

$$\min_{\beta_0,\boldsymbol{\beta}} \sum_i \left(p_i - \beta_0 - \mathbf{z}_i^{\mathrm{T}}\boldsymbol{\beta}\right)^2 + \lambda_p \sum_j \beta_j^2, \qquad \lambda_p = 10. \tag{38}$$

其中，p_i 为训练样本的实际电价，$\mathbf{z}_i$ 为标准化后的特征，β_0 和 $\boldsymbol{\beta}$ 分别为截距与回归系数。标准化所需的均值和标准差仅由当前训练窗口计算，正则系数 $\lambda_p = 10$ 用于限制过大的回归系数。该设置下，岭回归在候选比较中以接近最低的费用获得较少的紧急购电，因而用于日内分支。预测时使用

$$\hat{p}_{t|\tau} = \max\left\{0.001,\ \beta_0 + \mathbf{z}_{t|\tau}^{\mathrm{T}}\boldsymbol{\beta}\right\}, \tag{39}$$

其中 τ 沿用问题三的决策时点记号，t 为待预测时段。下限用于避免回归外推产生非正电价；提升树及历史参照的预测也统一保留这一正下限。

两类价格模型均每 7 天训练一次，使用最近 56 天已经发生的数据；历史不足 56 天时使用已有样本。日内各时点更新模型输入和预测结果，无需每天重新拟合四次模型。

8.2 波动电价下的购电模型

(1)问题四-2

问题四-2 直接沿用问题二的负载、光伏预测、误差准备和储能约束，将固定电价替换为 0:00 预测得到的波动电价。目标函数为

$$\min J_{4\text{-}2} = \sum_{t\in T_0} \hat{p}_{t|0} q_t - \nu S_T, \tag{40}$$

其中，T_0 为零点开始的规划时段，q_t 为普通购电量，S_T 为规划末端储电量，ν 沿用前文的末端电量参考价值。

规划延伸至次日 24:00，以避免模型在当天末尾过早放空电池，但只固定当天订单。实际运行中，普通购电和电池放电仍无法满足的缺口，按照该时段**实际电价的 5 倍**紧急补购。

(2)问题四-3

问题四-3 将本问的价格预测代入第 7.2 节的滚动规划，供需预测与误差准备沿用问题三。

设零点原计划为 q_t^0，调整后的订单为 q_t，则目标函数为

$$\min J_{4\text{-}3} = \sum_{t\in\mathcal{T}_\tau} \hat{p}_{t|\tau}\,(q_t + 5e_t) + \frac{1}{2}\sum_{t\in\mathcal{A}_\tau} \hat{p}_{t|\tau}\left|q_t - q_t^0\right| - \nu S_T, \tag{41}$$

其中，e_t 为预计紧急购电量，$\mathcal{T}_\tau$ 为当前时点至次日 24:00 的规划时段，$\mathcal{A}_\tau$ 为当天尚未执行且已在零点订电的时段。零点决策时 $\mathcal{A}_0$ 为空，次日辅助订单也不收调整费。绝对值按第 7.2 节线性化，末端电量参考价值不计入实际账单。

约束沿用。问题四-2 沿用第 6.2 节的三项约束，问题四-3 沿用第 7.2 节的三项约束及调整量线性化方法。规划使用预测电价，普通购电、调整和紧急购电均按执行时段的实际电价结算。

8.3 模型求解与方案比较

评价期与共同起始库存沿用第 6.3 节，两条分支按时间顺序连续运行 334 天。

步骤 1：更新电价预测。按第 8.1 节更新价格模型及输入，得到决策时可用的未来价格判断。

步骤 2：分别求解两条分支。从当前实际库存开始，调用 HiGHS 求解。问题四-2 仅在零点订电；问题四-3 在零点订电，并在 6:00、12:00 和 18:00 更新剩余订单。

步骤 3：实际执行与结算。按对应分支的供电规则执行，以实际价格结算各项费用，并将日末实际余电传递至次日。

为比较价格模型，在各分支内固定供需预测、设备参数和初始储电量，只改变价格预测方法，结果见表**??**。

表 16　波动电价下不同价格预测模型的比较

分支	价格模型	价格 MAE/(元/kWh)	总费用/万元	紧急购电量/kWh
问题四-2	历史同刻中位数	0.04984	1427.22	97728.89
问题四-2	岭回归	0.04472	1426.64	99205.19
问题四-2	**提升树**	0.03884	1425.52	98725.98
问题四-3	历史同刻中位数	0.04676	1383.78	40144.52
问题四-3	**岭回归**	0.04442	1382.29	38733.80
问题四-3	提升树	0.03834	1382.06	40116.12

注：加粗方法为所选路线。价格模型比较固定采用四周同星期等权负载，不作日内负载校正；表中提升树均为 15 叶。该表用于比较价格预测方法，完整策略的费用另见表**??**。

问题四-2 中，提升树的预测误差和购电费用均最低，说明非线性价格预测适合零点制定整日订单的条件。其叶节点数再由下文的复杂度检验确定。

价格预测的误差大小需要结合实际费用与紧急补购共同评价。问题四-3 中，提升树的预测误差更小，但在表中相同供需条件下，费用仅比岭回归低约 0.016%，紧急购电量却更多。因此，我们在日内分支选择岭回归，以接近最低的费用获得较少的紧急补购和更清楚的预测关系，再将其与问题三的负载校正和滚动调整结合。

8.4 运行结果分析

两种策略的评价期费用见表**??**。其中，问题四-2 采用 31 叶提升树；问题四-3 采用岭回归，误差准备取 0.60 分位。

两条分支的总费用分别为 **14248570.87 元**和 **13695929.70 元**。日内方案节省 **55.26 万元，降幅 3.88%：**虽然产生了调整费，但普通购电与紧急补购的费用下降更多。

表 17　问题四两种策略在 2025 年 2—12 月的费用结果

指标	问题四-2	问题四-3
普通购电费/元	13613522.32	13212221.00
调整费/元	0.00	299775.47
紧急购电费/元	635048.55	183933.22
总费用/元	**14248570.87**	**13695929.70**
紧急购电量/kWh	98756.33	32162.76

注：各项分别保留两位小数，合计由未舍入费用计算。

这部分节省来自供需判断、价格预测和订单调整的共同改进，体现了日内信息更新与滚动调度的综合价值。波动电价会改变提前购电是否合算，进而影响充放电安排、日末余电及后续日期的运行。

四个指定日期的结果见表**??**；指定 10 分钟时段的订单、各 4 小时区间的充放电量、日首尾库存及紧急购电时段详见附录 A。普通订单、紧急补购和日末余电共同反映各日实际安排，费用包含普通购电、调整及紧急购电。

表 18　问题四两条分支的指定日期结果

分支	日期	普通订单/kWh	紧急购电/kWh	日末储电/kWh	总费用/元
四-2	3 月 20 日	62329.63	329.13	1376.30	41909.16
四-2	6 月 21 日	42375.21	0.00	10380.22	20358.11
四-2	9 月 23 日	68066.93	10.23	2592.35	44700.97
四-2	12 月 21 日	94612.77	0.00	9798.14	71357.71
四-3	3 月 20 日	61627.63	16.86	1386.88	40429.92
四-3	6 月 21 日	42677.14	0.00	10800.00	20343.61
四-3	9 月 23 日	66331.89	247.01	2815.47	45166.06
四-3	12 月 21 日	93273.34	51.32	8176.08	72209.10

注：普通订单为全天最终订单之和；总费用按实际电价计算，包含普通购电、调整与紧急购电。

储电量跨日衔接，使单日费用同时受到当天预测偏差和此前库存的影响。因此，指定日期用于说明具体运行过程，完整评价期总费用用于衡量整体经济性。日内方案同期净节省 55.26 万元，表明其跨日综合收益能够覆盖部分日期的费用增加。

8.5　参数与风险检验

分别检验问题四-2 的树复杂度和问题四-3 的准备程度。前者影响价格预测的细致程度，后者决定多订普通电与承担紧急补购之间的取舍。

(1)提升树复杂度检验

以 15 叶模型为参照，7 叶模型费用略增，31 叶模型减少约 6606 元，说明适当增加叶节点有助于刻画价格差异。按实际电价的 5%、50%、95% 分位对期末可放电量估值后，31 叶仍节省 5890.24–6423.72 元，说明其费用优势不依赖于少留期末电量。

(2)误差准备与补购风险

比较 0.40 至 0.80 五档准备分位。准备不足容易触发五倍电价补购，准备过多又会增加普通订单及存不下的电量。0.60 的总费用最低；提高至 0.70 虽能减少紧急购电，但总费用增加约 4.41 万元，因此保留 0.60。

日内方案满足了各时段负载，评价期内 188 天发生紧急补购，总量约 3.22 万 kWh，低于零点分支的约 9.88 万 kWh。剩余补购中，约 1.07 万 kWh 发生在最高 5% 的实际电价区间，进一步改进可集中于这些时段的缺口识别和储能准备。

逐 10 分钟复核供需平衡、储电衔接、容量与功率限制，并按实际价格复算账单，各项结果一致，差异处于浮点误差范围内。训练与预测均以当前时点为数据截止，保证决策只依据当时已知的信息。

九、模型评价与推广

9.1 模型优点与创新

1. **以实际费用检验预测价值。**预测与调度共同服务于购电节费。近期加权准备较等权方案净节省 44.66 万元；日内价格预测则兼顾费用与紧急补购，使方法选择落实到运行代价。
2. **单独量化新增信息的节费贡献。**同一模型下，日内反馈与改单节省 50.63 万元，后发光伏预报再节省 6.95 万元。对照区分了两类收益，且新预报的节费并非来自多消耗期末电量。
3. **统一储能约束，支持连续运行与核验。**四问共用电量平衡与储能约束，逐 10 分钟核对库存衔接和设备限制，并复算账单，便于核验连续运行与节费结果。

9.2 模型局限与改进方向

现有参数比较基于 2025 年数据，后续可通过跨年验证完善校准，补充供需与电价共同异常的情景，并纳入电池老化和网络损耗成本，使方案更贴近长期运行条件。

9.3 模型推广性评估

该框架可为校园、社区和工业园区微网提供参考。按当地供需、设备和交易规则校准后，可扩展至风光互补、多台储能及可调负载，协调本地发电与外网购电，兼顾用能经济性和可再生能源利用，与我国新能源配套储能及微电网发展的政策方向相衔接 [**?**, **?**]。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于题意与数据分析辅助、候选方案讨论、程序编写与调试、计算验证、图表绘制、论文起草与修改以及参考文献核查，详细使用情况见支撑材料。

参考文献

[1] Dantzig G B. Linear Programming and Extensions[M]. Princeton: Princeton University Press, 1963. DOI: 10.1515/9781400884179.

[2] Virtanen P, Gommers R, Oliphant T E, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python[J]. Nature Methods, 2020, 17(3): 261–272. DOI: 10.1038/s41592-019-0686-2.

[3] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142. DOI: 10.1007/s12532-017-0130-5.

[4] Hoerl A E, Kennard R W. Ridge regression: Biased estimation for nonorthogonal problems[J]. Technometrics, 1970, 12(1): 55–67. DOI: 10.1080/00401706.1970.10488634.

[5] Friedman J H. Greedy function approximation: A gradient boosting machine[J]. The Annals of Statistics, 2001, 29(5): 1189–1232. DOI: 10.1214/aos/1013203451.

[6] Hyndman R J, Athanasopoulos G. Forecasting: Principles and Practice[M/OL]. 3rd ed. Melbourne: OTexts, 2021[2026-09-12]. https://otexts.com/fpp3/.

[7] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine learning in Python[J]. Journal of Machine Learning Research, 2011, 12(85): 2825–2830.

[8] 国务院. 国务院关于印发 2030 年前碳达峰行动方案的通知：国发〔2021〕23 号[EB/OL]. (2021-10-26)[2026-09-12]. https://www.gov.cn/zhengce/content/2021-10/26/content_5644984.htm.

[9] 国家发展改革委, 国家能源局. 关于印发《“十四五”现代能源体系规划》的通知：发改能源〔2022〕210 号 [EB/OL]. (2022-03-22)[2026-09-12]. https://www.ndrc.gov.cn/xxgk/zcfb/ghwb/202203/t20220322_1320016.html.

附录 A　问题四指定日期的详细调度结果

以下结果分别对应问题四的零点分支与日内分支。购电量和充放电量以 kWh 计，费用以元计；充放电量在设备外侧计量，储电量在电池内部计量。总费用包含普通购电、调整和紧急购电费用，各项分别四舍五入，合计由未舍入值计算。

问题四-2：零点固定订单

表 19　问题四-2：零点固定订单的指定时段订单与费用

项目	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00–10:10	0.00	0.00	0.00	0.00
12:00–12:10	610.16	0.00	484.48	943.63
14:00–14:10	0.00	0.00	0.00	0.00
16:00–16:10	434.90	116.65	453.86	798.03
18:00–18:10	716.29	437.88	826.38	742.86
20:00–20:10	33.92	0.00	839.24	0.00
全天普通订单	62329.63	42375.21	68066.93	94612.77
全天紧急购电量	329.13	0.00	10.23	0.00
普通购电费	39732.30	20358.11	44634.70	71357.71
调整费	0.00	0.00	0.00	0.00
紧急购电费	2176.87	0.00	66.27	0.00
全天总费用	41909.16	20358.11	44700.97	71357.71

表 20　问题四-2：零点固定订单的充放电与日首尾储电量

时间段	项目	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
00:00–04:00	充电	3659.75	309.75	8646.90	384.61
	放电	157.83	726.21	0.00	227.55
04:00–08:00	充电	1660.51	1628.23	157.72	1051.96
	放电	5081.26	1719.95	5934.88	699.45
08:00–12:00	充电	5327.26	10134.27	4409.06	4706.14
	放电	3265.56	0.00	2387.94	8254.51
12:00–16:00	充电	4459.46	28.05	6098.44	8347.54
	放电	364.97	0.00	471.90	2318.98
16:00–20:00	充电	167.58	0.00	219.05	19.45
	放电	5371.96	5132.01	5748.33	4873.96
20:00–24:00	充电	320.01	9594.69	1569.84	9357.20
	放电	3394.06	3017.50	2826.04	3622.79
0:00	内部储电量	6936.35	2627.70	2900.43	10537.08
24:00	内部储电量	1376.30	10380.22	2592.35	9798.14

表 21　问题四-2：零点固定订单的紧急购电时段

日期	时间段	紧急购电量/kWh	费用/元
3 月 20 日	20:40–20:50	329.13	2176.87
6 月 21 日	无	0.00	0.00
9 月 23 日	20:20–20:30	10.23	66.27
12 月 21 日	无	0.00	0.00

连续发生的紧急购电按相邻时段合并，电量与费用取区间内合计；未列出的时段紧急购电量为零。

问题四-3：日内调整订单

零点原计划作为调整基准；最终普通订单是各时段执行前最后确定的购电总量，不是调整差额。

表 22　问题四-3 指定日期的零点原计划

时间段	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00–10:10	0.00	0.00	0.00	0.00
12:00–12:10	568.44	0.00	424.94	957.12
14:00–14:10	0.00	0.00	0.00	0.00
16:00–16:10	480.78	115.62	443.64	798.21
18:00–18:10	722.54	426.86	810.48	727.22
20:00–20:10	0.00	0.00	807.54	0.00
全天原计划购电量	62160.53	42946.95	66502.59	92173.20

表 23　问题四-3：日内调整订单的指定时段订单与费用

项目	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00–10:10	0.00	0.00	0.00	0.00
12:00–12:10	574.44	0.00	422.94	985.17
14:00–14:10	0.00	0.00	0.00	0.00
16:00–16:10	480.78	71.01	443.64	827.19
18:00–18:10	722.54	403.12	810.48	727.30
20:00–20:10	0.00	0.00	807.54	0.00
全天普通订单	61627.63	42677.14	66331.89	93273.34
全天紧急购电量	16.86	0.00	247.01	51.32
普通购电费	39625.05	20194.90	43396.17	70814.67
调整费	702.82	148.72	432.81	1068.40
紧急购电费	102.05	0.00	1337.07	326.03
全天总费用	40429.92	20343.61	45166.06	72209.10

表 24　问题四-3：日内调整订单的充放电与日首尾储电量

时间段	项目	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
00:00–04:00	充电	4460.16	267.18	6038.71	677.12
	放电	136.36	809.07	20.53	246.48
04:00–08:00	充电	1058.70	1865.66	2073.34	247.18
	放电	5467.56	1753.56	6313.31	123.34
08:00–12:00	充电	4061.01	10134.27	4352.49	4443.59
	放电	3259.44	0.00	2369.61	8606.69
12:00–16:00	充电	6396.78	316.34	6816.66	8730.87
	放电	254.72	0.00	403.62	2012.03
16:00–20:00	充电	134.57	26.55	73.69	0.95
	放电	5915.24	5214.70	6269.48	4983.07
20:00–24:00	充电	337.98	10159.28	1854.33	7721.77
	放电	2515.42	3035.82	2478.29	3633.85
0:00	内部储电量	6081.20	2322.27	3565.89	10320.62
24:00	内部储电量	1386.88	10800.00	2815.47	8176.08

表 25　问题四-3：日内调整订单的紧急购电时段

日期	时间段	紧急购电量/kWh	费用/元
3 月 20 日	10:00–10:10	6.12	31.00
	20:40–20:50	10.74	71.05
6 月 21 日	无	0.00	0.00
9 月 23 日	09:40–10:00	18.33	89.36
	18:30–18:40	1.08	7.51
	20:20–20:30	10.23	66.27
	20:40–20:50	54.79	379.88
	21:00–22:10	162.57	794.05
12 月 21 日	08:00–08:10	22.46	157.95
	10:50–11:00	28.85	168.08

连续发生的紧急购电按相邻时段合并，电量与费用取区间内合计；未列出的时段紧急购电量为零。

附录 B　支撑材料文件清单

支撑材料与论文分别提交。以下文件路径均相对于支撑材料根目录；根目录中的五个 result 工作簿为已填写的结果，templates 目录中的同名文件为空白模板。

1. AI 工具使用详情.pdf
2. code/energy_paths.tex
3. code/explore_data.py
4. code/export_results.py
5. code/fig_data.py
6. code/fig_q1.py
7. code/fig_q2_sensitivity.py
8. code/fig_q3_days.py
9. code/fig_q3_quantile.py
10. code/framework.tex
11. code/prepare_inputs.py
12. code/prepare_source_tables.py
13. code/q1.py
14. code/q2.py
15. code/q3.py
16. code/q3_base.py
17. code/q4.py
18. code/q4_tree.py
19. code/q4_update.py
20. code/reproduce.py
21. code/source_map.json
22. code/validation.py
23. data/actual_power.csv.gz
24. data/actual_price.csv.gz
25. data/day_reference.csv.gz
26. data/inputs.npz
27. data/pv_forecasts.csv.gz
28. data/q1_config.json
29. data/raw_hashes.json
30. evidence/export_check.json
31. evidence/extra_reproduction_check.json
32. evidence/Q1_storage_sensitivity.csv
33. evidence/Q1_strategy_comparison.csv
34. evidence/Q2_annual_candidates.csv
35. evidence/Q2_refinement.csv
36. evidence/Q2_validation.csv
37. evidence/Q3_final_comparison.csv
38. evidence/Q4_final_comparison.csv

39. evidence/Q4_validation.csv
40. evidence/reproduction_check.json
41. evidence/workbook_readback.json
42. evidence/参考文献来源.tex
43. figure_inputs/derived_values.json
44. figure_inputs/eda/forecast_error_by_lead.csv
45. figure_inputs/eda/monthly_summary.csv
46. figure_inputs/eda/weekday_summary.csv
47. figure_inputs/q1/outputs/schedules/C03-p07.csv
48. figure_inputs/q1/outputs/schedules/C04.csv
49. figure_inputs/q1/outputs/schedules/C10-p05.csv
50. figure_inputs/q1/outputs/summary.json
51. figure_inputs/q2/refinement/comparison.csv
52. figure_inputs/q2/supplement/comparison.csv
53. figure_inputs/q2/validation/comparison.csv
54. figure_inputs/q3_daily.csv
55. figures/data_compact.pdf
56. figures/forecast_compact.pdf
57. figures/model.pdf
58. figures/q1_1.png
59. figures/q1_daily_dispatch.pdf
60. figures/q3_final_days.pdf
61. figures/q3_final_quantile.pdf
62. figures/sensitivity.png
63. figures/weiwang.png
64. figures/图源说明.md
65. requirements.txt
66. result1.xlsx
67. result2.xlsx
68. result3.xlsx
69. result4-2.xlsx
70. result4-3.xlsx
71. results/daily_costs.csv
72. results/Q1.csv
73. results/Q2.npz
74. results/Q3.npz
75. results/Q3_adjustments.csv.gz
76. results/Q4-2.npz
77. results/Q4-3.npz
78. results/Q4-3_adjustments.csv.gz
79. SHA256SUMS.txt
80. templates/result1.xlsx

81. templates/result2.xlsx
82. templates/result3.xlsx
83. templates/result4-2.xlsx
84. templates/result4-3.xlsx
85. 使用说明.md
86. 文件清单.md

附录 C　程序运行与结果文件说明

C.1　运行环境与入口

计算程序使用 Python 3.13，所需软件包及版本见支撑材料中的 requirements.txt。程序使用相对路径读取文件，在支撑材料根目录依次执行以下命令：

```
python -m pip install -r requirements.txt
python code/prepare_inputs.py
python code/reproduce.py
python code/export_results.py
```

第一条命令安装依赖；第二条根据随包数据重建计算输入并逐值核对；第三条重新计算问题一、衔接 1 月份储电量，再分别计算后续四条年度方案。复算输出写入 reproduced 目录，并与 results 目录的保存结果比较。最后一条命令依据 results 中的完整精度记录填写五份结果模板，核对费用和储能约束。

如需从赛题原始 Excel 重新整理数据，可将附件按原目录结构置于 data/raw/C 题/附件/，将题面置于 data/raw/C 题/C 题.pdf，再运行 code/prepare_source_tables.py。原始附件由赛题提供，不在支撑材料中重复保存。

C.2　结果与图表的对应关系

result1.xlsx 给出问题一完整一天的安排；result2.xlsx、result3.xlsx、result4-2.xlsx 与 result4-3.xlsx 分别给出对应方案 2025 年 2 月 1 日至 12 月 31 日的结果。工作簿沿用赛题模板样式，单元格保留完整精度，费用以未舍入值计算。results 目录保存逐时段记录和逐日费用，问题三及问题四日内分支的历次调整另存于相应的压缩 CSV 文件。

code/explore_data.py 生成描述统计，code/validation.py 提供已有对照检验的运行入口。五个以 fig 开头的 Python 程序根据 figure_inputs 中的数据生成图表；两份 TikZ 源文件可使用 XeLaTeX 编译。具体命令见支撑材料的使用说明。本文结果表由 Python 写入 Excel，不依赖 Excel 宏或 SPSS 交互命令。

附录 D　完整源程序

以下按数据整理、模型计算、检验与结果导出、图表绘制的顺序列出支撑材料中的全部程序。程序内容直接读取同名源文件，不以片段或省略号代替。

D.1　原始附件读取与整理

文件：code/prepare_source_tables.py

```
"""Execute approved Data-plan-v1. Copy source values; no interval interpolation.

python code/prepare_source_tables.py
Use --output <new project-relative directory> for non-overwriting reproduction.
"""
import argparse
import csv
from datetime import date, datetime, time, timedelta
import hashlib
import json
from pathlib import Path
import re
import sys
import xml.etree.ElementTree as ET
import zipfile

import openpyxl
from openpyxl.utils import get_column_letter

ROOT = Path(__file__).resolve().parents[1]
NS = {'s':'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}
REL = '{http://schemas.openxmlformats.org/officeDocument/2006/relationships}id'
SOURCES = {
    'annex1': ('data/raw/C 题/附件/附件 1.xlsx', 'Sheet1'),
    'annex2_load': ('data/raw/C 题/附件/附件 2.xlsx', ' 小区负载'),
    'annex2_pv': ('data/raw/C 题/附件/附件 2.xlsx', ' 光伏发电实际功率'),
    'annex3': ('data/raw/C 题/附件/附件 3.xlsx', 'Sheet1'),
    'annex4': ('data/raw/C 题/附件/附件 4.xlsx', 'Sheet1'),
}


def sha(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def label(value):
    return value.isoformat() if isinstance(value,(date,datetime,time)) else str(value or '')


def parse_date(value):
    if isinstance(value,datetime):
        return value.date()
    if isinstance(value,date):
        return value
    return date(*map(int,re.split(r'[-/.]',value.strip())))

```

```
47
48  def minutes(value):
49      if isinstance(value,time):
50          assert value.second == 0 and value.microsecond == 0
51          return value.hour*60+value.minute
52      m=re.fullmatch(r'(\d{1,2}):(\d{1,2})(\+1)?',value.strip())
53      assert m, value
54      return int(m[1])*60+int(m[2])+(1440 if m[3] else 0)
55
56
57  def source(sid):
58      path,sheet=SOURCES[sid]
59      wb=openpyxl.load_workbook(ROOT/path,read_only=True,data_only=False)
60      ws=wb[sheet]
61      rows=list(ws.iter_rows(values_only=True))
62      wb.close()
63      with zipfile.ZipFile(ROOT/path) as z:
64          tree=ET.fromstring(z.read('xl/workbook.xml'))
65          rid=next(s for s in tree.find('s:sheets',NS) if
            ↪  s.attrib['name']==sheet).attrib[REL]
66          target=next(r.attrib['Target'] for r in
            ↪  ET.fromstring(z.read('xl/_rels/workbook.xml.rels')) if
            ↪  r.attrib['Id']==rid)
67          target=target.lstrip('/') if target.startswith('/') else 'xl/'+target
68          values={}
69          for cell in ET.fromstring(z.read(target)).findall('.//s:sheetData/s:
            ↪  row/s:c',NS):
70              v=cell.find('s:v',NS)
71              if v is not None and cell.attrib.get('t','n')=='n':
72                  values[cell.attrib['r']]=v.text
73      return rows,values
74
75
76  def write_csv(out,name,fields,records):
77      path=out/name
78      with path.open('x',encoding='utf-8-sig',newline='') as stream:
79          writer=csv.DictWriter(stream,fieldnames=fields)
80          writer.writeheader()
81          writer.writerows(records)
82      return {'rows':len(records),'columns':fields,'sha256':sha(path)}
83
84
85  def main():
86      parser=argparse.ArgumentParser()
87      parser.add_argument('--output',default='reproduced/source_tables')
88      args=parser.parse_args()
89      out=(ROOT/args.output).resolve()
90      if ROOT not in out.parents or (ROOT/'data/raw').resolve() in out.parents
        ↪  or out==(ROOT/'data/raw').resolve():
91          raise ValueError('Output must be a new project directory outside
            ↪  raw')
92      if out.exists():
93          raise FileExistsError(f'Will not overwrite: {out}')
94      expected=json.loads((ROOT/'data/raw_hashes.json').read_text(encoding='ut
        ↪  f-8'))
95      if any(sha(ROOT/p)!=h for p,h in expected.items()):
96          raise RuntimeError('Source hashes differ from approved audit')
97      loaded={sid:source(sid) for sid in SOURCES}
98      tables={}
```

```
    rows,raw=loaded['annex1']
    records=[]
    for r in range(2,len(rows)+1):
        t=rows[r-1][0]
        records.append(dict(slot_index=r-1,source_time_label=label(t),source_time_type=type(t).__name__,source_minute=minutes(t),price_yuan_per_kwh=raw[f'B{r}'],load_kw=raw[f'C{r}'],pv_forecast_kw=raw[f'D{r}'],source_id='annex1',time_cell=f'A{r}',price_cell=f'B{r}',load_cell=f'C{r}',pv_forecast_cell=f'D{r}'))
    tables['day_reference.csv']=records

    def matrix(sid):
        rows,raw=loaded[sid]
        result={}
        for r in range(2,len(rows)+1):
            d=parse_date(rows[r-1][0])
            for c in range(2,len(rows[0])+1):
                t=rows[0][c-1]
                key=(d.isoformat(),minutes(t))
                if key in result:
                    raise ValueError(f'Duplicate source key: {sid} {key}')
                result[key]=dict(source_date=d.isoformat(),source_date_label=label(rows[r-1][0]),source_date_type=type(rows[r-1][0]).__name__,source_time_label=label(t),source_time_type=type(t).__name__,source_minute=minutes(t),value=raw[f'{get_column_letter(c)}{r}'],source_id=sid,date_cell=f'A{r}',time_cell=f'{get_column_letter(c)}1',value_cell=f'{get_column_letter(c)}{r}')
        return result
    load,pv,price=(matrix(s) for s in ['annex2_load','annex2_pv','annex4'])
    if set(load)!=set(pv) or set(load)!=set(price):
        raise ValueError('Actual axes do not align exactly')
    common=['source_date','source_date_label','source_date_type','source_time_label','source_time_type','source_minute']
    records=[]
    for key in sorted(load):
        a,b=load[key],pv[key]
        assert all(a[k]==b[k] for k in common)
        record={k:a[k] for k in common}
        record.update(load_kw=a['value'],pv_actual_kw=b['value'])
        for prefix,item in [('load',a),('pv',b)]:
            record.update({f'{prefix}_{k}':item[k] for k in
             ↪  ['source_id','date_cell','time_cell','value_cell']})
        records.append(record)
    tables['actual_power.csv']=records
    records=[]
    for key in sorted(price):
        a=price[key]
        record={k:a[k] for k in common}
        record.update(price_yuan_per_kwh=a['value'])
        record.update({k:a[k] for k in
        ↪  ['source_id','date_cell','time_cell','value_cell']})
        records.append(record)
    tables['actual_price.csv']=records

    actual_labels={datetime.fromisoformat(d)+timedelta(minutes=m) for d,m in
    ↪  load}
    rows,raw=loaded['annex3']
    records=[]
```

```
    inherited_rows=0
    for group_row in range(2,len(rows)+1,4):
        group_date=rows[group_row-1][0]
        d=parse_date(group_date)
        for offset in range(4):
            r=group_row+offset
            release_value=rows[r-1][1]
            release=datetime.combine(d,time())+timedelta(minutes=minutes(rel
            ↪ ease_value))
            inherited=int(rows[r-1][0] in (None,''))
            inherited_rows+=inherited
            for lead in range(1,25):
                c=lead+2
                target=release+timedelta(hours=lead)
                records.append(dict(release_datetime=release.isoformat(),tar
                ↪ get_datetime=target.isoformat(),lead_hours=lead,pv_forec
                ↪ ast_kw=raw[f'{get_column_letter(c)}{r}'],source_date_lab
                ↪ el=label(group_date),source_date_type=type(group_date).__
                ↪ _name__,source_row_date_label=label(rows[r-1][0]),source
                ↪ _row_date_type=type(rows[r-1][0]).__name__,source_releas
                ↪ e_label=label(release_value),source_release_type=type(re
                ↪ lease_value).__name__,date_inherited=inherited,has_actua
                ↪ l_target_label=int(target in
                ↪ actual_labels),source_id='annex3',group_date_cell=f'A{gr
                ↪ oup_row}',row_date_cell=f'A{r}',release_cell=f'B{r}',val
                ↪ ue_cell=f'{get_column_letter(c)}{r}'))
    tables['pv_forecasts.csv']=records
    expected_rows={'day_reference.csv':144,'actual_power.csv':52560,'actual_
    ↪ price.csv':52560,'pv_forecasts.csv':35040}
    assert {k:len(v) for k,v in tables.items()}==expected_rows
    out.mkdir(parents=True,exist_ok=False)
    outputs={name:write_csv(out,name,list(records[0]),records) for
    ↪ name,records in tables.items()}
    readme='''# 整理后的来源数据（prepared-v1）

按已批准的最小格式整理方案生成，当前是待用户确认的候选版本。
保留全部原始数值、来源位置和发布版本，没有训练、插值、能量换算或填写购电结果。

| 文件 | 每行表示什么 | 行数 | 数值单位 |
|---|---|---:|---|
| day_reference.csv | 附件1代表日的一个原始时间标签 | 144 | 功率kW；电价元/kWh |
| actual_power.csv | 一个来源日期和原时间标签的实际负载及光伏 | 52560 | kW |
| actual_price.csv | 一个来源日期和原时间标签的实际价格 | 52560 | 元/kWh |
| pv_forecasts.csv | 一次发布针对一个未来整点的光伏预报 | 35040 | kW |

## 时间字段

source_date 保留原表归属日；代表日没有日历日期，不假造日期。
source_minute 为距离归属日零点的分钟数：0:10 对应 10，0:00+1 对应 1440。
source_time_label保留原单元格值的文字表示：Excel时间转ISO文字，原文本不改；
↪ source_time_type保留原类型。
这只是来源标签，不说明功率代表前一段、后一段或平均值，不能直接把它当作已确认的购电区间。
CSV 保留日期和时间为明确文字、分钟数为整数。不要覆盖 CSV 后再声称原值未变。

## 预报字段

release_datetime是发布时刻；lead_hours为1—24；
↪ target_datetime等于发布时刻加提前小时数。
同一目标时刻的不同发布版本全部保留，不能按目标时刻去重或用较晚发布值覆盖较早发布。
```

```
date_inherited=1 表示原行日期空白,来自同组首行;原行日期及组首单元格都保留。
has_actual_target_label只表示目标标签是否在附件2中存在,
↪  不代表当时可提前获知其实际值。
全年最后实际标签是2025-12-31的0:00+1,即2026-01-01 00:00;36条预报晚于此时刻,
↪  仅标注而未删除。

## 信息身份与单位

actual_power 是实际观测;actual_price 是实际价格,在 Q4 中不可提前知道未来部分。
day_reference来自附件1,其负载/预测用于Q1和已确认的Q2首日启动参考;
↪  其电价用于固定日曲线场景。
pv_forecasts来自附件3,仅在相应发布时刻起可用,不能用于未经授权的Q2/Q4-2预报输入。
CSV 本身不强制这些权限,后续研究代码还需落实时间切片。
功率仍为 kW,价格仍为元/kWh;没有得到区间能量 kWh,未填补数值、裁剪、平滑、标准化或插值。

## 数值及来源

数值列直接保存 XLSX 内部的数值文本,没有按显示小数位四舍五入;重读时按数值解析。
source_id 在 manifest.json 的 sources 中映射到原文件与工作表。
所有 *_cell 列均为原工作表单元格地址;实际功率的 load_/pv_ 前缀分别指两张原表。
source_date_label/type保留原日期值表示及类型;
↪  source_row_date_label/type保留预报当前行的原日期空白。
source_release_label/type 保留预报发布时间的原值表示及类型。
slot_index 仅是附件 1 原行顺序索引,不是新定义的物理区间。

manifest.json 记录文件哈希、来源身份、行数、字段、生成脚本和授权。
逐值核对结果另存项目 audit/prepared_v1_verification.json,不是人工核验或数据冻结。

## 复现

在仓库根目录使用 Python 运行:
python code/prepare_source_tables.py --output tmp/prepared-v1-reproduction
只允许新目录;脚本拒绝覆盖已有输出或写入原始材料目录。
核验命令:python -X utf8 projects/2026-C/scripts/verify_prepared_tables.py
'''
    (out/'README.md').write_text(readme,encoding='utf-8')
    if any(sha(ROOT/p)!=h for p,h in expected.items()):
        raise RuntimeError('Original files changed during preparation')
```

```
    manifest={'version':'prepared-v1','status':'candidate_pending_user_confirmation','plan_version':'Data-plan-v1','authorization':'DATA-PREP-02','created_at':datetime.now().astimezone().isoformat(),'python':sys.version,'openpyxl':openpyxl.__version__,'script':'scripts/prepare_source_tables.py','script_sha256':sha(Path(__file__)),'raw_sha256':expected,'raw_unchanged_after_preparation':True,'sources':{sid:{'path':path,'sheet':sheet} for sid,(path,sheet) in SOURCES.items()},'outputs':outputs,'readme_sha256':sha(out/'README.md'),'transformations':{'original_numeric_values_changed':0,'original_numeric_cells':193152,'total_output_records':sum(expected_rows.values()),'forecast_rows_with_inherited_group_date':inherited_rows,'forecast_records_without_actual_target_label':sum(not r['has_actual_target_label'] for r in tables['pv_forecasts.csv']),'numeric_text':'copied verbatim from original XLSX numeric value elements','physical_interval_mapping':'not selected','hourly_interpolation':'not performed','power_to_energy':'not performed'},'information_roles':{'day_reference.csv':'attachment1_reference_not_Q2_future_actuals','actual_power.csv':'actual_observation_not_future_information','actual_price.csv':'Q4_actual_price_not_known_ahead','pv_forecasts.csv':'available_only_at_or_after_release_in_authorized_branches'}}
    (out/'manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding='utf-8')
    print(json.dumps({'output':str(out),'rows':expected_rows,'numeric_values_changed':0,'inherited_date_rows':inherited_rows,'forecast_outside_actual_labels':manifest['transformations']['forecast_records_without_actual_target_label']},ensure_ascii=False))


if __name__=='__main__':
    main()
```

D.2 计算输入重建

文件：code/prepare_inputs.py

```
"""Rebuild numerical inputs from the included, value-preserving CSV exports."""
from pathlib import Path
import numpy as np
import pandas as pd
ROOT=Path(__file__).resolve().parents[1]
def main():
 date=pd.Timestamp('2025-01-01');read=lambda n:pd.read_csv(ROOT/'data'/f'{n}.csv.gz',float_precision='round_trip')
 ref=read('day_reference');ref=ref.assign(t=ref.source_minute%1440).sort_values('t')
 a=read('actual_power');t=pd.to_datetime(a.source_date)+pd.to_timedelta(a.source_minute,unit='m')
 ix=((t-date).dt.total_seconds()/600).astype(int).to_numpy();assert np.array_equal(ix,np.arange(1,52561))
 actual=np.full((52561,2),np.nan);actual[ix]=a[['load_kw','pv_actual_kw']].to_numpy()
 f=read('pv_forecasts');release=np.arange(1460)*36;targets=release[:,None]+np.arange(1,25)[None,:]*6
 rr=((pd.to_datetime(f.release_datetime)-date).dt.total_seconds()/600).astype(int).to_numpy()
```

```
 tt=((pd.to_datetime(f.target_datetime)-date).dt.total_seconds()/600).astype(int).to_numpy()
 assert np.array_equal(rr,np.repeat(release,24)) and np.array_equal(tt,targets.ravel())
 p=read('actual_price');p['day']=(pd.to_datetime(p.source_date)-date).dt.days;p['slot']=p.source_minute.astype(int)%1440//10
 prices=p.pivot(index='day',columns='slot',values='price_yuan_per_kwh').reindex(index=range(365),columns=range(144)).to_numpy()
 data=dict(actual=actual,reference=ref[['load_kw','pv_forecast_kw']].to_numpy(),price=ref.price_yuan_per_kwh.to_numpy(),F=f.pv_forecast_kw.to_numpy().reshape(1460,24),release=release,targets=targets,actual_prices=prices)
 saved=np.load(ROOT/'data/inputs.npz')
 for k,v in data.items():np.testing.assert_array_equal(v,saved[k],err_msg=k)
 out=ROOT/'reproduced';out.mkdir(exist_ok=True);np.savez_compressed(out/'inputs.npz',**data)
 print('Prepared arrays match all saved inputs exactly.')
if __name__=='__main__':main()
```

D.3 数据探索性分析

文件：code/explore_data.py

```
"""Descriptive EDA of supplied labels; no forecasting, dispatch or energy integration."""
from pathlib import Path
import argparse
import hashlib
import json
import platform
import numpy as np
import pandas as pd

PROJECT = Path(__file__).resolve().parents[1]
SOURCE = PROJECT / 'data'
SIGNALS = ['load_kw', 'pv_actual_kw', 'net_load_kw', 'price_yuan_per_kwh']


def sha(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def stats(values):
    a = np.asarray(values, dtype=float)
    return dict(n=len(a), mean=float(a.mean()), std=float(a.std(ddof=1)),
                min=float(a.min()), p10=float(np.quantile(a,.1)),
                median=float(np.median(a)), p90=float(np.quantile(a,.9)),
                max=float(a.max()), zero_count=int((a==0).sum()))


def lagcorr(a, lag):
    a = np.asarray(a, dtype=float)
    return float(np.corrcoef(a[:-lag], a[lag:])[0,1])


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--output', default='reproduced/eda')
```

```
    args = ap.parse_args()
    out = (PROJECT / args.output).resolve()
    if not out.is_relative_to(PROJECT) or out.exists():
        raise ValueError('Use a new output directory inside the project.')
    raw_hashes = json.loads((SOURCE/'raw_hashes.json').read_text(encoding='utf-8'))
    input_hashes = {f.name: sha(f) for f in SOURCE.iterdir() if f.is_file()}
    ref = pd.read_csv(SOURCE/'day_reference.csv.gz')
    actual = pd.read_csv(SOURCE/'actual_power.csv.gz', parse_dates=['source_date'])
    price = pd.read_csv(SOURCE/'actual_price.csv.gz', parse_dates=['source_date'])
    forecast = pd.read_csv(SOURCE/'pv_forecasts.csv.gz', parse_dates=['release_datetime','target_datetime'])
    keys = ['source_date','source_minute']
    data = actual.merge(price[keys+['price_yuan_per_kwh']],on=keys,validate='one_to_one')
    data = data.sort_values(keys).reset_index(drop=True)
    assert len(data)==52560 and len(ref)==144 and len(forecast)==35040
    assert data.groupby('source_date').size().eq(144).all()
    data['timestamp'] = data.source_date + pd.to_timedelta(data.source_minute, unit='min')
    assert data.timestamp.diff().dropna().eq(pd.Timedelta(minutes=10)).all()
    data['month'] = data.source_date.dt.month
    data['weekday'] = data.source_date.dt.dayofweek
    data['net_load_kw'] = data.load_kw - data.pv_actual_kw
    data['pv_exceeds_load'] = data.net_load_kw < 0
    ref['net_load_kw'] = ref.load_kw-ref.pv_forecast_kw
    out.mkdir(parents=True)
    ref.to_csv(out/'reference_profile.csv',index=False)
    data[keys+['month','weekday']+SIGNALS].to_csv(out/'time_series.csv',index=False)
    daily = data.groupby('source_date')[SIGNALS].mean()
    daily['pv_exceeds_load_fraction'] = data.groupby('source_date').pv_exceeds_load.mean()
    daily['peak_net_load_kw'] = data.groupby('source_date').net_load_kw.max()
    daily['minimum_net_load_kw'] = data.groupby('source_date').net_load_kw.min()
    daily.to_csv(out/'daily_summary.csv')
    monthly = data.groupby('month')[SIGNALS].mean()
    monthly['days'] = data.groupby('month').source_date.nunique()
    monthly['pv_exceeds_load_fraction'] = data.groupby('month').pv_exceeds_load.mean()
    monthly.to_csv(out/'monthly_summary.csv')
    profile = data.groupby('source_minute')[SIGNALS].agg(['mean',lambda s:s.quantile(.1),lambda s:s.quantile(.9)])
    profile.columns = [f'{c}_{["mean","p10","p90"][i%3]}' for i,(c,_) in enumerate(profile.columns)]
    profile.to_csv(out/'intraday_summary.csv')
    mp = data.groupby(['month','source_minute'])[SIGNALS].mean()
    mp.to_csv(out/'month_intraday.csv')
    weekdays = data.groupby('weekday')[SIGNALS].mean()
    weekdays['days'] = data.groupby('weekday').source_date.nunique()
    weekdays.to_csv(out/'weekday_summary.csv')
    lag_rows = []
    summary_lags = {}
    for col in SIGNALS:
        # Subtract source-month x time-label mean only for retrospective diagnostics.
```

```
        residue = data[col]-data.groupby(['month','source_minute'])[col].transform('mean')
        summary_lags[col] = {str(h): {'raw':lagcorr(data[col],int(h*6)), 'month_time_residual':lagcorr(residue,int(h*6))} for h in [1,12,24,48,144,168,192,336]}
        for h in range(1,337):
            lag_rows.append(dict(signal=col,lag_hours=h,raw_correlation=lagcorr(data[col],h*6),residual_correlation=lagcorr(residue,h*6)))
    pd.DataFrame(lag_rows).to_csv(out/'lag_correlations.csv',index=False)
    matched = forecast.merge(data[['timestamp','pv_actual_kw']],left_on='target_datetime',right_on='timestamp',how='left',validate='many_to_one')
    assert matched.pv_actual_kw.isna().sum()==36
    matched['error_kw'] = matched.pv_forecast_kw-matched.pv_actual_kw
    matched['abs_error_kw'] = matched.error_kw.abs()
    matched['target_hour'] = matched.target_datetime.dt.hour
    matched['release_hour'] = matched.release_datetime.dt.hour
    valid = matched.dropna(subset=['pv_actual_kw']).copy()
    metrics=[]
    for name, frame in [('all',valid),('actual_positive',valid[valid.pv_actual_kw>0])]:
        for h,g in frame.groupby('lead_hours'):
            metrics.append(dict(scope=name,lead_hours=int(h),n=len(g),mae_kw=float(g.abs_error_kw.mean()),bias_kw=float(g.error_kw.mean()),rmse_kw=float(np.sqrt((g.error_kw**2).mean()))))
    pd.DataFrame(metrics).to_csv(out/'forecast_error_by_lead.csv',index=False)
    matched[['release_datetime','target_datetime','lead_hours','pv_forecast_kw','pv_actual_kw','error_kw','abs_error_kw','target_hour','release_hour']].to_csv(out/'forecast_matched.csv',index=False)
    # For each original midnight forecast targeting this day's h=7..24,
    # compare the original and the latest later published forecast strictly before target.
    initial = valid[(valid.release_hour==0)&(valid.lead_hours>=7)].copy()
    pairs=[]
    for r in initial.itertuples():
        pool=valid[(valid.target_datetime==r.target_datetime)&(valid.release_datetime>r.release_datetime)&(valid.release_datetime<r.target_datetime)]
        if pool.empty:
            continue
        new=pool.sort_values('release_datetime').iloc[-1]
        pairs.append(dict(target_datetime=r.target_datetime,actual_kw=r.pv_actual_kw,
                          initial_lead=r.lead_hours,latest_lead=int(new.lead_hours),
                          initial_forecast_kw=r.pv_forecast_kw,latest_forecast_kw=float(new.pv_forecast_kw),
                          initial_abs_error_kw=r.abs_error_kw,latest_abs_error_kw=float(new.abs_error_kw)))
    pairs=pd.DataFrame(pairs)
    pairs.to_csv(out/'forecast_same_target_pairs.csv',index=False)
    paired_stats={}
    for name,g in [('all',pairs),('actual_positive',pairs[pairs.actual_kw>0])]:
        diff=g.latest_abs_error_kw-g.initial_abs_error_kw
```

```
        paired_stats[name]=dict(n=len(g),initial_mae_kw=float(g.initial_abs_error_kw.mean()),latest_mae_kw=float(g.latest_abs_error_kw.mean()),improved_fraction=float((diff<0).mean()),worse_fraction=float((diff>0).mean()),tie_fraction=float((diff==0).mean()))
    def extreme_row(frame,col,which):
        r=frame.loc[getattr(frame[col],'idx'+which)()]
        return {'value':float(r[col]),'source_date':str(r.get('source_date','representative')),'source_minute':int(r.source_minute)}
    shapes={}
    for col in SIGNALS:
        matrix=data.pivot(index='source_date',columns='source_minute',values=col).to_numpy()
        corrs=np.array([np.corrcoef(matrix[i],matrix[i+1])[0,1] for i in range(364)])
        shapes[col]=stats(corrs)
    refstats={col:stats(ref[col]) for col in ['load_kw','pv_forecast_kw','net_load_kw','price_yuan_per_kwh']}
    refstats['positive_pv_labels'] = ref.loc[ref.pv_forecast_kw>0,'source_minute'].tolist()
    refstats['surplus_labels'] = ref.loc[ref.net_load_kw<0,'source_minute'].tolist()
    refstats['extrema']={col:{w:extreme_row(ref,col,w) for w in ['min','max']} for col in ['load_kw','pv_forecast_kw','net_load_kw','price_yuan_per_kwh']}
    price_spread=data.groupby('source_date').price_yuan_per_kwh.agg(['min','max'])
    price_spread['ratio']=price_spread['max']/price_spread['min']
    summary=dict(version='EDA-v1',scope='retrospective_descriptive_only',python=platform.python_version(),numpy=np.__version__,pandas=pd.__version__,script_sha256=sha(Path(__file__)),input_sha256=input_hashes,raw_sha256=raw_hashes,
                 reference=refstats,annual={c:stats(data[c]) for c in SIGNALS},
                 january_only={c:stats(data.loc[data.month==1,c]) for c in SIGNALS},
                 monthly=monthly.reset_index().to_dict('records'),lag_correlations=summary_lags,adjacent_day_shape_correlation=shapes,
                 annual_extrema={c:{w:extreme_row(data,c,w) for w in ['min','max']} for c in SIGNALS},
                 surplus=dict(sample_fraction=float(data.pv_exceeds_load.mean()),days_with_surplus=int((daily.pv_exceeds_load_fraction>0).sum()),max_surplus_kw=float(-data.net_load_kw.min())),
                 price_daily_max_min_ratio=stats(price_spread.ratio),forecast=dict(total=len(forecast),matched=len(valid),unmatched=36,actual_positive_count=int((valid.pv_actual_kw>0).sum()),mae_all_kw=float(valid.abs_error_kw.mean()),mae_actual_positive_kw=float(valid.loc[valid.pv_actual_kw>0,'abs_error_kw'].mean()),same_target=paired_stats),
                 caveats=['Source-label arithmetic means are not kWh or final interval semantics.','One year supports seasonal differences, not verified interannual periodicity.','Full-year diagnostics are hindsight; no future information is authorized for decisions.','Forecast metrics are at exact hourly target labels with no interpolation; economic value is not measured.','Month x time residuals are retrospective centering, not a trained forecasting model.'])
```

```
    assert input_hashes=={f.name:sha(f) for f in SOURCE.iterdir() if f.is_file()}
    # Raw file identifiers are retained; supplied CSV inputs are rechecked above.
    summary['source_unchanged']=True
    (out/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2,allow_nan=False)+'\n',encoding='utf-8')
    print(json.dumps({'output':str(out),'monthly':summary['monthly'],'surplus':summary['surplus'],'forecast':summary['forecast'],'lags':summary_lags},ensure_ascii=False,indent=2))


if __name__=='__main__':
    main()
```

D.4 问题一线性规划

文件：code/q1.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(obj,ensure_ascii=False,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
def arrays(inputs, config):
    dt=config['dt_hours']
    l=np.array([r['load_kw']*dt for r in inputs]); v=np.array([r['pv_kw']*dt for r in inputs])
    p=np.array([r['price'] for r in inputs])
    return l, v, p, np.maximum(l-v,0), np.maximum(v-l,0)

def terminal_bands(net, cfg):
```

```
    n=len(net); low=np.empty(n+1); high=np.empty(n+1)
    low[n]=high[n]=cfg['terminal_kwh']
    up=cfg['eta_charge']*cfg['external_charge_kw']*cfg['dt_hours']
    down=np.minimum(net,cfg['external_discharge_kw']*cfg['dt_hours'])/cfg['eta_discharge']
    for t in range(n-1,-1,-1):
        low[t]=max(cfg['min_kwh'],low[t+1]-up)
        high[t]=min(cfg['max_kwh'],high[t+1]+down[t])
    return low,high,up,down

def rule(inputs, cfg, low_q, high_q, target_fraction, reserve):
    l,v,p,net,excess=arrays(inputs,cfg); n=len(p)
    low_price=float(np.quantile(p,low_q)); high_price=float(np.quantile(p,high_q))
    cap_c=cfg['external_charge_kw']*cfg['dt_hours']; cap_d=cfg['external_discharge_kw']*cfg['dt_hours']
    eta_c=cfg['eta_charge']; eta_d=cfg['eta_discharge']; smin=cfg['min_kwh']; smax=cfg['max_kwh']
    target=smin+target_fraction*(smax-smin)
    lo,hi,up,down=terminal_bands(net,cfg)
    state=cfg['initial_kwh']; rows=[]
    for t in range(n):
        desired=0.; cap=target; reserve_amount=0.
        reason='idle'
        if excess[t]>1e-10:
            desired=min(eta_c*min(excess[t],cap_c),smax-state)
            reason='pv_charge'
        elif p[t]>=high_price:
            desired=-min(down[t],state-smin)
            reason='peak_discharge'
        elif p[t]<=low_price:
            future_prices=p[t+1:][net[t+1:]>1e-10]
            economical=len(future_prices)>0 and p[t]<eta_c*eta_d*float(future_prices.max())
            if reserve:
                future=np.flatnonzero(excess[t+1:]>1e-10)
                if len(future):
                    a=t+1+int(future[0]); b=a
                    while b<n and excess[b]>1e-10: b+=1
                    incoming=eta_c*float(np.minimum(excess[a:b],cap_c).sum())
                    release=float(down[t+1:a][p[t+1:a]>=high_price].sum())
                    reserve_amount=min(smax-smin,max(0.,incoming-release))
                    cap=min(target,max(smin,smax-reserve_amount))
            if economical:
                desired=min(up,max(0.,cap-state)); reason='low_price_charge'
        allowed_low=max(-down[t],lo[t+1]-state,smin-state)
        allowed_high=min(up,hi[t+1]-state,smax-state)
        assert allowed_low<=allowed_high+1e-7,
        (t,state,allowed_low,allowed_high)
        actual=float(np.clip(desired,allowed_low,allowed_high))
        c=max(actual,0.)/eta_c; discharge=max(-actual,0.)*eta_d
        cpv=min(c,excess[t]); cg=c-cpv
        before=state; state+=actual
        rows.append(dict(grid_charge=cg,pv_charge=cpv,discharge=discharge,state_start=before,state_end=state,
            action_reason=reason,desired_delta=desired,actual_delta=actual,
            terminal_projection=int(abs(actual-desired)>1e-7),
            reserve_kwh=reserve_amount,grid_charge_target=cap))
```

```
    return rows,dict(low_quantile=low_q,high_quantile=high_q,target_fraction=target_fraction,
        low_price=low_price,high_price=high_price,target_kwh=target)

def solve_lp(inputs,cfg):
    l,v,p,net,excess=arrays(inputs,cfg); n=len(p)
    # x = [grid_charge, pv_charge, load-side discharge, end storage]
    objective=np.r_[p,np.zeros(n),-p,np.zeros(n)]
    aeq=np.zeros((n,4*n)); beq=np.zeros(n)
    for t in range(n):
        aeq[t,t]=-cfg['eta_charge']; aeq[t,n+t]=-cfg['eta_charge']
        aeq[t,2*n+t]=1/cfg['eta_discharge']; aeq[t,3*n+t]=1
        if t: aeq[t,3*n+t-1]=-1
        else: beq[t]=cfg['initial_kwh']
    aub=np.zeros((n,4*n))
    for t in range(n): aub[t,t]=aub[t,n+t]=1
    cc=cfg['external_charge_kw']*cfg['dt_hours'];
    dc=cfg['external_discharge_kw']*cfg['dt_hours']
    lower=np.r_[np.zeros(3*n),np.full(n,cfg['min_kwh'])]
    upper=np.r_[np.full(n,cc),np.minimum(excess,cc),np.minimum(net,dc),np.full(n,cfg['max_kwh'])]
    lower[-1]=upper[-1]=cfg['terminal_kwh']
    before=time.perf_counter()
    result=linprog(objective,A_ub=aub,b_ub=np.full(n,cc),A_eq=aeq,b_eq=beq,
        bounds=list(zip(lower,upper)),method='highs',options=dict(time_limit=cfg['lp_time_limit_seconds'],
            primal_feasibility_tolerance=cfg['lp_feasibility_tolerance'],
            dual_feasibility_tolerance=cfg['lp_feasibility_tolerance']))
    elapsed=time.perf_counter()-before
    assert result.success, result.message
    x=result.x; base=float(p@net)
    dual=float(beq@result.eqlin.marginals+np.full(n,cc)@result.ineqlin.marginals+
        lower@result.lower.marginals+upper@result.upper.marginals+base)
    cert=dict(status=int(result.status),message=result.message,method='HiGHS LP',iterations=int(result.nit),
        time_seconds=elapsed,primal_cost=float(result.fun+base),dual_cost=dual,
        primal_dual_gap=float(result.fun+base-dual),objective=objective.tolist(),x=x.tolist(),
        equality_marginals=result.eqlin.marginals.tolist(),inequality_marginals=result.ineqlin.marginals.tolist(),
        lower_marginals=result.lower.marginals.tolist(),upper_marginals=result.upper.marginals.tolist())
    rows=[]
    for t in range(n):
        s0=cfg['initial_kwh'] if t==0 else x[3*n+t-1]
        rows.append(dict(grid_charge=float(x[t]),pv_charge=float(x[n+t]),discharge=float(x[2*n+t]),
            state_start=float(s0),state_end=float(x[3*n+t]),action_reason='whole_day_optimization',
            desired_delta=float(x[3*n+t]-s0),actual_delta=float(x[3*n+t]-s0),
            terminal_projection=0,reserve_kwh=0.,grid_charge_target=0.))
    return rows,cert

def schedule(inputs, cfg, actions):
    l,v,p,net,excess=arrays(inputs,cfg); records=[]
    for t,(r,a) in enumerate(zip(inputs,actions)):
```

```
        charge=a['grid_charge']+a['pv_charge'];
        ↪ q=net[t]-a['discharge']+a['grid_charge']
        rec=dict(r); rec.update(a)
        rec.update(load_kwh=float(l[t]),pv_kwh=float(v[t]),direct_pv_kwh=flo
        ↪ at(min(l[t],v[t])),
            grid_to_load_kwh=float(net[t]-a['discharge']),purchase_kwh=float
            ↪ (q),charge_kwh=float(charge),
            unused_pv_kwh=float(excess[t]-a['pv_charge']),
            loss_kwh=float((1-cfg['eta_charge'])*charge+(1/cfg['eta_discharg
            ↪ e']-1)*a['discharge']),
            cost_yuan=float(p[t]*q))
        records.append(rec)
    return records

def metrics(rows):
    totals={key:float(sum(r[key] for r in rows)) for key in
        ['cost_yuan','purchase_kwh','charge_kwh','discharge','pv_charge','gr
        ↪ id_charge','unused_pv_kwh','loss_kwh']}
    states=[rows[0]['state_start']]+[r['state_end'] for r in rows]
    totals.update(start_kwh=states[0],end_kwh=states[-1],min_kwh=min(states)
    ↪ ,max_kwh=max(states),
        max_charge_kw=max(r['charge_kwh']*6 for r in
        ↪ rows),max_discharge_kw=max(r['discharge']*6 for r in rows),
        simultaneous_slots=sum(r['charge_kwh']>1e-5 and r['discharge']>1e-5
        ↪ for r in rows),
        terminal_projection_slots=sum(r['terminal_projection'] for r in
        ↪ rows))
    return totals
```

D.5 问题二预测与跨日调度

文件：code/q2.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dump
 ↪ s(obj,ensure_ascii=False,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
```

```
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
class Data:
    def __init__(self,source):
        self.a={"load":source["actual"][:N,0].reshape(365,144),"pv":source["actual"][:N,1].reshape(365,144)}
        self.ref={"load":source["reference"][:,0],"pv":source["reference"][:,1]}
        self.price=source["price"]

    def refmean(self,var,d,h,mode,n):
        # Only target-day calendar is future-known; all observed values have dates < d.
        if mode=='weekday':idx=[d+h-7*j for j in range(1,n+1) if 0<=d+h-7*j<d]
        else:idx=list(range(max(0,d-n),d))
        fallback=False
        if not idx:idx=list(range(max(0,d-7),d));fallback=True
        assert idx and max(idx)<d
        z=self.a[var][idx];count=np.isfinite(z).sum(axis=0)
        ans=np.divide(np.nansum(z,axis=0),count,out=self.ref[var].copy(),where=count>0)
        meta={'variable':var,'decision_date':daystr(d),'horizon_day':h+1,'mode':mode,'requested_n':n,
              'history_dates':[daystr(i) for i in idx],'weight_per_available_value':'1/count_at_slot',
              'count_min':int(count.min()),'count_max':int(count.max()),
              'calendar_fallback':fallback,'reference_only_slots':np.flatnonzero(count==0).tolist(),
              'latest_history_interval_end':daystr(max(idx)+1)+' 00:00'}
        return ans,meta

    def history(self,d,w,pv_days):
        L=[];V=[];weights=[]
        for h in [0,1]:
            l,lm=self.refmean('load',d,h,'weekday',w)
            v,vm=self.refmean('pv',d,h,'recent',pv_days)
            L.append(l);V.append(v);weights.extend([lm,vm])
        return np.concatenate(L),np.concatenate(V),weights

    def truth(self,d,initialization=False):
        l=self.a['load'][d].copy();v=self.a['pv'][d].copy()
        if d==0 and initialization:l[0]=self.ref['load'][0];v[0]=self.ref['pv'][0]
        assert np.isfinite(l).all() and np.isfinite(v).all()
        return l,v

    def features(self,d,h,var):
        k=np.arange(144);t=2*np.pi*k/144;target=pd.Timestamp(daystr(d+h));dow=target.dayofweek
        cols=[np.sin(t),np.cos(t),np.sin(2*t),np.cos(2*t)]
        names=['time_sin1','time_cos1','time_sin2','time_cos2']
        for i in range(7):cols.append(np.full(144,int(dow==i)));names.append('weekday_'+str(i))
```

```
        cols.extend([np.full(144,h+1),np.full(144,np.sin(2*np.pi*target.dayofyear/365)),
                     np.full(144,np.cos(2*np.pi*target.dayofyear/365))])
        names.extend(['horizon_day','year_sin','year_cos'])
        for mode,n,name in [('recent',1,'recent_slot_last'),('recent',3,'recent_slot_mean3'),
                            ('recent',7,'recent_slot_mean7'),('weekday',1,'matching_week1'),('weekday',4,'matching_week4')]:
            value,_=self.refmean(var,d,h,mode,n);cols.append(value);names.append(name)
        cols.extend([np.full(144,np.nanmean(self.a[var][d-1])),
                     np.full(144,np.nanmean(self.a[var][max(0,d-7):d]))])
        names.extend(['last_daily_level','recent7_daily_level'])
        x=np.column_stack(cols);assert np.isfinite(x).all()
        return x,names

class Forecasts:
    def __init__(self,data,out,event,guard):
        self.data=data;self.out=out;self.event=event;self.guard=guard;self.models={};self.cache={};self.fitcount=0
        self.X={};self.y={};self.origins=[];self.targets=[]
        for var in ['load','pv']:
            xs=[];ys=[];orig=[];target=[]
            for d in range(2,365):
                for h in [0,1]:
                    if d+h>=365:continue
                    x,self.names=data.features(d,h,var);xs.append(x);ys.append(data.a[var][d+h])
                    orig.extend([d]*144);target.extend([d+h]*144)
            self.X[var]=np.vstack(xs);self.y[var]=np.concatenate(ys)
            self.origins=np.array(orig);self.targets=np.array(target)
        np.savez_compressed(out/'feature_bank.npz',load_X=self.X['load'],pv_X=self.X['pv'],
                            load_y=self.y['load'],pv_y=self.y['pv'],origins=self.origins,targets=self.targets)
        dump(out/'feature_names.json',self.names)
        (out/'models').mkdir()

    def fit(self,kind,param,d):
        fitid=f'{kind}-{param}-{daystr(d)}';modelpair={}
        eligible=(self.origins>=max(2,d-56))&(self.targets<d)
        assert eligible.any() and int(self.targets[eligible].max())<d
        for var in ['load','pv']:
            self.guard();x=self.X[var][eligible];y=self.y[var][eligible];start=time.perf_counter()
            scaler=None
            if kind=='T2':
                scaler=StandardScaler();xt=scaler.fit_transform(x);model=Ridge(alpha=float(param));model.fit(xt,y)
            else:
                model=HistGradientBoostingRegressor(max_depth=int(param),max_iter=200,learning_rate=.05,
                      min_samples_leaf=48,l2_regularization=1.,max_bins=63,early_stopping=False,random_state=2026)
                model.fit(x,y)
            elapsed=time.perf_counter()-start
            if elapsed>60:raise RuntimeError('Single fit budget exceeded')
            self.fitcount+=1
```

```
            joblib.dump({'model':model,'scaler':scaler,'feature_names':self.names,'cutoff':daystr(d)},
                        self.out/'models'/f'{fitid}-{var}.joblib',compress=3)
            record={'fit_id':fitid,'variable':var,'cutoff':daystr(d),'n_rows':int(eligible.sum()),
                    'history_origin_min':daystr(self.origins[eligible].min()),
                    'latest_target_end':daystr(self.targets[eligible].max()+1)+' 00:00','seconds':elapsed,
                    'sample_weight':'uniform1','hyperparameter':param,'model':kind}
            if kind=='T2':
                unscaled=model.coef_/scaler.scale_
                record.update(coefficients_standardized=model.coef_.tolist(),coefficients_original=unscaled.tolist(),
                    feature_means=scaler.mean_.tolist(),feature_scales=scaler.scale_.tolist(),
                    intercept_original=float(model.intercept_-np.dot(unscaled,scaler.mean_)),
                    intercept_standardized=float(model.intercept_))
            else:
                counts=np.zeros(len(self.names),dtype=int)
                for group in model._predictors:
                    for tree in group:
                        nodes=tree.nodes
                        for z in nodes[nodes['is_leaf']==0]:counts[int(z['feature_idx'])]+=1
                record.update(learning_rate=.05,iterations=int(model.n_iter_),split_feature_counts=counts.tolist(),
                              split_counts_are_not_weights_or_causal_importance=True)
            self.event('fit_log',record);modelpair[var]=(model,scaler)
        self.models[(kind,param)]=(d,fitid,modelpair)
        return self.models[(kind,param)]

    def get(self,kind,param,d,phase_start):
        key=(kind,str(param),d,phase_start)
        if key in self.cache:return self.cache[key]
        if kind=='T1':
            w,p=param;l,v,weights=self.data.history(d,w,p);fitid='equal-history'
            for row in weights:self.event('history_weights',dict(setting=f'w{w}-p{p}',**row))
        else:
            previous=self.models.get((kind,param))
            if previous is None or d==phase_start or d-previous[0]>=7:previous=self.fit(kind,param,d)
            fitday,fitid,models=previous
            assert fitday<=d
            forecasts=[]
            for var in ['load','pv']:
                x=np.vstack([self.data.features(d,h,var)[0] for h in [0,1]])
                model,scaler=models[var];pred=model.predict(scaler.transform(x) if scaler is not None else x)
                pred=np.maximum(pred,0.)
                if var=='pv':
                    support=np.any(self.data.a['pv'][max(0,d-28):d]>0,axis=0)
                    pred*=np.tile(support,2)
                forecasts.append(pred)
```

```
            l,v=forecasts
        assert np.isfinite(l).all() and np.isfinite(v).all() and
        ↪  min(l.min(),v.min())>=0
        self.event('forecast_registry',{'kind':kind,'parameter':str(param),'
        ↪  decision_date':daystr(d),
             'fit_id':fitid,'forecast_horizon_days':2,'latest_allowed_observ
             ↪  ation_end':daystr(d)+' 00:00',
             'prediction_sha256':hashlib.sha256(np.r_[l,v].tobytes()).hexdig
             ↪  est()})
        self.cache[key]=(l,v);return l,v

class Planner:
    def __init__(self,price,event,guard,hours=48,terminal=True):
        self.p=np.tile(price,hours//24);self.value=.9*price[0] if terminal
        ↪  else 0.;self.calls=0;self.event=event;self.guard=guard
        n=hours*6;self.n=n;eq=lil_matrix((n,4*n));ub=lil_matrix((n,4*n))
        for k in range(n):
            eq[k,k]=-.9;eq[k,n+k]=-.9;eq[k,2*n+k]=1/.9;eq[k,3*n+k]=1
            if k:eq[k,3*n+k-1]=-1
            ub[k,k]=ub[k,n+k]=1
        self.eq=eq.tocsr();self.ub=ub.tocsr();self.bub=np.full(n,5000/6)
        self.c=np.r_[self.p,np.zeros(n),-self.p,np.zeros(n)];self.c[-1]=-sel
        ↪  f.value

    def solve(self,L,V,state,margin,meta):
        self.guard();self.calls+=1
        if self.calls>8500:raise RuntimeError('LP hard budget exceeded')
        n=self.n;L=L[:n];V=V[:n];margin=margin[:n];net=np.maximum((L-V)/6+ma
        ↪  rgin,0);excess=np.maximum((V-L)/6-margin,0)
        lo=np.r_[np.zeros(3*n),np.full(n,1200.)]
        hi=np.r_[np.full(n,5000/6),np.minimum(excess,5000/6),np.minimum(net,
        ↪  5000/6),np.full(n,10800.)]
        beq=np.zeros(n);beq[0]=state;start=time.perf_counter()
        r=linprog(self.c,A_ub=self.ub,b_ub=self.bub,A_eq=self.eq,b_eq=beq,bo
        ↪  unds=np.column_stack((lo,hi)),
                  method='highs',options={'time_limit':10.,'primal_feasibili
                  ↪  ty_tolerance':1e-8,'dual_feasibility_tolerance':1e-8})
        elapsed=time.perf_counter()-start
        if not r.success:raise RuntimeError(r.message)
        x,y,z,s=np.split(r.x,4);q=net-z+x
        eqerr=np.max(np.abs(self.eq@r.x-beq));uberr=max(0.,np.max(self.ub@r.
        ↪  x-self.bub))
        berr=max(0.,np.max(lo-r.x),np.max(r.x-hi))
        dual=beq@r.eqlin.marginals+self.bub@r.ineqlin.marginals+lo@r.lower.m
        ↪  arginals+hi@r.upper.marginals
        gap=abs(r.fun-dual);sim=int(np.sum((x+y>1e-6)&(z>1e-6)))
        assert max(eqerr,uberr,berr)<1e-6 and gap<1e-4 and sim==0 and
        ↪  q.min()>-1e-6
        q=np.maximum(q,0)
        self.event('solver_checks',dict(**meta,status=int(r.status),seconds=
        ↪  elapsed,eq_residual=float(eqerr),
               inequality_residual=float(uberr),bound_residual=float(berr),p
               ↪  rimal_dual_gap=float(gap),
               simultaneous_slots=sim,objective=float(r.fun+self.p@net),init
               ↪  ial_storage=float(state),
               terminal_storage=float(s[-1]),terminal_value=float(self.value
               ↪  )))
        return q,x,y,z,s

```

```
def execute(L,V,q,initial,price):
    rows=[];state=float(initial)
    for k in range(144):
        load=float(L[k]/6);pv=float(V[k]/6);order=float(q[k]);m=min(load,pv);need=load-m;surplus=pv-m
        s0=state
        if need>order:
            z=min(need-order,5000/6,max(0.,.9*(state-1200)))
            e=max(0.,need-order-z);y=x=0.;received=order
        else:
            c=min(surplus+order-need,5000/6,max(0.,(10800-state)/.9))
            y=min(surplus,c);x=c-y;received=need+x;z=e=0.
        state+=.9*(x+y)-z/.9
        unused_order=max(0.,order-received);unused_pv=max(0.,surplus-y)
        bal=m+(received-x)+z+e-load
        assert abs(bal)<1e-6 and 1200-1e-6<=state<=10800+1e-6
        rows.append({'slot':k,'load_kw':float(L[k]),'pv_kw':float(V[k]),'price':float(price[k]),
                     'order_kwh':order,'received_kwh':received,'grid_to_load_kwh':received-x,
                     'direct_pv_kwh':m,'grid_charge_kwh':x,'pv_charge_kwh':y,'discharge_kwh':z,
                     'emergency_kwh':e,'unused_order_kwh':unused_order,'unused_pv_kwh':unused_pv,
                     'storage_start_kwh':s0,'storage_end_kwh':state,
                     'ordinary_cost':float(price[k]*order),'emergency_cost':float(5*price[k]*e),
                     'balance_residual':bal})
    return rows,state

def metrics(rows,Lhat,Vhat):
    df=pd.DataFrame(rows);r={n:float(df[n].sum()) for n in ['order_kwh','received_kwh','grid_charge_kwh',
          'pv_charge_kwh','discharge_kwh','emergency_kwh','unused_order_kwh','unused_pv_kwh','ordinary_cost','emergency_cost']}
    errorL=df.load_kw.to_numpy()-Lhat[:144];errorV=df.pv_kw.to_numpy()-Vhat[:144];pos=df.pv_kw.to_numpy()>0
    event=df.emergency_kwh.to_numpy()>1e-6
    longest=cur=0
    for yes in event:cur=cur+1 if yes else 0;longest=max(longest,cur)
    r.update(total_cost=r['ordinary_cost']+r['emergency_cost'],storage_start_kwh=rows[0]['storage_start_kwh'],
          storage_end_kwh=rows[-1]['storage_end_kwh'],emergency_slots=int(event.sum()),
          longest_emergency_minutes=int(longest*10),load_abs_error_sum=float(abs(errorL).sum()),
          pv_abs_error_sum=float(abs(errorV).sum()),pv_positive_abs_error_sum=float(abs(errorV[pos]).sum()),
          pv_positive_slots=int(pos.sum()),net_abs_error_sum=float(abs(errorL-errorV).sum()),
          load_squared_error_sum=float((errorL**2).sum()),pv_squared_error_sum=float((errorV**2).sum()))
    return r

class Risk:
    def __init__(self,data,base=(4,7)):self.data=data;self.base=base;self.errors={}
    def error(self,d):
        if d not in self.errors:
```

```
            l,v,_=self.data.history(d,*self.base);a,b=self.data.truth(d);sel
            ↪  f.errors[d]=((a-b)-(l[:144]-v[:144]))/6
        return self.errors[d]
    def margin(self,d,q=.7,days=28,group_hours=4,decay=None):
        ix=list(range(max(1,d-days),d));pool=np.array([self.error(i) for i in
        ↪  ix]);m=np.zeros(144);records=[]
        width=group_hours*6
        for j in range(0,144,width):
            vals=(pool[:,j:j+width] if len(ix)>=14 else pool).ravel()
            if len(ix)<7:raw=0.
            elif decay is None:raw=float(np.quantile(vals,q))
            else:
                w=np.repeat(decay**np.arange(len(ix)-1,-1,-1),width if
                ↪  len(ix)>=14 else 144)
                order=np.argsort(vals,kind='stable');cdf=np.cumsum(w[order])
                ↪  ;raw=float(vals[order][np.searchsorted(cdf,q*cdf[-1],sid
                ↪  e='left')])
            m[j:j+width]=max(0.,raw);records.append({'slot_start':j,'raw_qua
            ↪  ntile':raw,'value':max(0.,raw),'count':len(vals),'first':ix[
            ↪  0] if ix else None,'last':ix[-1] if ix else None})
        return np.tile(m,2),records
```

D.6　运行回放与共享计算

文件：code/q3_base.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dump
 ↪  s(obj,ensure_ascii=False,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
def history_and_features(actual,reference,F,release,targets):
```

```
    # Profile j only uses observations of that clock slot whose intervals ended by r.
    hist=np.empty((1460,144));last=np.empty(1460)
    for b,r in enumerate(release):
        last[b]=actual[r-1,1] if r>1 else reference[0,1]
        for k in range(144):
            q=r-1-((r-1-k)%144);ii=q-144*np.arange(7);ii=ii[ii>=1]
            hist[b,k]=np.mean(actual[ii,1]) if len(ii) else reference[k,1]
    L=np.empty((365,288))
    for d in range(365):
        for h in [0,1]:
            ds=[d+h-7*j for j in range(1,5) if 0<=d+h-7*j<d]
            if not ds:ds=list(range(max(0,d-7),d))
            if not ds:L[d,h*144:(h+1)*144]=reference[:,0];continue
            z=actual[np.array(ds)[:,None]*144+np.arange(144)[None,:],0]
            cnt=np.isfinite(z).sum(axis=0)
            L[d,h*144:(h+1)*144]=np.divide(np.nansum(z,axis=0),cnt,out=reference[:,0].copy(),where=cnt>0)
    H=np.take_along_axis(hist,targets%144,axis=1)
    hr=targets%144/6;year=targets//144+1
    X=np.stack([F,H,np.repeat(last[:,None],24,axis=1),np.repeat((release%144/6)[:,None],24,axis=1),
                np.tile(np.arange(1,25),(1460,1)),np.sin(2*np.pi*hr/24),np.cos(2*np.pi*hr/24),
                np.sin(2*np.pi*year/365),np.cos(2*np.pi*year/365)],axis=-1).reshape(-1,9)
    return hist,last,L,H,X

def build_fusion(actual,F,H,release,targets):
    rr=np.repeat(release,24);tt=targets.ravel();ff=F.ravel();hh=H.ravel()
    truth=np.full(len(tt),np.nan);valid=tt<N;truth[valid]=actual[tt[valid],1]
    w=np.ones((1460,4));counts=np.zeros((1460,4),int);fallback=np.zeros((1460,4),int);cut=np.full((1460,4),-1,int)
    leadgroup=np.tile(np.arange(24)//6,1460)
    for b,r in enumerate(release):
        pool=(rr>=r-28*144)&(rr<r)&(tt+1<=r)&(rr%144==r%144)&np.isfinite(truth)
        for g in range(4):
            use=pool&(leadgroup==g)
            if use.sum()<24:use=pool;fallback[b,g]=1
            counts[b,g]=use.sum()
            if use.any():cut[b,g]=int(np.max(tt[use]+1))
            a=ff[use]-hh[use];den=a@a
            if use.sum()<24 or den<1e-12:fallback[b,g]=2;continue
            w[b,g]=np.clip(a@(truth[use]-hh[use])/den,0,1)
    fused=np.repeat(w,6,axis=1)*F+(1-np.repeat(w,6,axis=1))*H
    return fused,w,counts,fallback,cut

def to_intervals(hourly,hist,last,release,zero_version=False):
    out=np.full((1460,288),np.nan)
    for b,r in enumerate(release):
        n=288-r%144;targets=r+np.arange(n);v=hist[b,targets%144].copy()
        if zero_version:
            midnight=(r//144)*144;ix=np.flatnonzero(midnight+np.arange(1,25)*6>r)
            xp=np.r_[r,midnight+(ix+1)*6];fp=np.r_[last[b],hourly[(r//144)*4,ix]]
            inside=targets<midnight+144
            v[inside]=np.interp(targets[inside],xp,fp)
```

```
        else:v[:144]=np.interp(np.arange(144),np.arange(0,145,6),np.r_[last[b],hourly[b]])
        out[b,:n]=v
    return out

class Planner:
    """Variables x,y,z,S,e,r,q,w; r is paid but unreceived normal energy."""
    def __init__(self,price,budget):self.price=price;self.budget=budget;self.cache={}
    def matrices(self,n,f):
        key=(n,f)
        if key in self.cache:return self.cache[key]
        eq=lil_matrix((2*n,8*n));ub=lil_matrix((2*n+2*f,8*n))
        for i in range(n):
            eq[i,i]=-.9;eq[i,n+i]=-.9;eq[i,2*n+i]=1/.9;eq[i,3*n+i]=1
            if i:eq[i,3*n+i-1]=-1
            eq[n+i,6*n+i]=1;eq[n+i,i]=-1;eq[n+i,2*n+i]=1;eq[n+i,4*n+i]=1;eq[n+i,5*n+i]=-1
            ub[i,i]=ub[i,n+i]=1;ub[n+i,2*n+i]=ub[n+i,4*n+i]=1
        for i in range(f):
            ub[2*n+i,6*n+i]=1;ub[2*n+i,7*n+i]=-1
            ub[2*n+f+i,6*n+i]=-1;ub[2*n+f+i,7*n+i]=-1
        self.cache[key]=eq.tocsr(),ub.tocsr();return self.cache[key]
    def solve(self,L,V,initial,margin,slot,q0=None,fixed=None,terminal=True):
        n=len(L);f=0 if q0 is None else min(144-slot,n);eq,ub=self.matrices(n,f)
        p=self.price[(slot+np.arange(n))%144];net=np.maximum((L-V)/6+margin,0);ex=np.maximum((V-L)/6-margin,0)
        c=np.zeros(8*n);c[4*n:5*n]=5*p;c[6*n:7*n]=p;c[7*n:7*n+f]=.5*p[:f]
        if terminal:c[4*n-1]=-.9*self.price[0]
        lo=np.zeros(8*n);hi=np.full(8*n,np.inf)
        hi[:n]=CAP;hi[n:2*n]=np.minimum(ex,CAP);hi[2*n:3*n]=CAP;lo[3*n:4*n]=1200;hi[3*n:4*n]=10800
        hi[7*n+f:]=0
        if fixed is not None:lo[6*n:7*n]=fixed;hi[6*n:7*n]=fixed
        beq=np.r_[np.zeros(n),net];beq[0]=initial
        bu=np.r_[np.full(n,CAP),net,([] if f==0 else q0[:f]),([] if f==0 else -q0[:f])]
        self.budget.calls+=1;self.budget.check();start=time.perf_counter()
        ans=linprog(c,A_ub=ub,b_ub=bu,A_eq=eq,b_eq=beq,bounds=np.c_[lo,hi],method='highs',
              options={'time_limit':10.,'primal_feasibility_tolerance':1e-8,'dual_feasibility_tolerance':1e-8})
        if not ans.success:raise RuntimeError(ans.message)
        err=max(float(np.max(abs(eq@ans.x-beq))),float(np.max(ub@ans.x-bu)),float(np.max(lo-ans.x)),float(np.max(ans.x-hi)))
        finite=np.isfinite(hi)
        dual=beq@ans.eqlin.marginals+bu@ans.ineqlin.marginals+lo@ans.lower.marginals+hi[finite]@ans.upper.marginals[finite]
        gap=abs(ans.fun-dual);z=ans.x.reshape(8,n)
        simultaneous=int(np.sum((z[0]+z[1]>1e-6)&(z[2]>1e-6)))
        assert err<1e-6 and gap<1e-4,(err,gap)
        # Fixed-order feasibility diagnostics may be degenerate; actual execution is independently causal.
        if fixed is None:assert simultaneous==0, 'planned simultaneous charge/discharge'
        check=dict(n=n,f=f,initial=float(initial),objective=float(ans.fun),residual=err,dual_gap=float(gap),
```

```
                    simultaneous=simultaneous,seconds=time.perf_counter()-sta
                    ↪ rt,status=int(ans.status))
        return z,check

def execute_block(L,V,q,initial):
    # Columns retain physical flows separately from paid order and
    ↪ settlement.
    out=np.empty((len(q),12));s=float(initial)
    for k in range(len(q)):
        l=float(L[k]/6);v=float(V[k]/6);order=max(0.,float(q[k]));direct=min
        ↪ (l,v);need=l-direct;surplus=v-direct;s0=s
        if need>order:
            z=min(need-order,CAP,max(0.,.9*(s-1200)));e=max(0.,need-order-z)
            ↪ ;x=y=0.;received=order
        else:
            ch=min(surplus+order-need,CAP,max(0.,(10800-s)/.9));y=min(surplu
            ↪ s,ch);x=ch-y;received=need+x;z=e=0.
        s+=.9*(x+y)-z/.9
        out[k]=[s0,s,received,received-x,direct,x,y,z,e,max(0.,order-receive
        ↪ d),max(0.,surplus-y),l]
        assert 1200-1e-6<=s<=10800+1e-6 and abs(received-x+direct+z+e-l)<1e-6
    return out,s
```

D.7 问题三日内滚动调度

文件：code/q3.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dump
 ↪ s(obj,ensure_ascii=False,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
```

```
import q3_base as base
class Planner(base.Planner):
    def solve(self,L,V,initial,margin,slot,q0=None,fixed=None,terminal=True,floor_ratio=0.):
        n=len(L);f=0 if q0 is None else min(144-slot,n);eq,ub=self.matrices(n,f)
        p=self.price[(slot+np.arange(n))%144];net=np.maximum((L-V)/6+margin,0);sur=np.maximum(-((L-V)/6+margin),0)
        c=np.zeros(8*n);c[4*n:5*n]=5*p;c[6*n:7*n]=p;c[7*n:7*n+f]=.5*p[:f]
        if terminal:c[4*n-1]=-.9*self.price[0]
        lo=np.zeros(8*n);hi=np.full(8*n,np.inf)
        hi[:n]=CAP;hi[n:2*n]=np.minimum(sur,CAP);hi[2*n:3*n]=CAP;lo[3*n:4*n]=1200;hi[3*n:4*n]=10800
        hi[7*n+f:]=0
        if f and floor_ratio>0:lo[6*n:6*n+f]=floor_ratio*np.asarray(q0[:f])
        if fixed is not None:lo[6*n:7*n]=fixed;hi[6*n:7*n]=fixed
        beq=np.r_[np.zeros(n),net];beq[0]=initial
        bu=np.r_[np.full(n,CAP),net,([] if f==0 else q0[:f]),([] if f==0 else -q0[:f])]
        self.budget.calls+=1;self.budget.check();start=time.perf_counter()
        ans=linprog(c,A_ub=ub,A_eq=eq,b_ub=bu,b_eq=beq,bounds=np.c_[lo,hi],method='highs',
            options={'time_limit':10.,'primal_feasibility_tolerance':1e-8,'dual_feasibility_tolerance':1e-8})
        if not ans.success:raise RuntimeError(ans.message)
        err=max(float(np.max(abs(eq@ans.x-beq))),float(np.max(ub@ans.x-bu)),float(np.max(lo-ans.x)),float(np.max(ans.x-hi)))
        finite=np.isfinite(hi)
        dual=beq@ans.eqlin.marginals+bu@ans.ineqlin.marginals+lo@ans.lower.marginals+hi[finite]@ans.upper.marginals[finite]
        gap=abs(ans.fun-dual);z=ans.x.reshape(8,n)
        simultaneous=int(np.sum((z[0]+z[1]>1e-6)&(z[2]>1e-6)))
        assert err<1e-6 and gap<1e-4 and simultaneous==0
        return z,dict(n=n,f=f,initial=float(initial),objective=float(ans.fun),residual=err,dual_gap=float(gap),
            simultaneous=simultaneous,seconds=time.perf_counter()-start,status=int(ans.status),floor_ratio=float(floor_ratio))

def exact_quantile(values,ages,q,beta):
    from fractions import Fraction
    qf=Fraction(str(q));bf=Fraction(str(beta));age_max=int(np.max(ages))
    weights=[bf.numerator**int(a)*bf.denominator**(age_max-int(a)) for a in ages]
    total=sum(weights);acc=0
    for i,w in enumerate(weights):
        acc+=w
        if acc*qf.denominator>=qf.numerator*total:return max(0.,float(values[i]))
    return max(0.,float(values[-1]))
def make_load(kind,gain):
    residual=src['actual'][:365*144,0].reshape(365,144)-fc['L'][:,:144]
    load=np.full((1460,288),np.nan);theta=np.zeros((365,4,8));obs=np.zeros((365,4));count=np.zeros((365,4,8),int)
    for d in range(365):
        for a,slot in enumerate((0,36,72,108)):
            b=d*4+a;n=288-slot;load[b,:n]=fc['L'][d,slot:]
            if not slot:continue
            start=0 if 'PREFIX' in kind else slot-18
            current=residual[d,start:slot]
```

```
            observed=float(np.nanmean(current)) if np.isfinite(current).any() else 0.;obs[d,a]=observed
            past=np.arange(max(0,d-28),d)
            X=np.array([np.nanmean(residual[j,start:slot]) for j in past])
            for g in range(slot//18,8):
                Y=np.array([np.mean(residual[j,g*18:(g+1)*18]) for j in past])
                valid=np.isfinite(X)&np.isfinite(Y);x=X[valid];y=Y[valid];count[d,a,g]=len(x)
                if kind=='PERSIST_PREFIX':coef=1.
                else:coef=float(np.clip((x@y)/(x@x),0,1)) if len(x)>=7 and x@x>1e-12 else 0.
                theta[d,a,g]=coef
                left=g*18-slot;right=left+18
                load[b,left:right]=np.maximum(0,load[b,left:right]+gain*coef*observed)
    return load,theta,obs,count

def errors_for(load,pv):
    arrays={k:[] for k in ('release','target','error','kind')}
    for b,r in enumerate(src['release']):
        n=288-r%144;t=r+np.arange(n);ok=(t>0)&(t<365*144);truth=src['actual'][t[ok]]
        arrays['release'].append(np.full(ok.sum(),r));arrays['target'].append(t[ok]);arrays['kind'].append((np.arange(n)[ok]>=144).astype(int))
        arrays['error'].append(((truth[:,0]-truth[:,1])-(load[b,:n][ok]-pv[b,:n][ok]))/6)
    return {k:np.concatenate(v) for k,v in arrays.items()}
def midnight(family):
 if family=='W4':return orig['L'].copy()
 weeks=1 if family=='W1' else 2 if family=='W2' else 4
 decay=.5 if family=='EW4' else 1.
 L=np.empty((365,288))
 for d in range(365):
  for h in (0,1):
   ds=[d+h-7*j for j in range(1,weeks+1) if 0<=d+h-7*j<d]
   if not ds:ds=list(range(max(0,d-7),d));weights=np.ones(len(ds))
   else:weights=decay**np.arange(len(ds))
   if not ds:L[d,h*144:(h+1)*144]=src['reference'][:,0];continue
   z=src['actual'][np.asarray(ds)[:,None]*144+np.arange(144),0];valid=np.isfinite(z)
   den=(weights[:,None]*valid).sum(0);num=(weights[:,None]*np.nan_to_num(z)).sum(0)
   L[d,h*144:(h+1)*144]=np.divide(num,den,out=src['reference'][:,0].copy(),where=den>0)
 return L

def group_index(err):
 global group_cache
 if group_cache is not None:return group_cache
 rr,tt,kind=(err[k] for k in ('release','target','kind'));bank=[]
 for b,r in enumerate(src['release']):
  n=288-r%144;t=r+np.arange(n);pool=np.flatnonzero((rr>=r-4032)&(rr<r)&(tt+1<=r)&(rr%144==r%144))
  for typ in (0,1):
   same=pool[kind[pool]==typ]
   for g in range(8):
    dest=np.flatnonzero((t%144//18==g)&((np.arange(n)>=144)==bool(typ)))
```

```
    if not len(dest):continue
    ids=same[tt[same]%144//18==g];fallback=0
    if len(ids)<24:ids=same;fallback=1
    bank.append((b,typ,g,dest,ids,(r-1)//144-tt[ids]//144,fallback))
 group_cache=bank;return bank

def quantile(values,ages,q):
 from fractions import Fraction
 order=np.argsort(values,kind='stable');values=values[order];ages=ages[order];m=int(ages.max())
 iw=[7**a*10**(m-a) for a in range(m+1)];hist=np.bincount(ages,minlength=m+1)
 total=sum(int(h)*w for h,w in zip(hist,iw));f=Fraction(str(q))
 cdf=np.cumsum(.7**ages);k=min(len(values)-1,int(np.searchsorted(cdf/cdf[-1],q)))
 histpre=np.bincount(ages[:k+1],minlength=m+1);acc=sum(int(h)*w for h,w in zip(histpre,iw))
 while k>0 and f.denominator*(acc-iw[int(ages[k])])>=f.numerator*total:acc-=iw[int(ages[k])];k-=1
 while f.denominator*acc<f.numerator*total:k+=1;acc+=iw[int(ages[k])]
 return max(0.,float(values[k]))
```

D.8 电价预测与调度基础

文件：code/q4.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
EXEC_COLUMNS=['storage_start','storage_end','received','grid_load','pv_load','grid_charge','pv_charge','discharge','emergency','unused_order','unused_pv','load_energy']
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(obj,ensure_ascii=False,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
```

```
budget=Budget();logs=Logs()
class Q2Risk:
    def __init__(self, data):
        self.data = data
        self.errors = {}

    def error(self, day):
        if day not in self.errors:
            load_hat, pv_hat, _ = self.data.history(day, 4, 7)
            load, pv = self.data.truth(day)
            self.errors[day] = ((load - pv) - (load_hat[:SLOTS] -
            ↪  pv_hat[:SLOTS])) / 6
        return self.errors[day]

    def margin(self, day, q=0.7, history_days=28, group_hours=4, decay=0.7):
        indices = list(range(max(1, day - history_days), day))
        pool = np.array([self.error(index) for index in indices])
        result = np.zeros(SLOTS)
        width = group_hours * 6
        records = []
        for start in range(0, SLOTS, width):
            values = (pool[:, start:start + width] if len(indices) >= 14 else
            ↪  pool).ravel()
            if len(indices) < 7:
                raw = 0.0
                count = int(values.size)
            else:
                weights = np.repeat(
                    decay ** np.arange(len(indices) - 1, -1, -1),
                    width if len(indices) >= 14 else SLOTS,
                )
                order = np.argsort(values, kind="stable")
                cdf = np.cumsum(weights[order])
                raw = float(values[order][np.searchsorted(cdf, q * cdf[-1],
                ↪  side="left")])
                count = int(values.size)
            result[start:start + width] = max(0.0, raw)
            records.append({"slot_start": start, "value": max(0.0, raw),
            ↪  "count": count})
        return np.tile(result, 2), records

class PriceFeatures:
    FEATURE_NAMES = [
        "attachment1_base", "matching_weekday_median4", "recent7_median",
        ↪  "lag7_price",
        "predicted_net_load_1000kw", "current_day_level",
        ↪  "current_level_decay6h",
        "last_residual_decay1h", "lead_hours", "season_sin", "season_cos",
        *[f"weekday_{i}" for i in range(7)], "release_fraction",
    ]

    def __init__(self, prices, base, supply, branch):
        self.prices = prices
        self.base = base
        self.supply = supply
        self.branch = branch
        self.releases = [0] if branch == "Q4-2" else [0, 36, 72, 108]
        self._history_cache = {}

```

```
    def supply_forecast(self, day, slot):
        if self.branch == "Q4-2":
            target = day * SLOTS + np.arange(288)
            net = self.supply["load"][day] - self.supply["pv"][day]
        else:
            batch = day * 4 + slot // 36
            n = 288 - slot
            target = day * SLOTS + slot + np.arange(n)
            net = self.supply["load"][day, slot:slot + n] -
            ↪  self.supply["pv"][batch, :n]
        return target.astype(int), np.asarray(net, dtype=float)

    def history_curve(self, decision_day, target_day):
        key = (int(decision_day), int(target_day))
        if key in self._history_cache:
            return self._history_cache[key]
        matching = [target_day - 7 * lag for lag in range(1, 5) if 0 <=
        ↪  target_day - 7 * lag < decision_day]
        if matching:
            curve = np.median(self.prices[matching], axis=0)
            source = "matching_weekday"
        else:
            recent = list(range(max(0, decision_day - 7), decision_day))
            curve = np.median(self.prices[recent], axis=0) if recent else
            ↪  self.base.copy()
            source = "recent_or_attachment1"
        self._history_cache[key] = (curve, matching, source)
        return self._history_cache[key]

    def simple(self, day, slot, targets=None):
        if targets is None:
            targets, _ = self.supply_forecast(day, slot)
        slots = targets % SLOTS
        days = targets // SLOTS
        out = np.empty(len(targets))
        for target_day in np.unique(days):
            mask = days == target_day
            curve, _, _ = self.history_curve(day, int(target_day))
            out[mask] = curve[slots[mask]]
        if slot > 0:
            current_curve, _, _ = self.history_curve(day, day)
            level = float(np.median(self.prices[day, :slot] -
            ↪  current_curve[:slot]))
            out[days == day] += level
        return np.maximum(out, 0.001)

    def matrix(self, day, slot, targets, predicted_net):
        target_slots = targets % SLOTS
        target_days = targets // SLOTS
        same = np.empty(len(targets))
        recent = np.empty(len(targets))
        lag7 = np.empty(len(targets))
        for target_day in np.unique(target_days):
            mask = target_days == target_day
            slots = target_slots[mask]
            curve, _, _ = self.history_curve(day, int(target_day))
            same[mask] = curve[slots]
            recent_days = list(range(max(0, day - 7), day))
```

```
            recent[mask] = np.median(self.prices[recent_days][:, slots], axis=0) if recent_days else self.base[slots]
            lag_day = int(target_day) - 7
            lag7[mask] = self.prices[lag_day, slots] if 0 <= lag_day < day else curve[slots]

        current_level = 0.0
        last_residual = 0.0
        if slot > 0:
            current_curve, _, _ = self.history_curve(day, day)
            residual = self.prices[day, :slot] - current_curve[:slot]
            current_level = float(np.median(residual))
            last_residual = float(residual[-1])
        lead_hours = (targets - (day * SLOTS + slot)) / 6
        same_day = (target_days == day).astype(float)
        current = current_level * same_day
        current_decay = current_level * np.exp(-lead_hours / 6) * same_day
        last_decay = last_residual * np.exp(-lead_hours) * same_day
        dates = DATE + pd.to_timedelta(target_days, unit="D")
        day_of_year = np.asarray(dates.dayofyear, dtype=float)
        weekday = np.asarray(dates.dayofweek, dtype=int)
        columns = [
            self.base[target_slots], same, recent, lag7, predicted_net / 1000,
            current, current_decay, last_decay, lead_hours,
            np.sin(2 * np.pi * day_of_year / 365), np.cos(2 * np.pi * day_of_year / 365),
        ]
        columns.extend((weekday == index).astype(float) for index in range(7))
        columns.append(np.full(len(targets), slot / 108 if slot else 0.0))
        matrix = np.column_stack(columns)
        assert matrix.shape[1] == len(self.FEATURE_NAMES) and np.isfinite(matrix).all()
        return matrix

class PriceModels:
    def __init__(self, features, branch):
        self.features = features
        self.branch = branch
        self.datasets = {}
        self.models = {}
        self.model_dir = O / "price_models" / branch.replace("-", "_")
        self.model_dir.mkdir(parents=True, exist_ok=True)

    def training(self, cutoff_day):
        if cutoff_day in self.datasets:
            return self.datasets[cutoff_day]
        rows, targets = [], []
        for origin_day in range(max(1, cutoff_day - 56), cutoff_day):
            for release in self.features.releases:
                absolute, net = self.features.supply_forecast(origin_day, release)
                valid = (absolute < cutoff_day * SLOTS) & (absolute < N_DAYS * SLOTS)
                if not valid.any():
                    continue
                rows.append(self.features.matrix(origin_day, release, absolute[valid], net[valid]))
```

```
                targets.append(self.features.prices.reshape(-1)[absolute[val
                ↪  id]])
        X = np.vstack(rows)
        y = np.concatenate(targets)
        assert len(y) >= 500 and np.isfinite(X).all() and
        ↪  np.isfinite(y).all()
        self.datasets[cutoff_day] = (X, y)
        return X, y

    def fit(self, candidate, cutoff_day):
        key = (candidate, cutoff_day)
        if key in self.models:
            return self.models[key]
        X, y = self.training(cutoff_day)
        start = time.perf_counter()
        budget.price_fits += 1
        budget.check()
        if candidate == "P2":
            scaler = StandardScaler()
            scaled = scaler.fit_transform(X)
            model = Ridge(alpha=10.0)
            with threadpool_limits(limits=1):
                model.fit(scaled, y)
            original_coef = model.coef_ / scaler.scale_
            record = {
                "coefficients_original":
                ↪  dict(zip(self.features.FEATURE_NAMES,
                ↪  original_coef.astype(float))),
                "intercept_original": float(model.intercept_ - original_coef
                ↪  @ scaler.mean_),
                "alpha": 10.0,
            }
        else:
            scaler = None
            model = HistGradientBoostingRegressor(
                max_leaf_nodes=15, max_iter=100, learning_rate=0.05,
                min_samples_leaf=50, l2_regularization=1.0,
                early_stopping=False, random_state=2026,
            )
            with threadpool_limits(limits=1):
                model.fit(X, y)
            record = {
                "max_leaf_nodes": 15, "iterations": int(model.n_iter_),
                "learning_rate": 0.05, "min_samples_leaf": 50,
                ↪  "random_state": 2026,
            }
        elapsed = time.perf_counter() - start
        path = self.model_dir / f"{candidate}-cutoff-{cutoff_day:03d}.joblib"
        joblib.dump({"model": model, "scaler": scaler, "features":
        ↪  self.features.FEATURE_NAMES}, path, compress=3)
        record.update({
            "branch": self.branch, "candidate": candidate, "cutoff_day":
            ↪  int(cutoff_day),
            "cutoff_datetime": daystr(cutoff_day) + " 00:00",
            ↪  "training_rows": int(len(y)),
            "training_origin_start": daystr(max(1, cutoff_day - 56)),
            "latest_target_end": daystr(cutoff_day) + " 00:00",
            "seconds": elapsed, "model_path": str(path.relative_to(R)),
            ↪  "model_sha256": sha(path),
```

```
        })
        logs.event("price_fit_log", record)
        self.models[key] = (model, scaler)
        budget.save()
        return self.models[key]

    def predict(self, candidate, day, slot):
        targets, net = self.features.supply_forecast(day, slot)
        if candidate == "P1" or day < 7:
            result = self.features.simple(day, slot, targets)
            fit_cutoff = None
            fallback = candidate != "P1"
        else:
            cutoff = 7 + ((day - 7) // 7) * 7
            model, scaler = self.fit(candidate, cutoff)
            X = self.features.matrix(day, slot, targets, net)
            with threadpool_limits(limits=1):
                result = model.predict(scaler.transform(X) if scaler is not
                ↪  None else X)
            result = np.maximum(result, 0.001)
            fit_cutoff = cutoff
            fallback = False
        logs.event("price_forecast_log", {
            "branch": self.branch, "candidate": candidate, "decision_day":
            ↪  int(day),
            "decision_date": daystr(day), "release_slot": int(slot),
            "known_price_intervals_today": int(slot), "fit_cutoff_day":
            ↪  fit_cutoff,
            "early_history_fallback": fallback, "forecast_points":
            ↪  int(len(result)),
            "minimum": float(np.min(result)), "maximum":
            ↪  float(np.max(result)),
            "sha256": hashlib.sha256(np.asarray(result,
            ↪  dtype=np.float64).tobytes()).hexdigest(),
        })
        return result

class Q42Planner:
    def __init__(self, terminal_value):
        n = 288
        self.n = n
        self.terminal_value = float(terminal_value)
        eq = lil_matrix((n, 4 * n))
        ub = lil_matrix((n, 4 * n))
        for k in range(n):
            eq[k, k] = -0.9
            eq[k, n + k] = -0.9
            eq[k, 2 * n + k] = 1 / 0.9
            eq[k, 3 * n + k] = 1
            if k:
                eq[k, 3 * n + k - 1] = -1
            ub[k, k] = ub[k, n + k] = 1
        self.eq = eq.tocsr()
        self.ub = ub.tocsr()
        self.bub = np.full(n, CAP)

    def solve(self, load, pv, state, margin, price, meta):
        budget.lp_calls += 1
        budget.check()
```

```
        n = self.n
        net = np.maximum((load[:n] - pv[:n]) / 6 + margin[:n], 0)
        excess = np.maximum((pv[:n] - load[:n]) / 6 - margin[:n], 0)
        lower = np.r_[np.zeros(3 * n), np.full(n, 1200.0)]
        upper = np.r_[np.full(n, CAP), np.minimum(excess, CAP),
        ↪  np.minimum(net, CAP), np.full(n, 10800.0)]
        beq = np.zeros(n)
        beq[0] = state
        objective = np.r_[price[:n], np.zeros(n), -price[:n], np.zeros(n)]
        objective[-1] = -self.terminal_value
        start = time.perf_counter()
        result = linprog(
            objective, A_ub=self.ub, b_ub=self.bub, A_eq=self.eq, b_eq=beq,
            bounds=np.column_stack((lower, upper)), method="highs",
            options={"time_limit": 10.0, "primal_feasibility_tolerance":
            ↪  1e-8, "dual_feasibility_tolerance": 1e-8},
        )
        if not result.success:
            raise RuntimeError(result.message)
        x, y, z, storage = np.split(result.x, 4)
        order = np.maximum(net - z + x, 0)
        residual = max(
            float(np.max(np.abs(self.eq @ result.x - beq))),
            float(np.max(self.ub @ result.x - self.bub)),
            float(np.max(lower - result.x)), float(np.max(result.x - upper)),
        )
        finite = np.isfinite(upper)
        dual = (beq @ result.eqlin.marginals + self.bub @
        ↪  result.ineqlin.marginals
                + lower @ result.lower.marginals + upper[finite] @
                ↪  result.upper.marginals[finite])
        gap = abs(result.fun - dual)
        simultaneous = int(np.sum((x + y > 1e-6) & (z > 1e-6)))
        assert residual < 1e-6 and gap < 1e-4 and simultaneous == 0
        logs.event("solver_checks", dict(meta, residual=residual,
        ↪  dual_gap=float(gap),
                                         simultaneous=simultaneous,
                                         ↪  status=int(result.status),
                                         seconds=time.perf_counter() -
                                         ↪  start))
        return order

class Q43Planner:
    def __init__(self, terminal_value):
        self.terminal_value = float(terminal_value)
        self.cache = {}

    def matrices(self, n, fixed_count):
        key = (n, fixed_count)
        if key in self.cache:
            return self.cache[key]
        eq = lil_matrix((2 * n, 8 * n))
        ub = lil_matrix((2 * n + 2 * fixed_count, 8 * n))
        for i in range(n):
            eq[i, i] = -0.9
            eq[i, n + i] = -0.9
            eq[i, 2 * n + i] = 1 / 0.9
            eq[i, 3 * n + i] = 1
            if i:
```

```
                eq[i, 3 * n + i - 1] = -1
            eq[n + i, 6 * n + i] = 1
            eq[n + i, i] = -1
            eq[n + i, 2 * n + i] = 1
            eq[n + i, 4 * n + i] = 1
            eq[n + i, 5 * n + i] = -1
            ub[i, i] = ub[i, n + i] = 1
            ub[n + i, 2 * n + i] = ub[n + i, 4 * n + i] = 1
        for i in range(fixed_count):
            ub[2 * n + i, 6 * n + i] = 1
            ub[2 * n + i, 7 * n + i] = -1
            ub[2 * n + fixed_count + i, 6 * n + i] = -1
            ub[2 * n + fixed_count + i, 7 * n + i] = -1
        self.cache[key] = eq.tocsr(), ub.tocsr()
        return self.cache[key]

    def solve(self, load, pv, state, margin, slot, q0, price, meta):
        budget.lp_calls += 1
        budget.check()
        n = len(load)
        fixed_count = 0 if q0 is None else min(SLOTS - slot, n)
        eq, ub = self.matrices(n, fixed_count)
        net = np.maximum((load - pv) / 6 + margin, 0)
        surplus = np.maximum(-((load - pv) / 6 + margin), 0)
        objective = np.zeros(8 * n)
        objective[4 * n:5 * n] = 5 * price
        objective[6 * n:7 * n] = price
        objective[7 * n:7 * n + fixed_count] = 0.5 * price[:fixed_count]
        objective[4 * n - 1] = -self.terminal_value
        lower = np.zeros(8 * n)
        upper = np.full(8 * n, np.inf)
        upper[:n] = CAP
        upper[n:2 * n] = np.minimum(surplus, CAP)
        upper[2 * n:3 * n] = CAP
        lower[3 * n:4 * n] = 1200
        upper[3 * n:4 * n] = 10800
        upper[7 * n + fixed_count:] = 0
        beq = np.r_[np.zeros(n), net]
        beq[0] = state
        bub = np.r_[np.full(n, CAP), net,
                    ([] if fixed_count == 0 else q0[:fixed_count]),
                    ([] if fixed_count == 0 else -q0[:fixed_count])]
        start = time.perf_counter()
        result = linprog(
            objective, A_ub=ub, b_ub=bub, A_eq=eq, b_eq=beq,
            bounds=np.column_stack((lower, upper)), method="highs",
            options={"time_limit": 10.0, "primal_feasibility_tolerance":
            ↪  1e-8, "dual_feasibility_tolerance": 1e-8},
        )
        if not result.success:
            raise RuntimeError(result.message)
        z = result.x.reshape(8, n)
        residual = max(
            float(np.max(np.abs(eq @ result.x - beq))), float(np.max(ub @
            ↪  result.x - bub)),
            float(np.max(lower - result.x)), float(np.max(result.x - upper)),
        )
        finite = np.isfinite(upper)
        dual = (beq @ result.eqlin.marginals + bub @ result.ineqlin.marginals
```

```
                + lower @ result.lower.marginals + upper[finite] @ result.upper.marginals[finite])
        gap = abs(result.fun - dual)
        simultaneous = int(np.sum((z[0] + z[1] > 1e-6) & (z[2] > 1e-6)))
        assert residual < 1e-6 and gap < 1e-4 and simultaneous == 0
        logs.event("solver_checks", dict(meta, residual=residual, dual_gap=float(gap),
                                         simultaneous=simultaneous, status=int(result.status),
                                         seconds=time.perf_counter() - start))
        return np.maximum(z[6], 0)

def execute_block(load_kw, pv_kw, order, initial):
    output = np.empty((len(order), len(EXEC_COLUMNS)))
    state = float(initial)
    for k in range(len(order)):
        load = float(load_kw[k] / 6)
        pv = float(pv_kw[k] / 6)
        paid = max(0.0, float(order[k]))
        direct = min(load, pv)
        need = load - direct
        surplus = pv - direct
        start = state
        if need > paid:
            discharge = min(need - paid, CAP, max(0.0, 0.9 * (state - 1200)))
            emergency = max(0.0, need - paid - discharge)
            grid_charge = pv_charge = 0.0
            received = paid
        else:
            charge = min(surplus + paid - need, CAP, max(0.0, (10800 - state) / 0.9))
            pv_charge = min(surplus, charge)
            grid_charge = charge - pv_charge
            received = need + grid_charge
            discharge = emergency = 0.0
        state += 0.9 * (grid_charge + pv_charge) - discharge / 0.9
        output[k] = [
            start, state, received, received - grid_charge, direct, grid_charge, pv_charge,
            discharge, emergency, max(0.0, paid - received), max(0.0, surplus - pv_charge), load,
        ]
        assert 1200 - 1e-6 <= state <= 10800 + 1e-6
        assert abs(received - grid_charge + direct + discharge + emergency - load) < 1e-6
    return output, state
```

D.9　问题四零点分支

文件：code/q4_tree.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
```

```
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dump
  ↪  s(obj,ensure_ascii=False,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
import q4
class ForecastEngine:
    """Rolling P3 direct-price or P2 residual-price model."""

    def __init__(self, name, features, model_kind, window=56, alpha=10.0,
                 feature_mode="full", max_leaf_nodes=15):
        self.name = name
        self.f = features
        self.model_kind = model_kind
        self.window = int(window)
        self.alpha = float(alpha)
        self.feature_mode = feature_mode
        self.max_leaf_nodes = int(max_leaf_nodes)
        self.cache = {}
        self.model_dir = O / "price_models" / name
        self.model_dir.mkdir(parents=True, exist_ok=True)
        self.index = {name: i for i, name in enumerate(self.f.FEATURE_NAMES)}
        if model_kind == "p3":
            if feature_mode == "full":
                self.feature_names = list(self.f.FEATURE_NAMES)
            elif feature_mode == "no_net":
                self.feature_names = [x for x in self.f.FEATURE_NAMES if x !=
                 ↪  "predicted_net_load_1000kw"]
            elif feature_mode == "history_only":
                self.feature_names = list(self.f.FEATURE_NAMES[:4])
            else:
                raise ValueError(feature_mode)
        else:
            self.feature_names = [
                "matching_weekday_level", "recent_minus_matching",
                 ↪  "lag7_minus_matching",
                "attachment1_minus_matching", "predicted_net_load_1000kw",
                "current_day_level", "current_level_decay6h",
                 ↪  "last_residual_decay1h",
```

```
                "lead_hours", "season_sin", "season_cos",
                *[f"weekday_{i}" for i in range(7)], "release_fraction",
            ]
            if feature_mode == "residual_no_net":
                self.feature_names.remove("predicted_net_load_1000kw")

    def transform(self, full):
        if self.model_kind == "p3":
            return full[:, [self.index[x] for x in self.feature_names]],
            ↪  full[:, self.index["matching_weekday_median4"]]
        match = full[:, self.index["matching_weekday_median4"]]
        cols = {
            "matching_weekday_level": match,
            "recent_minus_matching": full[:, self.index["recent7_median"]] -
            ↪  match,
            "lag7_minus_matching": full[:, self.index["lag7_price"]] - match,
            "attachment1_minus_matching": full[:,
            ↪  self.index["attachment1_base"]] - match,
        }
        for name in (
            "predicted_net_load_1000kw", "current_day_level",
            ↪  "current_level_decay6h",
            "last_residual_decay1h", "lead_hours", "season_sin",
            ↪  "season_cos",
            *[f"weekday_{i}" for i in range(7)], "release_fraction",
        ):
            cols[name] = full[:, self.index[name]]
        return np.column_stack([cols[x] for x in self.feature_names]), match

    def training(self, cutoff):
        xs, ys = [], []
        for origin in range(max(1, cutoff - self.window), cutoff):
            for release in self.f.releases:
                targets, net = self.f.supply_forecast(origin, release)
                valid = (targets < cutoff * q4.SLOTS) & (targets < q4.N_DAYS
                ↪  * q4.SLOTS)
                if not valid.any():
                    continue
                full = self.f.matrix(origin, release, targets[valid],
                ↪  net[valid])
                x, match = self.transform(full)
                y = self.f.prices.reshape(-1)[targets[valid]]
                xs.append(x)
                ys.append(y if self.model_kind == "p3" else y - match)
        x = np.vstack(xs)
        y = np.concatenate(ys)
        assert len(y) >= 500 and np.isfinite(x).all() and
        ↪  np.isfinite(y).all()
        return x, y

    def fit(self, cutoff):
        if cutoff in self.cache:
            return self.cache[cutoff]
        x, y = self.training(cutoff)
        budget.price_fits += 1
        budget.check()
        started = time.perf_counter()
        if self.model_kind == "p3":
            scaler = None
```

```
            model = HistGradientBoostingRegressor(
                max_leaf_nodes=self.max_leaf_nodes, max_iter=100,
                ↪  learning_rate=0.05,
                min_samples_leaf=50, l2_regularization=1.0,
                early_stopping=False, random_state=2026,
            )
            with threadpool_limits(limits=1):
                model.fit(x, y)
            detail = {"max_leaf_nodes": self.max_leaf_nodes, "iterations":
            ↪  int(model.n_iter_),
                      "learning_rate": 0.05, "min_samples_leaf": 50}
        else:
            scaler = StandardScaler()
            z = scaler.fit_transform(x)
            model = Ridge(alpha=self.alpha)
            with threadpool_limits(limits=1):
                model.fit(z, y)
            original = model.coef_ / scaler.scale_
            detail = {
                "alpha": self.alpha,
                "coefficients_original": dict(zip(self.feature_names,
                ↪  original.astype(float))),
                "intercept_original": float(model.intercept_ - original @
                ↪  scaler.mean_),
            }
        path = self.model_dir / f"cutoff-{cutoff:03d}.joblib"
        joblib.dump({"model": model, "scaler": scaler, "features":
        ↪  self.feature_names,
                     "model_kind": self.model_kind, "feature_mode":
                     ↪  self.feature_mode}, path, compress=3)
        record = {
            "strategy": self.name, "branch": self.f.branch, "model_kind":
            ↪  self.model_kind,
            "feature_mode": self.feature_mode, "window": self.window,
            ↪  "cutoff_day": int(cutoff),
            "cutoff_datetime": q4.daystr(cutoff) + " 00:00",
            ↪  "latest_target_end": q4.daystr(cutoff) + " 00:00",
            "training_rows": int(len(y)), "feature_names":
            ↪  self.feature_names,
            "model_path": str(path.relative_to(R)), "model_sha256":
            ↪  sha(path),
            "seconds": time.perf_counter() - started, **detail,
        }
        logs.event("price_fit_log", record)
        self.cache[cutoff] = model, scaler
        budget.save()
        return self.cache[cutoff]

    def predict(self, day, release):
        targets, net = self.f.supply_forecast(day, release)
        cutoff = 7 + ((day - 7) // 7) * 7
        model, scaler = self.fit(cutoff)
        full = self.f.matrix(day, release, targets, net)
        x, match = self.transform(full)
        pred = model.predict(x if scaler is None else scaler.transform(x))
        if self.model_kind != "p3":
            pred = match + pred
        pred = np.maximum(pred, 0.001)
        logs.event("price_forecast_log", {
```

```
            "strategy": self.name, "branch": self.f.branch, "decision_day":
            ↪ int(day),
            "decision_date": q4.daystr(day), "release_slot": int(release),
            "known_price_intervals_today": int(release), "fit_cutoff_day":
            ↪ int(cutoff),
            "forecast_points": int(len(pred)),
            "sha256": hashlib.sha256(np.asarray(pred,
            ↪ dtype=np.float64).tobytes()).hexdigest(),
        })
        return pred
```

D.10 问题四日内分支

文件：code/q4_update.py

```
from pathlib import Path
import os,json,time,hashlib,csv,itertools
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
import joblib
R=Path(__file__).resolve().parents[1]
P=R
O=R/'reproduced'
DATE=pd.Timestamp('2025-01-01')
N=365*144
N_DAYS=365
SLOTS=144
CAP=5000/6
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,obj):
 p=Path(p);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dump
 ↪ s(obj,ensure_ascii=False,indent=2),encoding='utf-8')
def daystr(d):return str((DATE+pd.Timedelta(days=int(d))).date())
class Budget:
 def __init__(self):self.calls=0;self.lp_calls=0;self.price_fits=0
 def check(self):pass
 def save(self):pass
class Logs:
 def event(self,*a,**kw):pass
budget=Budget();logs=Logs()
import q4
class Features(q4.PriceFeatures):
 def supply_forecast(self,day,slot):
  b=day*4+slot//36;n=288-slot
  return day*144+slot+np.arange(n),self.supply['load'][b,:n]-self.supply['pv
  ↪ '][b,:n]

class Prices(q4.PriceModels):
 def fit(self,candidate,cutoff_day):
  ans=super().fit(candidate,cutoff_day);self.datasets.clear();return ans
```

D.11　四问完整复算入口

文件：code/reproduce.py

```
"""Reproduce the paper's final schedules from prepared inputs; no parameter search."""
import os
for key in ('OMP_NUM_THREADS','OPENBLAS_NUM_THREADS','MKL_NUM_THREADS','NUMEXPR_NUM_THREADS'):os.environ[key]='1'
from pathlib import Path
import argparse,json
import numpy as np
import pandas as pd
from threadpoolctl import threadpool_limits
import q1,q2,q3_base,q3,q4,q4_tree,q4_update
ROOT=Path(__file__).resolve().parents[1]
SRC=dict(np.load(ROOT/'data/inputs.npz'))
OUT=Path(os.environ.get('SUPPORT_REPRO_OUTPUT',str(ROOT/'reproduced'))).resolve();OUT.mkdir(parents=True,exist_ok=True)

def save(label,order,execution,versions=None,qzero=None):
 arrays=dict(days=np.arange(31,31+len(order)),execution=execution)
 if versions is None:arrays['order']=order
 else:arrays.update(q0=qzero,versions=versions)
 np.savez_compressed(OUT/f'{label}.npz',**arrays)
 return arrays

def question1():
 cfg=json.loads((ROOT/'data/q1_config.json').read_text())
 ref=pd.read_csv(ROOT/'results/Q1.csv',float_precision='round_trip')
 # Only source inputs are reused; orders, states and costs are solved anew.
 inputs=ref[['k','start_minute','end_minute','load_kw','pv_kw','price']].to_dict('records')
 actions,certificate=q1.solve_lp(inputs,cfg);rows=q1.schedule(inputs,cfg,actions)
 pd.DataFrame(rows).to_csv(OUT/'Q1.csv',index=False,float_format='%.17g')
 assert abs(sum(r['cost_yuan'] for r in rows)-35126.85)<.005
 return np.array([r['purchase_kwh'] for r in rows])

def january(data,q1order):
 planner=q2.Planner(data.price,lambda *a:None,lambda:None);state=6000.;orders=[];ex=[]
 for d in range(31):
  order=q1order if d==0 else planner.solve(*data.history(d,1,3)[:2],state,np.zeros(288),{})[0][:144]
  l,v=data.truth(d,initialization=d==0);block,state=q3_base.execute_block(l,v,order,state)
  orders.append(order);ex.append(block)
 np.savez_compressed(OUT/'January.npz',days=np.arange(31),order=orders,execution=ex)
 assert abs(state-8592.693560925925)<1e-6,state
 return state

def question2(data,initial,end,quantile=.7,decay=.7):
 planner=q2.Planner(data.price,lambda *a:None,lambda:None);risk=q2.Risk(data);state=initial;orders=[];ex=[]
 for d in range(31,end):
  l,v,_=data.history(d,4,7);m,_=risk.margin(d,quantile,28,4,decay)
  order=planner.solve(l,v,state,m,{})[0][:144]
```

```
  a,b=data.truth(d);block,state=q3_base.execute_block(a,b,order,state);order.append(order);ex.append(block)
 return save('Q2',np.array(orders),np.array(ex))

def supply3(family='EW4',quantile=.6,information='full'):
 actual,reference,F,release,targets=(SRC[k] for k in ['actual','reference','F','release','targets'])
 hist,last,L,H,X=q3_base.history_and_features(actual,reference,F,release,targets)
 hourly,w,count,fb,cut=q3_base.build_fusion(actual,F,H,release,targets)
 if information=='no_new':
  oldF=H.copy()
  for b,r in enumerate(release):
   midnight=r//144*144;t=targets[b];use=t<=midnight+144
   oldF[b,use]=F[(r//144)*4,((t[use]-midnight)//6-1).astype(int)]
  hourly,w,count,fb,cut=q3_base.build_fusion(actual,oldF,H,release,targets)
 full=q3_base.to_intervals(hourly,hist,last,release)
 q3.src=SRC;q3.orig={'L':L};q3.fc={'L':q3.midnight(family)}
 load,theta,obs,count=q3.make_load('LEARN_PREFIX',1.)
 err=q3.errors_for(load,full);q3.group_cache=None;margin=np.full((1460,288),np.nan)
 for b,typ,g,dest,ids,ages,fallback in q3.group_index(err):
  margin[b,dest]=q3.quantile(err['error'][ids],ages,quantile) if len(ids)>=24 else 0.
 np.savez_compressed(OUT/'supply3.npz',load=load,pv=full,margin=margin,weights=w,theta=theta)
 return load,full,margin

def question3(initial,end,supply,locked=False):
 load,pv,margin=supply;planner=q3.Planner(SRC['price'],q3.Budget());state=initial
 orders=[];zeros=[];versions=[];ex=[]
 for d in range(31,end):
  order=np.zeros(144);dayversions=[];blocks=[]
  for a,r in enumerate((0,36,72,108)):
   b=d*4+a;n=288-r
   if a==0 or not locked:
    plan,_=planner.solve(load[b,:n],pv[b,:n],state,margin[b,:n],r,None if a==0 else qzero[r:])
    order[r:]=np.maximum(0,plan[6,:144-r])
   if a==0:qzero=order.copy()
   dayversions.append(order.copy());truth=SRC['actual'][d*144+r:d*144+r+36]
   block,state=q3_base.execute_block(truth[:,0],truth[:,1],order[r:r+36],state);blocks.append(block)
  orders.append(order.copy());zeros.append(qzero);versions.append(dayversions)
  if (d-30)%100==0:print('Q3 completed days',d-30,flush=True)
 return save('Q3',np.array(orders),np.array(ex),np.array(versions),np.array(zeros))

def question4(data,initial,end,supply,branch):
 for m in (q4,q4_tree,q4_update):m.R=OUT;m.O=OUT
 prices=SRC['actual_prices'];reference=SRC['price'];state=initial
 if branch=='Q4-2':
  load=np.empty((365,288));pv=load.copy();margin=load.copy();risk=q4.Q2Risk(data)
  load[0]=np.tile(SRC['reference'][:,0],2);pv[0]=np.tile(SRC['reference'][:,1],2);margin[0]=0
```

```
  for d in range(1,365):load[d],pv[d],_=data.history(d,4,7);margin[d]=risk.margin(d)[0]
  features=q4.PriceFeatures(prices,reference,dict(load=load,pv=pv),branch)
  engine=q4_tree.ForecastEngine('Q4-2-leaves31',features,'p3',max_leaf_nodes=31)
  planner=q4.Q42Planner(.9*reference[0]);orders=[];ex=[]
  for d in range(31,end):
   forecast=engine.predict(d,0);order=planner.solve(load[d],pv[d],state,margin[d],forecast,dict(day=d))[:144]
   a,b=data.truth(d);block,state=q4.execute_block(a,b,order,state);orders.append(order);ex.append(block)
   if (d-30)%50==0:print(branch,'completed days',d-30,flush=True)
  return save(branch,np.array(orders),np.array(ex))
 load,pv,margin=supply
 features=q4_update.Features(prices,reference,dict(load=load,pv=pv),branch)
 engine=q4_update.Prices(features,branch);planner=q4.Q43Planner(.9*reference[0])
 orders=[];zeros=[];versions=[];ex=[]
 for d in range(31,end):
  order=np.zeros(144);dayversions=[];blocks=[]
  for a,r in enumerate((0,36,72,108)):
   b=d*4+a;n=288-r;forecast=engine.predict('P2',d,r)
   order[r:]=planner.solve(load[b,:n],pv[b,:n],state,margin[b,:n],r,None if a==0 else qzero[r:],forecast,dict(day=d,release_slot=r))[:144-r]
   if a==0:qzero=order.copy()
   dayversions.append(order.copy());truth=SRC['actual'][d*144+r:d*144+r+36]
   block,state=q4.execute_block(truth[:,0],truth[:,1],order[r:r+36],state);blocks.append(block)
  orders.append(order.copy());zeros.append(qzero);versions.append(dayversions);ex.append(np.concatenate(blocks))
  if (d-30)%100==0:print(branch,'completed days',d-30,flush=True)
 return save(branch,np.array(orders),np.array(ex),np.array(versions),np.array(zeros))

def compare(label,z):
 old=np.load(ROOT/'results'/f'{label}.npz');n=len(z['days']);errors={}
 for k in ('order','q0','versions','execution'):
  if k in z:
   errors[k]=float(np.max(abs(z[k]-old[k][:n])))
   np.testing.assert_allclose(z[k],old[k][:n],rtol=0,atol=1e-5,err_msg=label+' '+k)
 print(label,'reproduced',n,'days; max differences',errors,flush=True)
 return errors

def main():
 p=argparse.ArgumentParser();p.add_argument('--question',choices=['all','Q1','Q2','Q3','Q4-2','Q4-3'],default='all');p.add_argument('--days',type=int,default=334);a=p.parse_args()
 assert 1<=a.days<=334;end=31+a.days
 data=q2.Data(SRC);order=question1();initial=january(data,order);report={}
 if a.question in ('all','Q2'):report['Q2']=compare('Q2',question2(data,initial,end))
 supply=supply3() if a.question in ('all','Q3','Q4-3') else None
 if a.question in ('all','Q3'):report['Q3']=compare('Q3',question3(initial,end,supply))
 for branch in ('Q4-2','Q4-3'):
  if a.question in ('all',branch):report[branch]=compare(branch,question4(data,initial,end,supply,branch))
```

```
 (OUT/'reproduction_check.json').write_text(json.dumps(dict(days=a.days,errors=report),indent=2),encoding='utf-8')
if __name__=='__main__':
 with threadpool_limits(limits=1):main()
```

D.12 已有参数与信息对照

文件：code/validation.py

```
"""Reproduce the existing paper comparisons; choose one setting per invocation."""
import argparse,itertools,json
import numpy as np
import pandas as pd
import reproduce as r
from threadpoolctl import threadpool_limits
def main():
 p=argparse.ArgumentParser();p.add_argument('question',choices=['Q1','Q2','Q3']);p.add_argument('--quantile',type=float);p.add_argument('--decay',type=float,default=.7);p.add_argument('--family',choices=['W1','W2','W4','EW4'],default='EW4');p.add_argument('--information',choices=['full','no_new','zero_only'],default='full');p.add_argument('--days',type=int,default=334);a=p.parse_args()
 assert 1<=a.days<=334
 if a.question=='Q1':
  cfg=json.loads((r.ROOT/'data/q1_config.json').read_text());inputs=pd.read_csv(r.ROOT/'results/Q1.csv',float_precision='round_trip')[['k','start_minute','end_minute','load_kw','pv_kw','price']].to_dict('records');rows=[]
  for reserve in [False,True]:
   for low,high,target in itertools.product(cfg['low_quantiles'],cfg['high_quantiles'],cfg['target_fractions_of_usable_capacity']):
    actions,_=r.q1.rule(inputs,cfg,low,high,target,reserve)
    rows.append(dict(reserve=reserve,low=low,high=high,target=target,**r.q1.metrics(r.q1.schedule(inputs,cfg,actions))))
  for initial in [1200,6000,10800]:
   cfg.update(initial_kwh=initial,terminal_kwh=initial);actions,_=r.q1.solve_lp(inputs,cfg);rows.append(dict(initial=initial,**r.q1.metrics(r.q1.schedule(inputs,cfg,actions))))
  pd.DataFrame(rows).to_csv(r.OUT/'Q1_comparisons.csv',index=False);print('Q1: 54 rule settings and three initial-storage settings reproduced.');return
 initial=r.january(r.q2.Data(r.SRC),r.question1())
 if a.question=='Q2':z=r.question2(r.q2.Data(r.SRC),initial,31+a.days,a.quantile if a.quantile is not None else .7,a.decay)
 else:
  supply=r.supply3(a.family,a.quantile if a.quantile is not None else .6,'no_new' if a.information=='no_new' else 'full')
  z=r.question3(initial,31+a.days,supply,a.information=='zero_only')
 ex=z['execution'];order=z['order'] if 'order' in z else z['versions'][:,-1];zero=z.get('q0',order);price=r.SRC['price']
 normal=float((order*price).sum());adjust=float((.5*price*abs(order-zero)).sum());emergency=float((5*price*ex[:,:,8]).sum())
 print(json.dumps(dict(settings=vars(a),normal_cost=normal,adjustment_cost=adjust,emergency_cost=emergency,total_cost=normal+adjust+emergency,emergency_kwh=float(ex[:,:,8].sum())),ensure_ascii=False,indent=2))
if __name__=='__main__':
 with threadpool_limits(limits=1):main()
```

D.13　结果表填写与约束核验

文件：code/export_results.py

```
"""Export saved final results into the five official workbooks and check balances."""
from pathlib import Path
from datetime import datetime,timedelta
from copy import copy,deepcopy
import csv,gzip,json
import numpy as np
import pandas as pd
import openpyxl
from openpyxl.packaging.core import DocumentProperties
ROOT=Path(__file__).resolve().parents[1]
EXPECTED={'Q1':35126.85,'Q2':13583810.84,'Q3':13014172.33,'Q4-2':14248570.87,'Q4-3':13695929.70}
EXEC_NAMES=['storage_start_kwh','storage_end_kwh','received_kwh','grid_load_kwh','pv_load_kwh','grid_charge_kwh','pv_charge_kwh','discharge_kwh','emergency_kwh','unused_order_kwh','unused_pv_kwh','load_kwh']
def time_label(k):return f'{k//6:02d}:{k%6*10:02d}'
def metadata(w,template_name):
 w.properties=DocumentProperties(creator='',lastModifiedBy='',title='',description='',created=datetime(2026,9,13),modified=datetime(2026,9,13))
 for s in w:
  for row in s:
   for c in row:
    c.comment=None
    if isinstance(c.value,(int,float)):c.number_format='0.00'
 template=openpyxl.load_workbook(ROOT/'templates'/template_name)
 format_workbook(w,template)
 template.close()

def format_workbook(w,template):
 """Restore template styles, extending daily blocks without changing any values."""
 cache={}
 for s in w:
  t=template[s.title]
  for attr in ['sheet_format','sheet_properties','views','auto_filter',
               'page_setup','page_margins','print_options']:
   setattr(s,attr,deepcopy(getattr(t,attr)))
  s.print_title_rows=t.print_title_rows
  s.print_title_cols=t.print_title_cols
  s.print_area=str(t.print_area) if t.print_area else ''
  s.row_dimensions.clear();s.column_dimensions.clear()
  for key,dim in t.column_dimensions.items():
   s.column_dimensions[key]=copy(dim);s.column_dimensions[key].parent=s
  for key,dim in t.row_dimensions.items():
   s.row_dimensions[key]=copy(dim);s.row_dimensions[key].parent=s
  annual_storage=s.title==' 充放电量' and s.max_column==6
  for row in s:
   r=row[0].row;single=False
   if r==1:tr=1
   elif annual_storage:tr=2+(r-2)%6
   elif s.title==' 紧急购电量':
    date=s.cell(r,1).value
    first=r==2 or s.cell(r-1,1).value!=date
    last=r==s.max_row or s.cell(r+1,1).value!=date
```

```
    tr=2 if first else (4 if last else 3)
    single=first and last
   else:tr=r if r<=t.max_row else 2
   if tr in t.row_dimensions:
    s.row_dimensions[r]=copy(t.row_dimensions[tr])
    s.row_dimensions[r].index=r;s.row_dimensions[r].parent=s
   for c in row:
    ref=t.cell(tr,c.column)
    # All retained dates use the template's date format, including continuation rows.
    number_format=t.cell(2,1).number_format if isinstance(c.value,datetime) else ref.number_format
    key=(s.title,ref.style_id,number_format,single,c.column if single else 0)
    if key not in cache:
     for attr in ['font','fill','border','alignment','protection']:
      setattr(c,attr,copy(getattr(ref,attr)))
     c.number_format=number_format
     if single:
      border=copy(c.border);border.bottom=copy(t.cell(4,c.column).border.bottom)
      c.border=border
     cache[key]=copy(c._style)
    c._style=copy(cache[key])
def numeric(s,r,c,v):
 if abs(v)<1e-10:v=0.
 s.cell(r,c,float(v)).number_format='0.00'
def clear_data(s):
 for row in s.iter_rows(min_row=2):
  for c in row:c.value=None
def fill_orders(s,orders,fee):
 for i,order in enumerate(orders):
  r=i+2
  for k,v in enumerate(order):numeric(s,r,145 if k==0 else k+1,v)
  numeric(s,r,146,order.sum());numeric(s,r,147,fee[i])
 s.freeze_panes='B2'
 # The official template's last header is retained; its same-date slot-0 meaning is documented.
def fill_storage(s,ex):
 styles=[copy(c._style) for c in s[2]];clear_data(s)
 for i,day in enumerate(ex):
  date=datetime(2025,2,1)+timedelta(days=i)
  for g in range(6):
   r=2+i*6+g
   for j,style in enumerate(styles,1):s.cell(r,j)._style=copy(style)
   s.cell(r,1,date).number_format='yyyy-mm-dd'
   s.cell(r,2,f'{4*g}:00-{4*(g+1)}:00')
   numeric(s,r,3,day[g*24:(g+1)*24,5:7].sum());numeric(s,r,4,day[g*24:(g+1)*24,7].sum())
   if g<2:s.cell(r,5,'0:00' if g==0 else '24:00');numeric(s,r,6,day[0,0] if g==0 else day[-1,1])
 s.freeze_panes='C2'
def fill_emergency(s,ex):
 clear_data(s);r=2;events=0
 for i,day in enumerate(ex):
  k=0;found=False;date=datetime(2025,2,1)+timedelta(days=i)
  while k<144:
   if day[k,8]<=1e-8:k+=1;continue
   end=k+1
   while end<144 and day[end,8]>1e-8:end+=1
```

```
    s.cell(r,1,date).number_format='yyyy-mm-dd';s.cell(r,2,f'{time_label(k)}-{time_label(end)}');numeric(s,r,3,day[k:end,8].sum())
    k=end;r+=1;events+=1;found=True
   if not found:s.cell(r,1,date).number_format='yyyy-mm-dd';s.cell(r,2,'无');numeric(s,r,3,0);r+=1
  s.column_dimensions['A'].width=15;s.column_dimensions['B'].width=22;s.column_dimensions['C'].width=18
  return events
def main():
  src=np.load(ROOT/'data/inputs.npz');report={};dailyrows=[]
  q1=pd.read_csv(ROOT/'results/Q1.csv',float_precision='round_trip')
  w=openpyxl.load_workbook(ROOT/'templates/result1.xlsx')
  for k,v in enumerate(q1.purchase_kwh):numeric(w['计划购电量'],145 if k==0 else k+1,2,v)
  for g in range(6):
   part=q1.iloc[g*24:(g+1)*24];numeric(w['充放电量'],g+2,2,part.charge_kwh.sum());numeric(w['充放电量'],g+2,3,part.discharge.sum())
  numeric(w['充放电量'],2,5,q1.state_start.iloc[0]);numeric(w['充放电量'],3,5,q1.state_end.iloc[-1]);metadata(w,'result1.xlsx');w.save(ROOT/'result1.xlsx')
  report['Q1']={'total_cost':float(q1.cost_yuan.sum()),'order_kwh':float(q1.purchase_kwh.sum())}
  for label in ['Q2','Q3','Q4-2','Q4-3']:
   z=dict(np.load(ROOT/'results'/f'{label}.npz'));ex=z['execution'];rolling='versions' in z
   order=z['versions'][:,-1,:] if rolling else z['order'];zero=z['q0'] if rolling else order
   price=src['actual_prices'][31:365] if label.startswith('Q4') else np.tile(src['price'],(334,1))
   normal=(price*order).sum(1);adjust=(.5*price*abs(order-zero)).sum(1);emergency=(5*price*ex[:,:,8]).sum(1)
   cost=normal+adjust+emergency;assert abs(cost.sum()-EXPECTED[label])<.005,(label,cost.sum())
   balance=ex[:,:,3]+ex[:,:,4]+ex[:,:,7]+ex[:,:,8]-ex[:,:,11]
   storage=ex[:,:,1]-ex[:,:,0]-.9*(ex[:,:,5]+ex[:,:,6])+ex[:,:,7]/.9
   assert max(abs(balance).max(),abs(storage).max())<1e-6
   assert ex[:,:,:2].min()>=1200-1e-6 and ex[:,:,:2].max()<=10800+1e-6
   assert max((ex[:,:,5]+ex[:,:,6]).max(),ex[:,:,7].max())<=5000/6+1e-6
   assert np.max(abs(ex[1:,0,0]-ex[:-1,-1,1]))<1e-6
   if rolling:
    for a,r in enumerate([36,72,108],1):assert np.max(abs(z['versions'][:,a,:r]-z['versions'][:,a-1,:r]))<1e-7
   file={'Q2':'result2','Q3':'result3','Q4-2':'result4-2','Q4-3':'result4-3'}[label]
   w=openpyxl.load_workbook(ROOT/'templates'/f'{file}.xlsx');fill_orders(w['计划购电量'],zero,(price*zero).sum(1))
   if rolling:fill_orders(w[' 调整购电量'],order,normal+adjust)
   fill_storage(w['充放电量'],ex);events=fill_emergency(w['紧急购电量'],ex);metadata(w,f'{file}.xlsx');w.save(ROOT/f'{file}.xlsx')
   if rolling:
    with gzip.open(ROOT/'results'/f'{label}_adjustments.csv.gz','wt',encoding='utf-8-sig',newline='') as f:
     writer=csv.writer(f);writer.writerow(['date','release_time','interval_start','interval_end','midnight_kwh','revised_total_kwh','net_change_vs_midnight_kwh'])
     for i in range(334):
      date=str((datetime(2025,2,1)+timedelta(days=i)).date())
      for a,r in enumerate([0,36,72,108]):
```

```
      for k in range(r,144):writer.writerow([date,time_label(r),time_label(k),time_label(k+1),zero[i,k],z['versions'][i,a,k],z['versions'][i,a,k]-zero[i,k]])
  for i in range(334):dailyrows.append(dict(question=label,date=str((datetime(2025,2,1)+timedelta(days=i)).date()),midnight_order_kwh=float(zero[i].sum()),final_order_kwh=float(order[i].sum()),normal_cost=normal[i],adjustment_cost=adjust[i],emergency_cost=emergency[i],total_cost=cost[i],emergency_kwh=float(ex[i,:,8].sum()),initial_kwh=ex[i,0,0],final_kwh=ex[i,-1,1]))
  report[label]=dict(days=334,slots=48096,storage_groups=2004,emergency_events=events,total_cost=float(cost.sum()),normal_cost=float(normal.sum()),adjustment_cost=float(adjust.sum()),emergency_cost=float(emergency.sum()),energy_balance_max_error=float(abs(balance).max()),storage_max_error=float(abs(storage).max()))
 pd.DataFrame(dailyrows).to_csv(ROOT/'results/daily_costs.csv',index=False,encoding='utf-8-sig',float_format='%.17g')
 (ROOT/'evidence/export_check.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
 print(json.dumps(report,ensure_ascii=False,indent=2))
if __name__=='__main__':main()
```

D.14 数据分析图

文件：code/fig_data.py

```
"""Re-layout saved EDA only; no fitting or model changes."""
from pathlib import Path
import os,hashlib,json
P=Path(__file__).resolve().parents[1]
paper=P/'reproduced/plots'
(paper/'fig').mkdir(parents=True,exist_ok=True)
(paper/'scripts').mkdir(exist_ok=True)
os.environ.setdefault('MPLCONFIGDIR',str(P/'tmp/matplotlib'))
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import pandas as pd
source=P/'figure_inputs/eda'
monthly=pd.read_csv(source/'monthly_summary.csv')
weekday=pd.read_csv(source/'weekday_summary.csv')
errors=pd.read_csv(source/'forecast_error_by_lead.csv')
plt.rcParams.update({'font.family':'SimSun','font.size':11,'axes.unicode_minus':False,'pdf.fonttype':42,
 'axes.spines.top':False,'axes.spines.right':False,'legend.frameon':False,'savefig.facecolor':'white'})
out=paper/'fig'
fig,ax=plt.subplots(1,2,figsize=(7.2,2.6),layout='constrained')
ax[0].plot(monthly.month,monthly.load_kw,'o-',color='#b85518',label='负载',lw=1.4,ms=3)
ax[0].plot(monthly.month,monthly.pv_actual_kw,'s--',color='#2166ac',label='光伏',lw=1.4,ms=3)
ax[0].set(title='(a)负载与光伏的月平均水平',xlabel='月份',ylabel='平均功率 / kW',xticks=[1,3,5,7,9,12],ylim=(0,6500))
ax[0].legend(ncol=2,fontsize=10)
ax[1].bar(weekday.weekday,weekday.load_kw,color=['#b85518']*4+['#2166ac']*2+['#b85518'],width=.6)
ax[1].set(title='(b)不同星期位置的平均负载',xlabel='星期',ylabel='平均功率 / kW',xticks=list(range(7)),xticklabels=list('一二三四五六日'),ylim=(0,6500))
```

```
for a in ax:a.grid(axis='y',alpha=.18);a.set_axisbelow(True)
fig.savefig(out/'data_compact.pdf',bbox_inches='tight');fig.savefig(out/'data_compact.png',dpi=220,bbox_inches='tight');plt.close(fig)
fig,ax=plt.subplots(figsize=(6.4,2.25),layout='constrained')
for scope,label,color,style in [('all','全部匹配样本','#2166ac','--'),('actual_positive','实际光伏大于零','#b85518','-')]:
    g=errors[errors.scope==scope]
    ax.plot(g.lead_hours,g.mae_kw,label=label,color=color,ls=style,lw=1.5,marker='o',ms=2.5)
ax.set(xlabel='预报提前时间 / h',ylabel='平均绝对误差 / kW',xticks=[1,6,12,18,24],ylim=(0,None))
ax.legend(fontsize=10);ax.grid(axis='y',alpha=.18)
fig.savefig(out/'forecast_compact.pdf',bbox_inches='tight');fig.savefig(out/'forecast_compact.png',dpi=220,bbox_inches='tight');plt.close(fig)
files=[source/n for n in ['monthly_summary.csv','weekday_summary.csv','forecast_error_by_lead.csv']]
manifest={'sources':[{'path':str(p),'sha256':hashlib.sha256(p.read_bytes()).hexdigest()} for p in files],
 'scope':'Saved descriptive statistics re-plotted without altering data or model; monthly and weekday means follow original source-date grouping.',
 'outputs':['fig/data_compact.pdf','fig/data_compact.png','fig/forecast_compact.pdf','fig/forecast_compact.png']}
(paper/'scripts/compact_figures_manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),'utf-8')
print('Saved compact figures')
```

D.15　问题一结果图

文件：code/fig_q1.py

```
from pathlib import Path
import json, csv, hashlib
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
PROJECT=Path(__file__).resolve().parents[1]
HERE=PROJECT/'reproduced/plots/fig'
HERE.mkdir(parents=True,exist_ok=True)
plt.rcParams.update({'font.family':'SimSun','font.size':10,'axes.unicode_minus':False,
 'text.color':'black','axes.labelcolor':'black','xtick.color':'black','ytick.color':'black',
 'pdf.fonttype':42,'axes.spines.top':False,'axes.spines.right':False,
 'axes.axisbelow':True,'savefig.facecolor':'white','legend.frameon':False})
def save(fig, name):
    fig.canvas.draw()
    for ext in ('pdf','png'):
        kw = {'metadata':{'CreationDate':None,'ModDate':None}} if ext=='pdf' else {}
        fig.savefig(HERE/f'{name}.{ext}',dpi=300,bbox_inches='tight',**kw)
    fig.savefig(HERE/f'{name}_preview.png',dpi=120,bbox_inches='tight')
    plt.close(fig)

ROOT=PROJECT/'figure_inputs/q1'
def rows(path):
    with path.open(encoding='utf-8-sig',newline='') as f: return list(csv.DictReader(f))
```

```
summary=json.loads((ROOT/'outputs/summary.json').read_text('utf-8'))
selected={row['candidate']:row for row in summary['comparison']}
schedules={c:rows(ROOT/'outputs/schedules'/f"{row['key']}.csv") for c,row in selected.items()}
p=np.array([float(r['price']) for r in schedules['04']]); t=np.arange(144)/6
peak=int(p.argmax())
names_cn={'03':'峰谷策略','04':'线性规划','10':'光伏预留策略'}
colors={'03':'#b65a32','04':'#1f66a5','10':'#32805a'}
styles={'03':'--','04':'-','10':':'}
fig=plt.figure(figsize=(8.2,9.1))
gs=fig.add_gridspec(4,1,height_ratios=[1,.21,1,1],left=.115,right=.975,bottom=.10,top=.94,hspace=.18)
axs=[fig.add_subplot(gs[r]) for r in (0,2,3)]
legend_ax=fig.add_subplot(gs[1]); legend_ax.axis('off')
axs[0].step(np.r_[t,24],np.r_[p,p[-1]],where='post',color='#444444',lw=1.5)
axs[0].set_ylabel('电价(元/kWh)'); axs[0].set_title('同一天，不同策略怎样安排电池和购电')
axs[0].axvline(t[peak],color='#999999',lw=.8,ls=':')
axs[0].annotate('当天最高价',xy=(t[peak],p[peak]),xytext=(16,p[peak]+.12),
    arrowprops={'arrowstyle':'->','color':'black'},fontsize=10)
axs[0].set_ylim(0,max(p)+.3)
for c in ['03','10','04']:
    rr=schedules[c]; ss=[float(rr[0]['state_start'])]+[float(r['state_end']) for r in rr]
    axs[1].plot(np.arange(145)/6,ss,label=names_cn[c],color=colors[c],ls=styles[c],lw=1.6)
    qq=np.array([float(r['purchase_kwh']) for r in rr])*6
    axs[2].step(np.r_[t,24],np.r_[qq,qq[-1]],where='post',label=names_cn[c],color=colors[c],ls=styles[c],lw=1.25)
for bound in [1200,10800]: axs[1].axhline(bound,color='#888888',ls=':',lw=.8)
axs[1].set_ylim(0,12000); axs[1].set_yticks([1200,6000,10800]); axs[1].set_ylabel('内部储电量(kWh)')
handles,labels=axs[1].get_legend_handles_labels()
legend_ax.legend(handles,labels,ncol=3,loc='center',fontsize=11); axs[2].set_ylabel('外网购电功率(kW)'); axs[2].set_xlabel('时刻(时)')
for ax in axs:
    ax.set_xlim(0,24); ax.set_xticks(range(0,25,2)); ax.grid(axis='y',alpha=.18)
fig.text(.06,.022,'同一代表日;两种规则各选27组参数中费用最低者。首尾均为6000 kWh，充放效率各90%。',fontsize=9)
for axis in axs[:2]: axis.tick_params(labelbottom=False)
save(fig,'03_Q1_daily_dispatch')
```

D.16 问题二灵敏度图

文件:code/fig_q2_sensitivity.py

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
ROOT=Path(__file__).resolve().parents[1]
B=ROOT/'figure_inputs/q2'
out=ROOT/'reproduced/plots/fig';out.mkdir(parents=True,exist_ok=True)
read=lambda p:pd.read_csv(p)
def save(fig,name):
```

```
 for ext in ('png','pdf'):fig.savefig(out/f'{name}.{ext}',dpi=300,bbox_inches='tight')
 plt.close(fig)
plt.rcParams.update({'font.family':'SimSun', 'font.size':10,
    'axes.unicode_minus':False,
    'text.color':'black', 'axes.labelcolor':'black', 'xtick.color':'black', 'ytick.color':'black',
    'pdf.fonttype':42, 'axes.spines.top':False, 'axes.spines.right':False,
    'axes.axisbelow':True, 'savefig.facecolor':'white'})
BLUE, GREEN, ORANGE, RED = '#386a94', '#5f8768', '#c39243', '#b34e4a'
allrows=pd.concat([read(B/f'{p}/comparison.csv') for p in ['refinement','validation','supplement']]).set_index('variant')
base=allrows.loc['error-decay70']
choices=[['selected-q60','error-decay70','selected-q80'],['error-decay70','weighted-error','error-decay90']]
fig,axes=plt.subplots(1,2,figsize=(7.3,3.1),sharey=True)
sens=[]
for j,(keys,labels,title) in enumerate(zip(choices,[['0.6','0.7','0.8'],['0.7','0.8','0.9']],['分位位置 α(固定 β=0.7)','时间衰减 β(固定 α=0.7)'])):
    rows=allrows.loc[keys];xs=np.arange(3)
    ordinary=(rows.ordinary_cost-base.ordinary_cost)/1e4
    emergency=(rows.emergency_cost-base.emergency_cost)/1e4
    total=(rows.total_cost-base.total_cost)/1e4
    axes[j].bar(xs-.16,ordinary,width=.29,color=BLUE,label=' 普通电费变化')
    axes[j].bar(xs+.16,emergency,width=.29,color=ORANGE,label=' 紧急电费变化')
    axes[j].plot(xs,total,'ko-',lw=1,ms=3,label=' 总电费变化')
    axes[j].axhline(0,color='black',lw=.7)
    axes[j].set(xticks=xs,xticklabels=labels,xlabel=title,ylim=(-40,55))
    axes[j].grid(axis='y',alpha=.15)
    for x,y in zip(xs,total):axes[j].annotate(f'{y:.2f}',(x,y),xytext=(0,6),textcoords='offset points',ha='center',fontsize=8)
    for key,l in zip(keys,labels):sens.append({'parameter':'alpha' if j==0 else 'beta','value':l,'variant':key,**allrows.loc[key,['ordinary_cost','emergency_cost','total_cost']].to_dict()})
axes[0].set_ylabel(' 相对 α=β=0.7 的费用变化 / 万元')
handles,labels=axes[0].get_legend_handles_labels();fig.legend(handles,labels,loc='upper center',ncol=3,frameon=False,fontsize=9)
fig.tight_layout(rect=(0,0,1,.88));save(fig,'sensitivity')
pd.DataFrame(sens).to_csv(out/'sensitivity_input.csv',index=False)
```

D.17 问题三指定日期图

文件：code/fig_q3_days.py

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
from matplotlib import pyplot as plt,font_manager
ROOT=Path(__file__).resolve().parents[1]
FIG=ROOT/'reproduced/plots/fig';FIG.mkdir(parents=True,exist_ok=True)
z=dict(np.load(ROOT/'results/Q3.npz'))
df=pd.read_csv(ROOT/'figure_inputs/q3_daily.csv')
truth=np.load(ROOT/'data/inputs.npz')['actual']
font_manager.findfont('SimSun', fallback_to_default=False)
plt.rcParams.update({'font.family': ['Times New Roman','SimSun'],
    'font.size':11,
```

```
 'axes.titlesize':12, 'axes.labelsize':11, 'legend.fontsize':10,
 'axes.unicode_minus':False, 'pdf.fonttype':42, 'ps.fonttype':42,
 'text.color':'black','axes.labelcolor':'black','xtick.color':'black','ytick.color':'black',
 'axes.spines.top':False,'axes.spines.right':False,'axes.linewidth':.8,
 'savefig.facecolor':'white','figure.facecolor':'white','axes.axisbelow':True})
BLUE='#396e96'; GOLD='#bf913c'; GREEN='#62896c'; GRAY='#888888'
outputs=[]
def save(fig, name):
    for ext in ('pdf','png'):
        p=FIG/f'{name}.{ext}'
        kw={'metadata':{'CreationDate':None,'ModDate':None}} if ext=='pdf' else {'dpi':300}
        fig.savefig(p, bbox_inches='tight', pad_inches=.07, **kw); outputs.append(p)
    plt.close(fig)
def grid(ax): ax.grid(axis='y',color='#dddddd',lw=.5)

# Q3 monthly charges: sum raw daily bills, without selecting favorable months.
monthly=df.assign(month=pd.to_datetime(df.date).dt.month).groupby('month').sum(numeric_only=True)
fig,ax=plt.subplots(figsize=(9.1,3.3), layout='constrained')
x=monthly.index.to_numpy()
ax.bar(x,monthly.ordinary_cost/1e4,.63,color=BLUE,label=' 普通结算')
ax.bar(x,monthly.emergency_cost/1e4,.63,bottom=monthly.ordinary_cost/1e4,color=GOLD,label='紧急购电')
ax.set(xlabel=' 月份',ylabel=' 月度费用 / 万元',xticks=x,ylim=(0,180))
ax.legend(loc='upper left',frameon=False,ncol=2);grid(ax)
save(fig,'q3_final_monthly')

# Q3 10-minute physical curves. Arrays are chronological: slot k starts k/6 h.
# Storage arrays refer to edges; energy orders are multiplied by six for kW.
fig,axes=plt.subplots(3,2,figsize=(10,7.3),sharex=True,sharey='row')
fig.subplots_adjust(left=.08,right=.985,bottom=.13,top=.9,hspace=.48,wspace=.17)
hours=np.arange(144)/6; edges=np.arange(145)/6
handles0=None;handles2=None
for col,date in enumerate(('2025-03-20','2025-09-23')):
    i=int(df.index[df.date==date][0]); day=int(z['days'][i]); ex=z['execution'][i]
    actual=truth[day*144:(day+1)*144]
    q0=z['q0'][i]; final=z['versions'][i,-1]
    ax=axes[0,col]
    l1,=ax.plot(hours,actual[:,0],color='black',lw=1.05,label=' 实际负载')
    l2,=ax.plot(hours,actual[:,1],color=GREEN,lw=1.05,label=' 实际光伏')
    l3,=ax.step(hours,q0*6,where='post',color=GRAY,lw=.9,ls='--',label='零点订单')
    l4,=ax.step(hours,final*6,where='post',color=BLUE,lw=1.05,label='最终订单')
    handles0=[l1,l2,l3,l4]
    ax.set_title('3 月 20 日' if col==0 else '9 月 23 日',pad=9)
    ax.set_ylim(0,14500);grid(ax)
    ax=axes[1,col]
    ax.plot(edges,np.r_[ex[0,0],ex[:,1]],color=BLUE,lw=1.2)
    for limit in (1200,10800): ax.axhline(limit,color=GRAY,ls='--',lw=.7)
    ax.set(ylim=(0,11600),yticks=[1200,6000,10800]);grid(ax)
```

```
    ax=axes[2,col]
    deficit=np.maximum((actual[:,0]-actual[:,1])/6-final,0)
    f=ax.fill_between(hours,deficit,step='post',color='#d9d9d9',label='订单外缺口')
    d,=ax.step(hours,ex[:,7],where='post',color=BLUE,lw=1.1,label='电池补充')
    e=ax.bar(hours,ex[:,8],width=1/6,align='edge',color=GOLD,label='紧急购电')
    handles2=[f,d,e];ax.set(xlabel='时刻 / h',xlim=(0,24),xticks=[0,6,12,18,24],ylim=(0,1000));grid(ax)
axes[0,0].set_ylabel('功率 / kW');axes[1,0].set_ylabel('储电量 / kWh');axes[2,0].set_ylabel('每时段补缺电量 / kWh')
fig.legend(handles0,[h.get_label() for h in handles0],loc='upper center',bbox_to_anchor=(.53,.99),ncol=4,frameon=False)
fig.legend(handles2,['订单外缺口','电池补充','紧急购电'],loc='lower center',bbox_to_anchor=(.53,.015),ncol=3,frameon=False)
save(fig,'q3_final_days')
```

D.18 问题三参数比较图

文件：code/fig_q3_quantile.py

```
"""Redraw the approved Q3 quantile panel from existing verified results; no experiments."""
from pathlib import Path
import json, hashlib
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

ROOT=Path(__file__).resolve().parents[1]
base=ROOT/'reproduced/plots'
(base/'fig').mkdir(parents=True,exist_ok=True)
(base/'scripts').mkdir(exist_ok=True)
source=ROOT/'figure_inputs/derived_values.json'
rows=sorted(json.loads(source.read_text('utf-8'))['q3_quantile_comparison'],key=lambda r:r['q'])
plt.rcParams.update({'font.family':['Times New Roman','SimSun'],'font.size':11,
                     'axes.unicode_minus':False,'pdf.fonttype':42,'ps.fonttype':42})
q=[r['q'] for r in rows]
total=[r['total_cost']/1e4 for r in rows]
ordinary=[r['ordinary_cost']/1e4 for r in rows]
fig,ax=plt.subplots(figsize=(6.8,3.5),layout='constrained')
ax.plot(q,total,'o-',color='#396e96',label='总费用',lw=1.15)
ax.plot(q,ordinary,'s--',color='#bf913c',label='普通结算',lw=1.05)
ax.fill_between(q,ordinary,total,color='#bf913c',alpha=.16,label='紧急购电费')
ax.annotate(f'{total[2]:.2f}',(.6,total[2]),xytext=(0,9),textcoords='offset points',ha='center',fontsize=10)
ax.set(xticks=q,xlabel='误差准备分位',ylabel='评价期费用 / 万元',ylim=(1267,1342))
ax.legend(loc='upper center',ncol=3,frameon=False,fontsize=10,columnspacing=1,handlelength=1.5)
ax.grid(axis='y',color='#dddddd',linewidth=.5)
ax.set_axisbelow(True)
ax.spines[['top','right']].set_visible(False)
for ext in ('pdf','png'):
    kw={'metadata':{'CreationDate':None,'ModDate':None}} if ext=='pdf' else {'dpi':300}
```

```
    fig.savefig(base/f'fig/q3_final_quantile.{ext}',bbox_inches='tight',**kw)
plt.close(fig)
(base/'scripts/q3_quantile_manifest.json').write_text(json.dumps({
    'source':'scripts/derived_values.json','source_sha256':hashlib.sha256(so
    ↪ urce.read_bytes()).hexdigest(),
    'data_key':'q3_quantile_comparison','script':'scripts/draw_q3_quantile.p
    ↪ y',
    'outputs':['fig/q3_final_quantile.pdf','fig/q3_final_quantile.png'],
    'scope':'Existing result visualization only; no new modeling or
    ↪ validation.'},ensure_ascii=False,indent=2),'utf-8')
print('Q3 quantile PDF and PNG generated from existing result data.')
```

D.19 电量关系示意图

文件：code/energy_paths.tex

```
\documentclass{standalone}
\usepackage[UTF8]{ctex}
\usepackage{tikz}
\usetikzlibrary{arrows.meta,positioning}
\begin{document}
% Q1 energy-path definition: report 12.2, 15; external-side kWh.
\begin{tikzpicture}[>=Stealth, node distance=16mm, every
↪ node/.style={font=\small},
  box/.style={draw=black,rounded corners=2pt,minimum width=23mm,minimum
  ↪ height=10mm,align=center},
  lab/.style={fill=white,inner sep=2pt}]
\node[box] (grid) at (0,1.25) {外部电网};
\node[box] (pv) at (0,-1.25) {光伏发电};
\node[box] (battery) at (5,0) {储能设备\\$S_{k-1}\longrightarrow S_k$};
\node[box] (load) at (10,1.25) {小区负载};
\node[box] (waste) at (10,-1.25) {未利用光伏};
\draw[->] (grid.east) -- node[lab,above] {$n_k-z_k$} (load.west);
\draw[->] (grid.south east) -- node[lab,above,sloped] {$x_k$} (battery.north
↪ west);
\draw[->] (pv.north east) -- node[lab,above,sloped] {$y_k$} (battery.south
↪ west);
\draw[->] (battery.north east) -- node[lab,below,sloped] {$z_k$} (load.south
↪ west);
\draw[->] (pv.south) -- (0,-2.65) -- node[lab,below] {$m_k$:光伏直供}
↪ (11.9,-2.65) -- (11.9,1.25) -- (load.east);
\draw[->] (pv.east) -- node[lab,pos=.70,above] {$w_k=r_k-y_k$} (waste.west);
\end{tikzpicture}

\end{document}
```

D.20 四问总体框架图

文件：code/framework.tex

```
\documentclass{article}
\usepackage[paperwidth=198mm,paperheight=154mm,margin=3mm]{geometry}
\usepackage[UTF8]{ctex}
\usepackage{fontspec,amsmath,tikz}
\setmainfont{Times New Roman}
\setCJKmainfont{SimSun}[BoldFont=SimHei]
\usetikzlibrary{arrows.meta}
```

```
\pagestyle{empty}
\setlength{\parindent}{0pt}
\linespread{1.0}\selectfont
\begin{document}
\centering
\begin{tikzpicture}[x=1cm,y=1cm,
 every node/.style={font=\fontsize{10}{13}\selectfont,align=center,inner sep=3pt,text=black},
 panel/.style={draw=black,line width=.9pt,fill=white},
 box/.style={draw=black,line width=.6pt,fill=white,minimum height=1cm,text width=3.7cm,font=\fontsize{10}{13}\selectfont\bfseries},
 core/.style={box,line width=1.15pt},
 arr/.style={-{Stealth[length=1.7mm,width=1.3mm]},line width=.7pt},
 head/.style={font=\fontsize{11}{14}\selectfont\bfseries,anchor=west},
 note/.style={font=\fontsize{9}{12}\selectfont}]
\path[use as bounding box] (0,.3) rectangle (19.1,14.9);

% Labels remaining after the five requested screenshot deletions are centered.
\draw[panel] (0,12.55) rectangle (19.1,14.9);
\draw[panel] (0,8.95) rectangle (19.1,11.85);
\draw[panel] (0,5.35) rectangle (19.1,8.25);
\draw[panel] (0,.3) rectangle (19.1,4.65);
\foreach \yt in {14.9,11.85,8.25,4.65}{\draw[line width=.5pt] (0,\yt-.72)--(19.1,\yt-.72);}

\node[head] at (.27,14.53) {问题一\quad 全时段电量流线性规划};
\node[box] (a1) at (2.35,13.37) {代表日供需与电价};
\node[box] (a2) at (7.12,13.37) {用电缺口与光伏余量};
\node[core] (a3) at (11.91,13.37) {全时段线性规划};
\node[box] (a4) at (16.7,13.37) {当天最低购电费};
\foreach \from/\to in {a1/a2,a2/a3,a3/a4}{\draw[arr] (\from.east)--(\to.west);}

\draw[arr] (9.55,12.55)--(9.55,11.85);
\node[note,fill=white,anchor=east] at (9.30,12.20) {继承供需与储能约束};
\node[note,fill=white,anchor=west] at (9.80,12.20) {新增:供需未知、跨日运行与紧急购电};

\node[head] at (.27,11.48) {问题二\quad 历史分项预测—近期加权准备—跨日线性规划};
\node[box] (b1) at (2.35,10.30) {零点可用信息};
\node[box] (b2) at (7.12,10.30) {负载、光伏分别预测};
\node[box] (b3) at (11.91,10.30) {近期加权误差准备};
\node[core] (b4) at (16.7,10.30) {跨日线性规划};
\foreach \from/\to in {b1/b2,b2/b3,b3/b4}{\draw[arr] (\from.east)--(\to.west);}
\node[draw,line width=.5pt,text width=17.8cm,minimum height=.57cm,font=\fontsize{10}{13}\selectfont\bfseries] (bex) at (9.55,9.40) {
在供需不确定时,降低普通购电与紧急购电的总费用};
\draw[arr] (b4.east)--(18.9,10.30)--(18.9,9.40)--(bex.east);

\draw[arr] (9.55,8.95)--(9.55,8.25);
\node[note,fill=white,anchor=east] at (9.30,8.60) {继承负载预测、误差准备与跨日储能};
\node[note,fill=white,anchor=west] at (9.80,8.60) {新增:光伏预报更新、有代价的改单};

```

```
\node[head] at (.27,7.88) {问题三\quad 融合预测与日内校正滚动规划};
\node[box] (c1) at (2.35,6.70) {0、6、12、18 点新预报};
\node[box] (c2) at (7.12,6.70) {负载校正与光伏融合};
\node[box] (c3) at (11.91,6.70) {近期误差准备};
\node[core] (c4) at (16.7,6.70) {日内滚动线性规划};
\foreach \from/\to in {c1/c2,c2/c3,c3/c4}{\draw[arr] (\from.east)--(\to.west);}
\node[draw,line width=.5pt,text width=17.8cm,minimum height=.57cm,font=\fontsize{10}{13}\selectfont\bfseries] (cex) at (9.55,5.80) {
降低普通、调整与紧急购电的总费用，并评价日内光伏预报的价值};
\draw[arr] (c4.east)--(18.9,6.70)--(18.9,5.80)--(cex.east);

\draw[arr] (9.55,5.35)--(9.55,4.65);
\node[note,fill=white,anchor=east] at (9.30,5.0) {分别扩展问题二、问题三的调度结构};
\node[note,fill=white,anchor=west] at (9.80,5.0) {新增：未来电价未知};

\node[head,font=\fontsize{10.5}{14}\selectfont\bfseries] at (.27,4.28) {问题四\quad 梯度提升树—零点线性规划 / 岭回归—日内调整线性规划};
\node[anchor=west,font=\fontsize{10}{13}\selectfont\bfseries] at (.27,3.59) {问题四-2：沿用问题二的决策权限};
\node[box] (d1) at (2.35,2.86) {零点以前的历史信息};
\node[core] (d2) at (7.12,2.86) {梯度提升树预测电价};
\node[core] (d3) at (11.91,2.86) {零点线性规划};
\node[box] (d4) at (16.7,2.86) {当天固定购电计划};
\foreach \from/\to in {d1/d2,d2/d3,d3/d4}{\draw[arr] (\from.east)--(\to.west);}
\draw[dashed,line width=.5pt] (.27,2.17)--(18.83,2.17);
\node[anchor=west,font=\fontsize{10}{13}\selectfont\bfseries] at (.27,1.88) {问题四-3：沿用问题三的决策权限};
\node[box] (e1) at (2.35,1.15) {历史与当日新增信息};
\node[core] (e2) at (7.12,1.15) {岭回归预测电价};
\node[core] (e3) at (11.91,1.15) {日内调整线性规划};
\node[box] (e4) at (16.7,1.15) {零点计划与日内调整};
\foreach \from/\to in {e1/e2,e2/e3,e3/e4}{\draw[arr] (\from.east)--(\to.west);}
\end{tikzpicture}
\end{document}

```